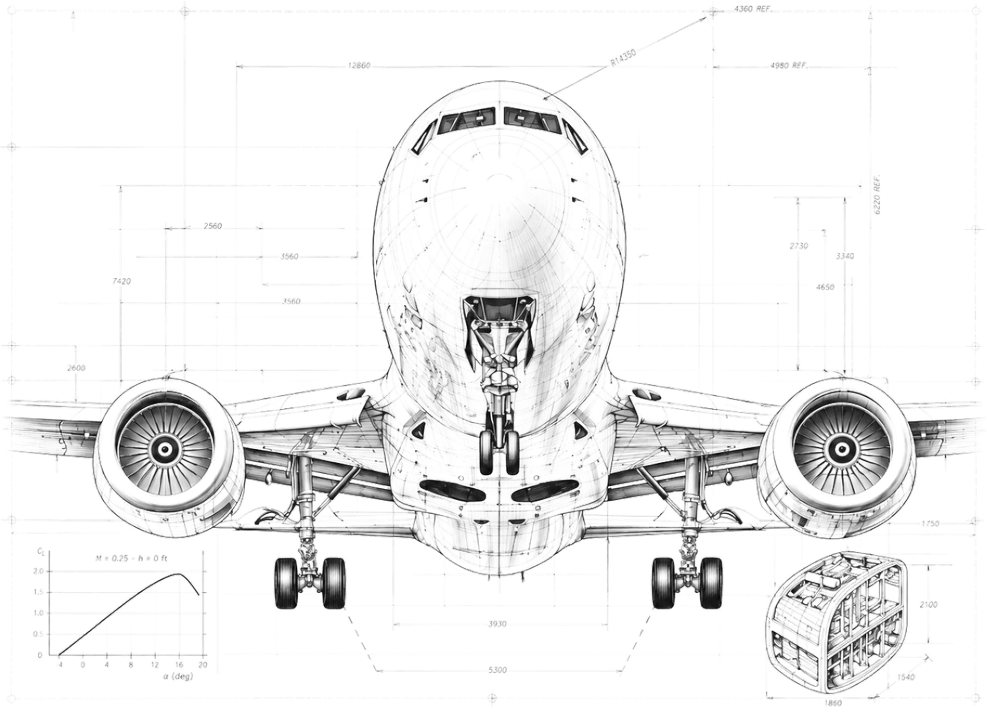
Costruzioni Aeronautiche

ITIS “E. Majorana” — Fluidodinamica numerica

# Problemi ai valori iniziali in fluidodinamica

## Moti governati da equazioni differenziali ordinarie: dalla caduta di una sfera alla scia vorticosa di un’ala




Monografia didattica di fluidodinamica numerica
Modulo 1: integrazione nel tempo delle equazioni del moto

**Prof. Ing. Biagio Raucci**

Esempi numerici in Python (NumPy, SciPy, Matplotlib)

# Indice

**Presentazione** **v**

**1 Metodi numerici per i problemi ai valori iniziali** **1**
1.1 Il problema ai valori iniziali . . . 1
1.2 Discretizzazione dell'asse dei tempi e serie di Taylor . . . 2
1.3 Errore di troncamento e metodo di Eulero . . . 3
1.4 I metodi di Runge–Kutta . . . 4
1.4.1 L'idea . . . 4
1.4.2 Il secondo ordine: dimostrazione . . . 4
1.4.3 Il quarto ordine classico . . . 5
1.5 Verifica sperimentale dell'ordine . . . 6
1.6 Stabilità assoluta: quando il passo è troppo grande . . . 7
1.7 Sistemi di equazioni ed equazioni di ordine superiore . . . 9
1.8 Passo adattivo: `solve_ivp` come analogo di `ODE45` . . . 12
1.9 Sintesi del capitolo . . . 12

**2 Caduta libera di una sfera in un fluido** **14**
2.1 L'esperimento di Galileo rifatto al computer . . . 14
2.2 Le forze su un corpo che si muove in un fluido . . . 14
2.3 La curva $c_d(Re)$ della sfera e la sua approssimazione . . . 16
2.4 L'equazione del moto . . . 17
2.5 Programma 1.1 . . . 18
2.6 La velocità terminale . . . 21
2.7 Problemi svolti . . . 22
2.7.1 Problema 2.1: la pallina da ping-pong in acqua . . . 22
2.7.2 Problema 2.2: la velocità terminale non dipende dalla velocità iniziale . . . 23
2.7.3 Le gocce di pioggia . . . 23
2.8 Sintesi del capitolo . . . 24

**3 Simulazione di moti vincolati: pendolo e ala su molla** **25**
3.1 La simulazione numerica come esperimento . . . 25
3.2 Il pendolo semplice in un fluido . . . 25

3.2.1 Il modello . . . 25
3.2.2 Programma 1.2 . . . 27
3.3 Un sistema aeroelastico elementare: l'ala su molla . . . 29
3.3.1 Il modello . . . 29
3.3.2 Adimensionalizzazione . . . 30
3.3.3 Programma 1.3 e risultati . . . 31
3.3.4 Risposta alle raffiche . . . 34
3.4 Sintesi del capitolo . . . 35

**4 Moti bidimensionali e balistica di un proiettile sferico 36**
4.1 Runge–Kutta per il moto piano di un corpo in un fluido . . . 36
4.2 Balistica di un proiettile sferico . . . 37
4.3 Il calcolo della gittata . . . 40
4.4 La gittata massima: il metodo del dimezzamento dell'intervallo . . . 41
4.4.1 L'angolo ottimo in funzione del diametro . . . 43
4.4.2 Problema 4.3: il proietto d'artiglieria e l'atmosfera stratificata . . . 44
4.5 Sintesi del capitolo . . . 46

**5 La traiettoria di un aliante 47**
5.1 Le equazioni del volo planato . . . 47
5.1.1 Adimensionalizzazione . . . 47
5.2 Problema 5.1: volo verticale di un razzo . . . 51
5.3 Sintesi del capitolo . . . 53

**6 Avvolgimento della scia vorticosa dietro un'ala finita 55**
6.1 Dal profilo all'ala finita: il sistema di vortici . . . 55
6.2 Il foglio di scia non resta piano . . . 55
6.3 Discretizzazione in vortici puntiformi . . . 56
6.3.1 Forma adimensionale e condizione sul passo . . . 58
6.4 Programma 1.7 e risultati . . . 59
6.5 Applicazione: la scia di un Boeing 747 . . . 63
6.6 Possibili miglioramenti . . . 63
6.7 Sintesi del capitolo . . . 64

**A Simboli, listati di supporto e istruzioni 65**
A.1 Corrispondenza fra simboli matematici e variabili dei programmi . . . 65
A.2 Listati di supporto . . . 65
A.3 Come riprodurre i risultati . . . 72
A.4 Codici in rete e QR code . . . 72
A.5 Come sostituire le figure . . . 73

**B Esercizi svolti 75**

B.1 Esercizio 1 – I metodi di Heun e di Ralston . . . . . 75
B.2 Esercizio 2 – Il limite di stabilità di RK4 . . . . . 77
B.3 Esercizio 3 – L'errore di arrotondamento . . . . . 78
B.4 Esercizio 4 – Sfera d'acciaio in acqua . . . . . 79
B.5 Esercizio 5 – La goccia che gela . . . . . 80
B.6 Esercizio 6 – Caduta da 20 km in atmosfera stratificata . . . . . 80
B.7 Esercizio 7 – Il guscio pieno d'aria in acqua . . . . . 82
B.8 Esercizio 8 – La linea elettrica nel vento oscillante . . . . . 83
B.9 Esercizio 9 – Il vento critico dell'ala su molla . . . . . 84
B.10 Esercizio 10 – La frequenza di massima risposta alle raffiche . . . . . 85
B.11 Esercizio 11 – L'effetto Magnus . . . . . 86
B.12 Esercizio 12 – Colpire un bersaglio a distanza assegnata . . . . . 87
B.13 Esercizio 13 – Aliante con richiamata . . . . . 89
B.14 Esercizio 14 – Il razzo e il nodo allineato allo spegnimento . . . . . 89
B.15 Esercizio 15 – Vortex blob . . . . . 90
B.16 Esercizio 16 – Convergenza in $m$ . . . . . 91
B.17 Esercizio 17 – Vortici di uguale intensità . . . . . 92
B.18 Listato completo . . . . . 93

**Bibliografia** **103**

# Elenco dei listati


1.1 Verifica dell'ordine di accuratezza (estratto da `es01_ordine_stabilita.py`). 6

1.2 Il modulo `ode_solvers.py`: integratori a un passo in forma vettoriale e curva $c_d(Re)$ della sfera. . . . . 10

1.3 Uso di `solve_ivp` (RK45 adattivo) per un sistema del secondo ordine. . . . . 12

2.1 Programma 1.1 – caduta libera di una sfera (`prog1_1_caduta_sfera.py`). . . 18

3.1 Programma 1.2 – pendolo in un fluido (`prog1_2_pendolo.py`). . . . . 27

3.2 Programma 1.3 – ala su molla in galleria del vento (`prog1_3_ala_molla.py`). 31

4.1 Programma 1.4 – traiettoria di un proiettile sferico (`prog1_4_proiettile.py`). 38

4.2 Programma 1.5 – funzione `gittata_massima` . . . . . 42

4.3 Problema 4.3 – modello del proietto . . . . . 44

5.1 Programma 1.6 – traiettorie di un aliante a portanza e resistenza costanti (`prog1_6_aliante.py`). . . . . 48

5.2 Problema 5.1 – razzo a massa variabile, RK2 e confronto con `solve_ivp` (`prog1_9_razzo.py`). . . . . 52

6.1 Programma 1.7 – avvolgimento di un foglio vorticoso . . . . . 59

A.1 `stile_grafici.py`: impostazioni comuni di Matplotlib. . . . . 65

A.2 `es01_ordine_stabilita.py` (versione completa) . . . . . 67

A.3 `prog1_1b_problemi.py`: problemi collegati al Programma 1.1 (Cap. 2). . . . . 69

A.4 `prog1_5_gittata_max.py`: Programma 1.5 completo (Cap. 4). . . . . 70

A.5 `prog1_8_cannone.py`: Problema 4.3 completo (Cap. 4). . . . . 71

B.1 `esercizi_svolti.py` (esercizi 1–17) . . . . . 93

# Presentazione

## Di che cosa parla questa monografia

Molti problemi di meccanica del volo e di fluidodinamica non riguardano *campi* (velocità e pressione in ogni punto di una regione), ma *corpi*: una sfera che cade nell'aria, un pendolo che oscilla in acqua, un proiettile, un aliante, un filamento vorticoso che si avvolge dietro un'ala. In tutti questi casi l'incognita è una funzione del solo tempo e la fisica si riduce alla seconda legge di Newton, cioè a un sistema di *equazioni differenziali ordinarie* (ODE, *ordinary differential equations*) corredato dalle condizioni iniziali: posizione e velocità all'istante $t_0$. Si parla allora di **problema ai valori iniziali**.

La difficoltà non sta nella forma dell'equazione, che è sempre $m\,\mathrm{d}\mathbf{v}/\mathrm{d}t = \mathbf{F}$, ma nel fatto che le forze fluidodinamiche dipendono in modo non lineare dalla velocità (resistenza proporzionale a $v|v|$, coefficiente di resistenza che cambia con il numero di Reynolds, portanza che si annulla allo stallo). Non esiste, salvo casi banali, una soluzione in forma chiusa: l'unica strada è *integrare numericamente* le equazioni, marciando nel tempo a piccoli passi. È esattamente quello che fa un simulatore di volo, un codice di balistica o il modulo di traiettoria di un lanciatore.

Questa monografia costituisce il primo modulo del corso di fluidodinamica numerica: prima di affrontare i campi (equazioni alle derivate parziali, griglie, condizioni al contorno) si impara a integrare nel tempo, su problemi di meccanica del volo che hanno un significato fisico immediato e una risposta verificabile. I temi sono organizzati in sette esempi classici della letteratura, qui rielaborati con dimostrazioni complete, programmi in Python corredati di grafici ed esercizi interamente svolti. La bibliografia in fondo raccoglie le fonti di riferimento: fra queste, Chapra e Canale [3] per la teoria dei metodi di Runge–Kutta, dell'errore di troncamento e della stabilità; Ferziger e Perić [4] per il legame fra integrazione nel tempo e metodi per le equazioni alle derivate parziali; LeVeque [5] per i concetti di consistenza e convergenza; i testi classici di meccanica dei fluidi computazionale [1, 2] per gli esempi applicativi.

## Come è organizzata

**Capitolo 1 – Metodi numerici per i problemi ai valori iniziali.**
Serie di Taylor, metodo di Eulero, metodi di Runge–Kutta del secondo e quarto ordine, estensione ai sistemi e alle equazioni del secondo ordine. Si dimostra l'ordine di accuratezza, si introduce la stabilità assoluta e si verifica tutto con esperimenti numerici.

**Capitolo 2 – Caduta libera di una sfera.**
Il primo esperimento numerico: le forze fluidodinamiche su un corpo in moto (galleggiamento, massa aggiunta, resistenza viscosa), la curva $c_d(Re)$ della sfera, la velocità terminale, l'esperimento di Galileo rifatto al computer.

**Capitolo 3 – Moti vincolati.**
Il pendolo in un fluido e un'ala sospesa a una molla in galleria del vento: il primo modello aeroelastico, con l'adimensionalizzazione delle equazioni e la risposta alle raffiche.

**Capitolo 4 – Moti bidimensionali e balistica.**
Runge–Kutta per sistemi di equazioni del secondo ordine, traiettorie di un proiettile sferico con e senza vento, calcolo della gittata e ricerca dell'angolo ottimo con il metodo del dimezzamento dell'intervallo.

**Capitolo 5 – Traiettoria di un aliante.**
Equazioni del volo planato in forma adimensionale, moto fugoide, loop; come complemento, il volo verticale di un razzo a massa variabile.

**Capitolo 6 – Avvolgimento della scia vorticosa.**
Dal teorema di Kutta–Joukowski al vortice a ferro di cavallo; il foglio vorticoso dietro un'ala a carico ellittico, discretizzato in vortici puntiformi e fatto evolvere nel tempo: un problema a $N$ corpi della fluidodinamica.

**Appendice.**
Listati completi dei moduli condivisi, tabella dei simboli dei programmi, esercizi proposti.

Ogni capitolo segue lo stesso schema: prima l'*intuizione fisica* del fenomeno, poi il *modello matematico*, quindi l'*algoritmo* e il *programma*, infine i *risultati* discussi criticamente. I riquadri colorati hanno un significato fisso:

- **teoria** (viola): definizioni, enunciati, dimostrazioni;
- **fasi** (blu): procedure e algoritmi passo per passo;
- **ipotesi** (giallo): le assunzioni semplificative del modello;
- **dati** (grigio): valori numerici usati nei calcoli;
- **esempi** (arancio): applicazioni pratiche e analogie aeronautiche;
- **risultati** (verde): i numeri ottenuti e il loro significato;
- **avvisi** (rosso): errori frequenti e limiti del modello.

## I programmi Python

Tutti i programmi sono scritti in Python 3 con le librerie `numpy` (vettori e algebra), `scipy` (integratori adattivi e funzioni speciali) e `matplotlib` (grafici). Il codice è stato scritto privilegiando la leggibilità: le formule di Runge–Kutta compaiono nel programma esattamente come nel testo, con gli stessi nomi dei simboli. Nella letteratura classica gli stessi esempi sono trattati in FORTRAN o MATLAB; la traduzione in Python, con NumPy, rende le formule vettoriali ancora più vicine alla notazione matematica.

Tutti i programmi sono disponibili in rete all'indirizzo `https://braucci.github.io/CFD/monografia1/codici/`. Accanto a ogni listato, nel margine destro, un codice QR rimanda al file corrispondente: si inquadra con la fotocamera del telefono oppure, nella versione PDF, si clicca direttamente.

codici/

### Struttura della cartella `codice/`

**`ode_solvers.py`** i tre integratori (Eulero, RK2, RK4), la funzione di marcia `integra` e la curva $c_d(Re)$ della sfera; è il modulo importato da tutti gli altri.

**`stile_grafici.py`** impostazioni comuni di Matplotlib (palette della casa, formato delle figure).

**`es01_ordine_stabilita.py`** esperimenti sull'ordine di accuratezza e sulla stabilità (Cap. 1).

**`prog1_1_caduta_sfera.py`, `prog1_1b_problemi.py`** Programma 1.1 e problemi collegati (Cap. 2).

**`prog1_2_pendolo.py`, `prog1_3_ala_molla.py`** Programmi 1.2 e 1.3 (Cap. 3).

**`prog1_4_proiettile.py`, `prog1_5_gittata_max.py`** Programmi 1.4 e 1.5 (Cap. 4).

**`prog1_6_aliante.py`, `prog1_9_razzo.py`** Programma 1.6 e Problema 5.1 (Cap. 5).

**`prog1_7_scia_vorticosa.py`** Programma 1.7 (Cap. 6).

Ogni script si esegue dalla cartella `codice/` con `python3 nome_script.py`; stampa a video le tabelle riportate nel testo e salva le figure in formato PDF nella cartella `figure/`, da cui il documento LaTeX le preleva. Le figure della monografia sono quindi *riproducibili*: modificando un parametro e rilanciando lo script si rigenerano.

### Prerequisiti

Si danno per note le nozioni di base di meccanica (seconda legge di Newton, moto del proiettile nel vuoto), di analisi (derivate, sviluppo in serie di Taylor, integrali) e di aerodinamica elementare (numero di Reynolds, coefficienti di portanza e di resistenza, teorema di Kutta–Joukowski). Per Python è sufficiente la dimestichezza acquisita nel modulo introduttivo del corso (variabili, cicli, funzioni, array NumPy, grafici con Matplotlib).

# Capitolo 1

# Metodi numerici per i problemi ai valori iniziali

## 1.1 Il problema ai valori iniziali

Consideriamo l'equazione differenziale ordinaria del primo ordine nella forma più generale

$$\frac{\mathrm{d}x}{\mathrm{d}t} = f(x,t), \tag{1.1}$$

dove $f$ è una funzione nota e sufficientemente regolare (*analitica*, cioè sviluppabile in serie di Taylor). Se all'istante iniziale $t = t_0$ è assegnato il valore $x(t_0) = x_0$, cerchiamo la funzione $x(t)$ che per $t > t_0$ soddisfi sia la (1.1) sia la condizione iniziale. Questo è un **problema ai valori iniziali** (in inglese *initial-value problem*, IVP).

**L'esempio più semplice: un corpo lanciato verso l'alto**

Un corpo lanciato verticalmente nel vuoto con velocità $v_0$ obbedisce a $\mathrm{d}v/\mathrm{d}t = -g$, con $v(0) = v_0$. Qui $f(v,t) = -g$ è costante e la soluzione è immediata: $v(t) = v_0 - gt$. Se però il corpo si muove nell'aria, alla gravità si somma la resistenza aerodinamica, proporzionale a $v|v|$ e diretta in verso opposto alla velocità:

$$\frac{\mathrm{d}v}{\mathrm{d}t} = -g - k\,v|v|, \qquad k = \frac{\rho_f c_d S}{2m}.$$

L'equazione resta del primo ordine, ma $f$ è ora non lineare e, se $c_d$ dipende dal numero di Reynolds (e quindi da $v$), non esiste alcuna formula chiusa per $v(t)$. È la situazione tipica dei problemi di questo capitolo, e il motivo per cui servono i metodi numerici.

**Esistenza e unicità (cenno)**

Il teorema di Cauchy–Lipschitz (o di Picard–Lindelöf) garantisce che, se $f$ è continua in $t$ e lipschitziana in $x$ in un intorno del punto $(x_0, t_0)$, cioè se esiste una costante $L$ tale che $|f(x_1,t) - f(x_2,t)| \le L|x_1 - x_2|$, allora il problema (1.1) ammette una e una sola soluzione in un intervallo attorno a $t_0$. Tutte le funzioni $f$ incontrate nel capitolo (polinomi in $v$, prodotti del tipo $v|v|$, seni, radici di somme di quadrati) soddisfano questa condizione lontano dai punti singolari. La costante $L$, che misura quanto rapidamente $f$ varia con $x$, tornerà utile nello studio della stabilità: più $L$ è grande, più il problema è "rigido" e più piccolo dovrà essere il passo di integrazione.

## 1.2 Discretizzazione dell'asse dei tempi e serie di Taylor

Per risolvere numericamente il problema, l'asse della variabile indipendente viene suddiviso in intervalli di ampiezza costante $h$ (il *passo*), i cui estremi sono

$$t_i = t_0 + i\,h, \qquad i = 0, 1, 2, \ldots \tag{1.2}$$

La soluzione calcolata nel punto $t_i$ si indica con $x_i$. Il metodo numerico sostituisce quindi la funzione continua $x(t)$ con la successione di valori discreti $x_0, x_1, x_2, \ldots$ (Fig. 1.1); tra un nodo e l'altro la soluzione non è definita, o al più si interpola.

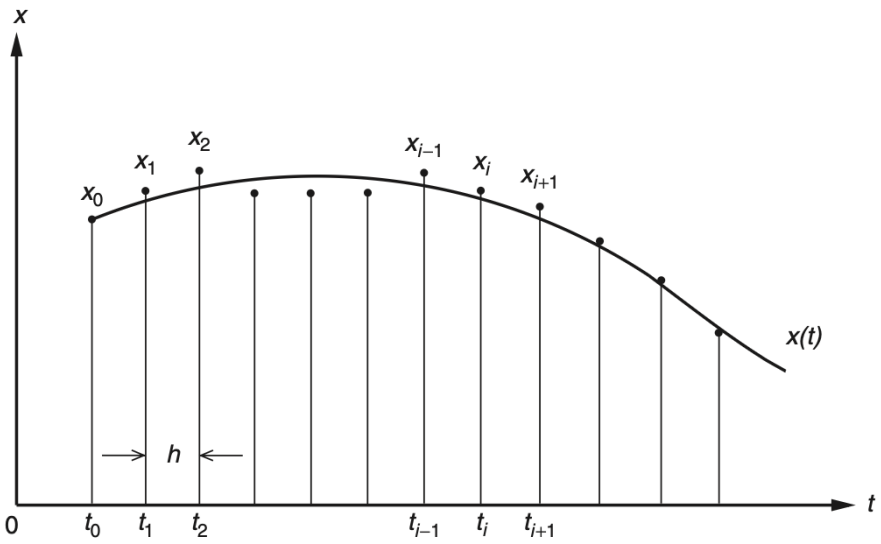



Figura 1.1: Soluzione numerica di un problema ai valori iniziali: la curva continua $x(t)$ è sostituita dai valori $x_i$ nei nodi $t_i$ equispaziati di $h$.

Poiché $h$ è piccolo e $f$ è analitica, il valore della soluzione nel nodo successivo si può esprimere con lo *sviluppo in serie di Taylor* attorno al nodo precedente:

$$x_{i+1} \equiv x(t_i + h) = x_i + h\left(\frac{\mathrm{d}x}{\mathrm{d}t}\right)_i + \frac{h^2}{2!}\left(\frac{\mathrm{d}^2x}{\mathrm{d}t^2}\right)_i + \frac{h^3}{3!}\left(\frac{\mathrm{d}^3x}{\mathrm{d}t^3}\right)_i + \cdots \tag{1.3}$$

Le derivate di $x$ si ricavano tutte dall'equazione differenziale stessa: la prima è $f_i \equiv f(x_i, t_i)$, la seconda è la derivata totale di $f$ lungo la soluzione,

$$\frac{\mathrm{d}f}{\mathrm{d}t} = \frac{\partial f}{\partial t} + \frac{\mathrm{d}x}{\mathrm{d}t}\frac{\partial f}{\partial x} = \frac{\partial f}{\partial t} + f\,\frac{\partial f}{\partial x}, \tag{1.4}$$

e così via applicando ripetutamente la regola della catena. Indicando con $f_i^{(n)}$ la derivata $n$-esima di $f$ calcolata in $(x_i, t_i)$, la (1.3) si scrive

$$x_{i+1} = x_i + \Delta x_i, \qquad \Delta x_i = h f_i + \frac{h^2}{2!}f_i' + \frac{h^3}{3!}f_i'' + \cdots \tag{1.5}$$

Partendo da $i = 0$ con $x_0$ noto, si calcola $\Delta x_0$ con il numero di termini desiderato e si ottiene $x_1$; poi, con $i = 1$, si ottiene $x_2$, e così via. Lo schema *marcia* nel tempo: ogni nuovo valore dipende solo dal precedente, e per questo si parla di *metodi a un passo*.

**Un'analogia aeronautica: la navigazione stimata**

Lo schema (1.5) è concettualmente identico alla *navigazione stimata* (*dead reckoning*) di un pilota senza GPS: nota la posizione a un certo istante, la velocità e la rotta, la posizione dopo un intervallo $h$ si stima come "posizione precedente più velocità per tempo". Se la velocità cambia durante l'intervallo (vento, virata), la stima è tanto più accurata quanto più breve è $h$ e quanto meglio si tiene conto della variazione della velocità entro l'intervallo: è esattamente il ruolo dei termini di ordine superiore della serie.

## 1.3 Errore di troncamento e metodo di Eulero

In teoria, se il numero di termini in (1.5) crescesse indefinitamente, la soluzione numerica tenderebbe a quella esatta. In pratica la serie va *troncata*, e l'errore che ne consegue si chiama **errore di troncamento**. Se si trattengono due termini, il valore calcolato di $x_{i+1}$ differisce da quello esatto per

$$\frac{h^3}{3!} f_i'' + \frac{h^4}{4!} f_i''' + \cdots,$$

e per $h$ piccolo il primo termine domina: l'errore è "dell'ordine di $h^3$", e si scrive $\mathcal{O}(h^3)$.

**Metodo di Eulero**

Trattenendo un solo termine nella (1.5) si ottiene il **metodo di Eulero**:

$$x_{i+1} = x_i + h\, f(x_i, t_i). \tag{1.6}$$

Geometricamente, da ogni nodo si avanza lungo la *tangente* alla curva soluzione (Fig. 1.2): la pendenza calcolata all'inizio dell'intervallo è usata su tutto l'intervallo. L'errore di troncamento *locale* (commesso in un singolo passo) è $\mathcal{O}(h^2)$: dimezzando $h$ l'errore di un passo si riduce a un quarto.

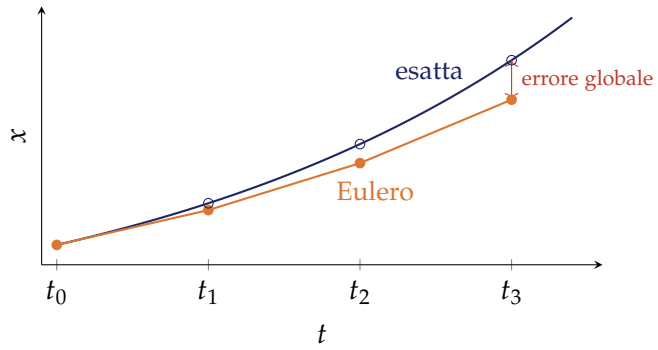



Figura 1.2: Interpretazione geometrica del metodo di Eulero: a ogni passo si procede lungo la tangente nel nodo di partenza. Gli errori locali si accumulano nell'errore globale.

**Dall'errore locale all'errore globale: perché Eulero è del primo ordine**

L'errore che interessa davvero non è quello di un singolo passo ma l'*errore globale* $E_N = |x_N - x(t_N)|$ dopo $N$ passi, cioè alla fine dell'intervallo di integrazione $[t_0, t_0 + T]$. Il ragionamento essenziale è il seguente.

1. Per raggiungere $t_0 + T$ occorrono $N = T/h$ passi.
2. In ciascun passo si introduce un errore locale $\leq Ch^2$, con $C$ proporzionale al massimo di $|x''(t)| = |f'|$.
3. Se gli errori semplicemente si sommassero, l'errore globale sarebbe $N \cdot Ch^2 = C\,T\,h$: proporzionale a $h$, non a $h^2$. Un ordine si "perde" perché il numero dei passi cresce come $1/h$.
4. In realtà l'errore commesso a un passo viene anche *propagato* dai passi successivi, amplificato al più di un fattore $(1 + hL)$ per passo, dove $L$ è la costante di Lipschitz di $f$. Dopo $N$ passi il fattore accumulato è al più $(1 + hL)^N \leq e^{LT}$, costante rispetto ad $h$. La stima rigorosa è quindi

$$E_N \leq \frac{C}{L}\left(e^{LT} - 1\right) h = \mathcal{O}(h)\,.$$

Il metodo di Eulero è dunque del **primo ordine**: dimezzando il passo l'errore finale si dimezza soltanto. Per questo il metodo non va usato se si richiede accuratezza nel risultato. Lo stesso ragionamento vale in generale: *un metodo con errore locale $\mathcal{O}\left(h^{p+1}\right)$ ha errore globale $\mathcal{O}(h^p)$ e si dice di ordine $p$.*

Il metodo della serie di Taylor con più termini è, in linea di principio, accurato quanto si vuole, ma presenta due difetti pratici che lo rendono inutilizzabile come metodo generale: richiede il calcolo analitico delle derivate di $f$ (con la (1.4) e le sue iterate, rapidamente ingestibili se $f$ è complicata, come la resistenza $v|v|c_d(Re)$), e non consente di scrivere un programma *generale*, perché le derivate cambiano da problema a problema. I metodi di Runge–Kutta superano entrambe le difficoltà.

## 1.4 I metodi di Runge–Kutta

### 1.4.1 L'idea

L'idea dei metodi di Runge–Kutta è di ottenere la stessa accuratezza della serie di Taylor con $n$ termini *valutando la sola funzione $f$* (mai le sue derivate) in punti opportunamente scelti all'interno dell'intervallo $[t_i, t_{i+1}]$, e combinando linearmente le pendenze così ottenute. In altre parole: invece di calcolare come varia la pendenza (derivate di $f$), la si *misura* in più punti.

### 1.4.2 Il secondo ordine: dimostrazione

**Derivazione dei metodi RK del secondo ordine**

Cerchiamo uno schema della forma

$$x_{i+1} = x_i + h\,(a_1 k_1 + a_2 k_2)\,, \qquad k_1 = f(x_i, t_i), \quad k_2 = f(x_i + q\,h\,k_1,\ t_i + p\,h)\,, \tag{1.7}$$

con $a_1, a_2, p, q$ costanti da determinare. Sviluppiamo $k_2$ in serie di Taylor di due variabili attorno a $(x_i, t_i)$:

$$k_2 = f_i + p\,h\,\frac{\partial f}{\partial t} + q\,h\,k_1\frac{\partial f}{\partial x} + \mathcal{O}\left(h^2\right) = f_i + p\,h\,\frac{\partial f}{\partial t} + q\,h\,f_i\frac{\partial f}{\partial x} + \mathcal{O}\left(h^2\right).$$

Sostituendo in (1.7):

$$x_{i+1} = x_i + (a_1 + a_2)\, h f_i + a_2 h^2 \left( p\, \frac{\partial f}{\partial t} + q\, f_i \frac{\partial f}{\partial x} \right) + \mathcal{O}\left(h^3\right).$$

Confrontiamo con la serie di Taylor (1.5) troncata al secondo ordine, in cui per la (1.4) $f_i' = \partial f/\partial t + f_i\, \partial f/\partial x$:

$$x_{i+1} = x_i + h f_i + \frac{h^2}{2} \left( \frac{\partial f}{\partial t} + f_i \frac{\partial f}{\partial x} \right) + \mathcal{O}\left(h^3\right).$$

Le due espressioni coincidono fino al secondo ordine se e solo se

$$a_1 + a_2 = 1, \qquad a_2\, p = \tfrac{1}{2}, \qquad a_2\, q = \tfrac{1}{2}. \tag{1.8}$$

Tre equazioni in quattro incognite: esiste una *famiglia* di metodi del secondo ordine, tutti con errore locale $\mathcal{O}\left(h^3\right)$ ed errore globale $\mathcal{O}\left(h^2\right)$ [3, §25.3].

Le scelte più comuni sono:

- **metodo del punto medio** ($a_2 = 1$, $a_1 = 0$, $p = q = \frac{1}{2}$): è il metodo adottato nei programmi di questa monografia quando serve un RK2,

$$x_{i+1} = x_i + h\, f\left(x_i + \tfrac{1}{2}\Delta_1 x_i,\ t_i + \tfrac{1}{2}h\right), \qquad \Delta_1 x_i = h f(x_i, t_i); \tag{1.9}$$

  si avanza con la pendenza stimata *a metà* dell'intervallo;
- **metodo di Heun** ($a_1 = a_2 = \frac{1}{2}$, $p = q = 1$): media delle pendenze all'inizio e alla fine dell'intervallo (quest'ultima stimata con un passo di Eulero);
- **metodo di Ralston** ($a_2 = \frac{2}{3}$, $p = q = \frac{3}{4}$): minimizza il limite superiore dell'errore di troncamento.

### 1.4.3 Il quarto ordine classico

Ripetendo la procedura con quattro valutazioni di $f$ si ottiene il metodo più usato in assoluto, il **Runge–Kutta classico del quarto ordine** (RK4):

$$x_{i+1} = x_i + \tfrac{1}{6}\left(\Delta_1 x_i + 2\Delta_2 x_i + 2\Delta_3 x_i + \Delta_4 x_i\right), \tag{1.10}$$

con gli incrementi calcolati *nell'ordine*

$$\begin{aligned} \Delta_1 x_i &= h\, f(x_i, t_i), \\ \Delta_2 x_i &= h\, f\left(x_i + \tfrac{1}{2}\Delta_1 x_i,\ t_i + \tfrac{1}{2}h\right), \\ \Delta_3 x_i &= h\, f\left(x_i + \tfrac{1}{2}\Delta_2 x_i,\ t_i + \tfrac{1}{2}h\right), \\ \Delta_4 x_i &= h\, f(x_i + \Delta_3 x_i,\ t_i + h). \end{aligned} \tag{1.11}$$

Il termine dominante dell'errore locale è $(h^5/5!)\, f_i^{iv}$: il metodo è del quarto ordine. Ogni incremento è una *stima della pendenza* moltiplicata per $h$: all'inizio dell'intervallo, due volte a metà (la seconda usando la prima per correggere il punto in cui si valuta $f$), e alla fine. La media pesata $(1, 2, 2, 1)/6$ è la stessa della regola di Simpson: se $f$ non dipende da $x$, la (1.10)

si riduce esattamente a Simpson per l'integrale $\int_{t_i}^{t_{i+1}} f(t)\,\mathrm{d}t$. I coefficienti sono spesso riassunti nel *tableau di Butcher*:

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

**Quanto costa l'accuratezza?**

RK4 richiede quattro valutazioni di $f$ per passo contro una di Eulero. A parità di numero di valutazioni (cioè di tempo di calcolo), Eulero può usare un passo quattro volte più piccolo, e il suo errore si riduce di un fattore 4; RK4 nello stesso tempo riduce il proprio di un fattore $2^4 = 16$ per ogni dimezzamento di $h$. Dalla Tab. 1.1: con $h = 0{,}025$ Eulero commette un errore di $6{,}5 \cdot 10^{-2}$ con 80 valutazioni di $f$; RK4 con $h = 0{,}2$ (40 valutazioni) commette $1{,}1 \cdot 10^{-4}$, seicento volte meno con metà del lavoro. Per questo RK4 è il metodo di riferimento in tutti i programmi della monografia.

## 1.5 Verifica sperimentale dell'ordine

Il modo più convincente per capire l'ordine di un metodo è misurarlo. Scegliamo un problema con soluzione esatta nota,

$$\frac{\mathrm{d}x}{\mathrm{d}t} = x - t^2 + 1, \qquad x(0) = \tfrac{1}{2}, \qquad x(t) = (t+1)^2 - \tfrac{1}{2}e^t, \tag{1.12}$$

e integriamo fino a $t = 2$ con passi via via dimezzati, registrando l'errore globale $E(h) = |x_N - x(2)|$. Se $E(h) \simeq Kh^p$, allora

$$p \simeq \log_2 \frac{E(h)}{E(h/2)}, \tag{1.13}$$

e su un grafico bilogaritmico i punti si dispongono su una retta di pendenza $p$.

```python
from ode_solvers import integra          # integra(f, t0, y0, h, n_passi, metodo)

def f_test(t, x):
    return x - t ** 2 + 1.0

def x_esatta(t):
    return (t + 1.0) ** 2 - 0.5 * np.exp(t)

passi = [0.4, 0.2, 0.1, 0.05, 0.025]
errori = {m: [] for m in ("eulero", "rk2", "rk4")}
for h in passi:
    n = int(round(2.0 / h))
    for m in errori:
        t, X = integra(f_test, 0.0, 0.5, h, n, metodo=m)
        errori[m].append(abs(X[-1, 0] - x_esatta(2.0)))

for m in errori:                          # ordine osservato p = log2(E(h)/E(h/2))
    e = np.array(errori[m])
    print(m, np.log2(e[:-1] / e[1:]))
```

es01_ordine_stabilita.py

Listato 1.1: Verifica dell'ordine di accuratezza (estratto da `es01_ordine_stabilita.py`).

Tabella 1.1: Errore globale in $t = 2$ per il problema (1.12) e ordine osservato $p$.

| $h$ | Eulero | $p$ | RK2 | $p$ | RK4 | $p$ |
|---|---|---|---|---|---|---|
| 0,400 | $7{,}459 \times 10^{-1}$ | – | $5{,}924 \times 10^{-2}$ | – | $1{,}640 \times 10^{-3}$ | – |
| 0,200 | $4{,}397 \times 10^{-1}$ | 0,76 | $1{,}510 \times 10^{-2}$ | 1,97 | $1{,}089 \times 10^{-4}$ | 3,91 |
| 0,100 | $2{,}420 \times 10^{-1}$ | 0,86 | $3{,}747 \times 10^{-3}$ | 2,01 | $6{,}990 \times 10^{-6}$ | 3,96 |
| 0,050 | $1{,}275 \times 10^{-1}$ | 0,92 | $9{,}277 \times 10^{-4}$ | 2,01 | $4{,}421 \times 10^{-7}$ | 3,98 |
| 0,025 | $6{,}550 \times 10^{-2}$ | 0,96 | $2{,}304 \times 10^{-4}$ | 2,01 | $2{,}779 \times 10^{-8}$ | 3,99 |

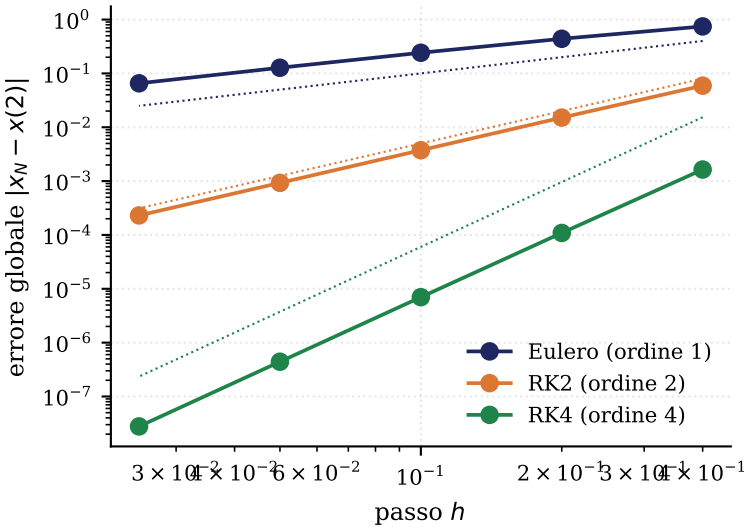



Figura 1.3: Errore globale in funzione del passo per i tre metodi (scala bilogaritmica). Le rette tratteggiate hanno pendenza 1, 2 e 4.

**Lettura dei risultati**

L'ordine osservato tende a 1, 2 e 4 al diminuire di $h$ (Tab. 1.1, Fig. 1.3), confermando la teoria. Per $h$ grande l'ordine misurato è leggermente inferiore perché i termini trascurati della serie non sono ancora trascurabili; per $h$ molto piccolo, al contrario, l'errore di arrotondamento della macchina ($\sim 10^{-16}$ per operazione, accumulato su $N$ passi) finirebbe per prevalere sull'errore di troncamento: ridurre il passo *oltre un certo limite non migliora* il risultato.

## 1.6 Stabilità assoluta: quando il passo è troppo grande

L'ordine di accuratezza dice come l'errore *decresce* al diminuire di $h$, ma non dice che cosa succede se $h$ è *grande*. La questione è spesso rimandata ai testi di analisi numerica; la affrontiamo qui perché è decisiva nei problemi del Cap. 2 (si veda il problema della pallina da ping-pong, §2.7.1) e, più avanti nel corso, nella condizione CFL per le equazioni alle derivate parziali.

## Il problema modello e il fattore di amplificazione

Consideriamo l'equazione lineare

$$\frac{\mathrm{d}x}{\mathrm{d}t} = -\lambda x, \qquad \lambda > 0, \qquad x(t) = x_0 e^{-\lambda t}, \tag{1.14}$$

la cui soluzione decade esponenzialmente con costante di tempo $\tau = 1/\lambda$. Il metodo di Eulero dà

$$x_{i+1} = x_i - h\lambda x_i = (1 - h\lambda)\, x_i \quad \Longrightarrow \quad x_N = (1 - h\lambda)^N x_0.$$

La soluzione numerica decade (come quella esatta) solo se il *fattore di amplificazione* $R(h\lambda) = 1 - h\lambda$ ha modulo minore di uno:

$$|1 - h\lambda| \leq 1 \quad \Longleftrightarrow \quad 0 < h \leq \frac{2}{\lambda} = 2\tau. \tag{1.15}$$

Per $1 < h\lambda < 2$ la soluzione numerica decade ma *cambia segno a ogni passo* (oscillazione spuria); per $h\lambda > 2$ cresce in modulo ed esplode (Fig. 1.4). Applicando le formule (1.9) e (1.11) allo stesso problema si trova, con $z = -h\lambda$,

$$R_{\mathrm{RK2}}(z) = 1 + z + \frac{z^2}{2}, \qquad R_{\mathrm{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}, \tag{1.16}$$

cioè i polinomi di Taylor di $e^z$ troncati all'ordine del metodo (è il modo più rapido per ricordarli). La condizione $|R(z)| \leq 1$ per $z$ reale negativo dà $h\lambda \leq 2$ per Eulero e RK2, $h\lambda \leq 2{,}785$ per RK4. L'insieme dei valori complessi di $z = h\lambda$ per cui $|R(z)| \leq 1$ si chiama *regione di stabilità assoluta* (Fig. 1.5); i valori complessi di $\lambda$ compaiono quando il sistema ha anche componenti oscillanti (pendolo, ala su molla).

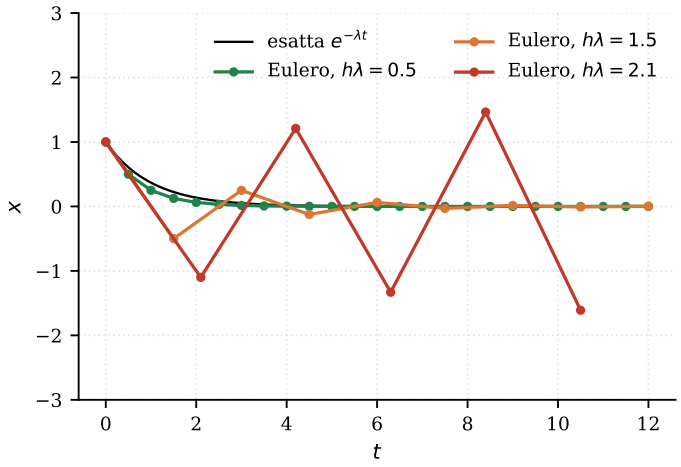



Figura 1.4: Metodo di Eulero su $\mathrm{d}x/\mathrm{d}t = -x$ con tre valori del passo: decadimento corretto ($h\lambda = 0{,}5$), oscillazione smorzata (1,5), instabilità (2,1).

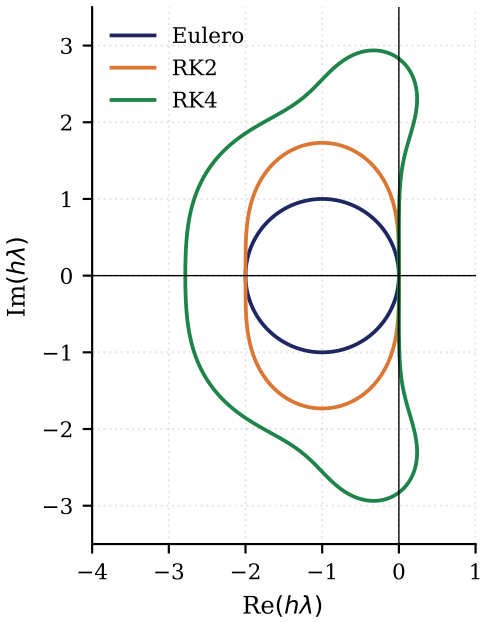



Figura 1.5: Regioni di stabilità assoluta nel piano complesso $h\lambda$ per Eulero, RK2 e RK4.

**Stabilità e accuratezza sono cose diverse**

Un passo che rispetta la (1.15) garantisce che la soluzione non esploda, non che sia accurata: per $h\lambda = 1{,}5$ Eulero è stabile ma del tutto inaccettabile (Fig. 1.4). Viceversa, con RK4 e $h\lambda = 3$ l'errore *cresce esponenzialmente* con il numero dei passi qualunque sia l'ordine del metodo. Nei problemi non lineari di questo capitolo la "$\lambda$" locale è $\partial f/\partial v$: per la caduta di una sfera con resistenza quadratica, $\lambda \simeq 2Cvc_d/A$, grande quando il corpo è leggero rispetto al fluido ($\bar{\rho}$ grande) e piccolo ($C \propto 1/d$). È il motivo per cui la pallina da ping-pong in acqua richiede $h = 0{,}01$ s mentre per la sfera d'acciaio in aria basta $h = 0{,}1$ s.

**Problemi rigidi e metodi impliciti (cenno)**

Quando in uno stesso sistema coesistono costanti di tempo molto diverse (un transitorio rapidissimo seguito da un'evoluzione lenta) il problema si dice *rigido* (*stiff*): il passo è dettato dalla scala più rapida anche quando questa si è ormai esaurita. Il rimedio sono i metodi *impliciti*, come l'Eulero all'indietro $x_{i+1} = x_i + hf(x_{i+1}, t_{i+1})$, il cui fattore di amplificazione $1/(1+h\lambda)$ ha modulo minore di uno per ogni $h$ (stabilità incondizionata), al prezzo di risolvere un'equazione, in generale non lineare, a ogni passo [3, Cap. 26]. Li ritroveremo con i metodi impliciti per le equazioni paraboliche.

## 1.7 Sistemi di equazioni ed equazioni di ordine superiore

Le equazioni del moto sono del secondo ordine. Consideriamo la forma generale

$$\frac{\mathrm{d}^2x}{\mathrm{d}t^2} = F\left(x, \frac{\mathrm{d}x}{\mathrm{d}t}, t\right), \qquad x(t_0) = x_0, \quad \frac{\mathrm{d}x}{\mathrm{d}t}(t_0) = p_0. \tag{1.17}$$

Chiamando $p$ la derivata prima, l'equazione equivale al sistema di due equazioni del primo ordine

$$\frac{\mathrm{d}x}{\mathrm{d}t} = p, \qquad \frac{\mathrm{d}p}{\mathrm{d}t} = F(x, p, t), \tag{1.18}$$

con $x(t_0) = x_0$ e $p(t_0) = p_0$. In meccanica $x$ è la posizione e $p$ la velocità; per un corpo di massa $m$, $F$ è la risultante delle forze divisa per $m$. Le formule RK4 per avanzare da $t_i$ a $t_{i+1}$ diventano

$$x_{i+1} = x_i + \tfrac{1}{6}\left(\Delta_1 x_i + 2\Delta_2 x_i + 2\Delta_3 x_i + \Delta_4 x_i\right), \qquad p_{i+1} = p_i + \tfrac{1}{6}\left(\Delta_1 p_i + 2\Delta_2 p_i + 2\Delta_3 p_i + \Delta_4 p_i\right), \tag{1.19}$$

con

$$\begin{aligned}
\Delta_1 x_i &= h\, p_i, & \Delta_1 p_i &= h\, F(x_i, p_i, t_i),\\
\Delta_2 x_i &= h\left(p_i + \tfrac{1}{2}\Delta_1 p_i\right), & \Delta_2 p_i &= h\, F\left(x_i + \tfrac{1}{2}\Delta_1 x_i,\ p_i + \tfrac{1}{2}\Delta_1 p_i,\ t_i + \tfrac{1}{2}h\right),\\
\Delta_3 x_i &= h\left(p_i + \tfrac{1}{2}\Delta_2 p_i\right), & \Delta_3 p_i &= h\, F\left(x_i + \tfrac{1}{2}\Delta_2 x_i,\ p_i + \tfrac{1}{2}\Delta_2 p_i,\ t_i + \tfrac{1}{2}h\right),\\
\Delta_4 x_i &= h\left(p_i + \Delta_3 p_i\right), & \Delta_4 p_i &= h\, F\left(x_i + \Delta_3 x_i,\ p_i + \Delta_3 p_i,\ t_i + h\right).
\end{aligned} \tag{1.20}$$

Si noti che le otto quantità vanno calcolate *a coppie e nell'ordine indicato*: $\Delta_2$ usa $\Delta_1$ di entrambe le variabili, e così via. Lo schema si estende senza modifiche a un numero qualsiasi di equazioni: basta pensare a $x$ come a un *vettore* $\mathbf{y} = (x, p)$ e a $F$ come alla funzione vettoriale $\mathbf{f}(t, \mathbf{y}) = (p,\ F(x, p, t))$. Le formule (1.10)–(1.11) valgono allora identiche, con $\mathbf{y}$ al posto di $x$. È così che sono scritte nel modulo `ode_solvers.py` (Listato 1.2): grazie agli array NumPy, la stessa funzione `passo_rk4` integra l'equazione scalare della verifica dell'ordine, il sistema

$(z, v)$ della sfera in caduta, i quattro gradi di libertà $(x, y, u, v)$ del proiettile e le 80 coordinate dei vortici del Cap. 6.

ode_solvers.py

```python
"""
ode_solvers.py -- integratori a un passo per problemi ai valori iniziali
                  dy/dt = f(t, y),  y(t0) = y0

Lo stato y puo' essere uno scalare o un array NumPy: le stesse formule
valgono per una singola equazione e per un sistema di equazioni.
"""
import numpy as np


# ---------------------------------------------------------------------
# Passi elementari: da (t, y) a y(t + h)
# ---------------------------------------------------------------------
def passo_eulero(f, t, y, h):
    """Metodo di Eulero esplicito (ordine 1)."""
    return y + h * f(t, y)


def passo_rk2(f, t, y, h):
    """Runge-Kutta del secondo ordine (metodo del punto medio)."""
    k1 = h * f(t, y)
    k2 = h * f(t + 0.5 * h, y + 0.5 * k1)
    return y + k2


def passo_rk4(f, t, y, h):
    """Runge-Kutta classico del quarto ordine."""
    k1 = h * f(t, y)
    k2 = h * f(t + 0.5 * h, y + 0.5 * k1)
    k3 = h * f(t + 0.5 * h, y + 0.5 * k2)
    k4 = h * f(t + h, y + k3)
    return y + (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0


def passo_heun(f, t, y, h):
    """Runge-Kutta del secondo ordine, variante di Heun (a1 = a2 = 1/2, p = q =
    1)."""
    k1 = h * f(t, y)
    k2 = h * f(t + h, y + k1)
    return y + 0.5 * (k1 + k2)


def passo_ralston(f, t, y, h):
    """Runge-Kutta del secondo ordine, variante di Ralston (a2 = 2/3, p = q =
    3/4)."""
    k1 = h * f(t, y)
    k2 = h * f(t + 0.75 * h, y + 0.75 * k1)
    return y + (k1 + 2.0 * k2) / 3.0


METODI = {"eulero": passo_eulero, "rk2": passo_rk2, "rk4": passo_rk4,
          "heun": passo_heun, "ralston": passo_ralston}


# ---------------------------------------------------------------------
# Marcia nel tempo a passo costante
# ---------------------------------------------------------------------
def integra(f, t0, y0, h, n_passi, metodo="rk4", stop=None):
```

```
    """
    Integra dy/dt = f(t, y) da t0 con passo h per n_passi passi.

    f       : funzione f(t, y) -> dy/dt (stessa forma di y)
    stop    : funzione opzionale stop(t, y) -> bool; se vera la marcia
              si arresta (es. proiettile che tocca il suolo)
    Restituisce gli array t e y (y ha forma (n+1, len(y0))).
    """
    passo = METODI[metodo]
    y = np.atleast_1d(np.asarray(y0, dtype=float))
    t = float(t0)
    T, Y = [t], [y.copy()]
    for _ in range(n_passi):
        y = passo(f, t, y, h)
        t = t + h
        T.append(t)
        Y.append(y.copy())
        if stop is not None and stop(t, y):
            break
    return np.array(T), np.array(Y)


# ---------------------------------------------------------------------
# Coefficiente di resistenza di una sfera liscia (approssimazione a spezzata)
# ---------------------------------------------------------------------
def cd_sfera(Re):
    """
    Approssimazione a tratti della curva sperimentale c_d(Re)
    per sfere lisce nel piano log-log (punti di raccordo a, b, c-d, e).
    """
    Re = abs(Re)
    if Re == 0.0:
        return 0.0
    if Re <= 1.0:
        return 24.0 / Re                        # regime di Stokes
    if Re <= 400.0:
        return 24.0 / Re ** 0.646               # retta a-b nel piano log-log
    if Re <= 3.0e5:
        return 0.5                              # plateau laminare
    if Re <= 2.0e6:
        return 0.000366 * Re ** 0.4275          # risalita post-crisi
    return 0.18                                 # plateau turbolento
```

Listato 1.2: Il modulo `ode_solvers.py`: integratori a un passo in forma vettoriale e curva $c_d(Re)$ della sfera.

**Come si usa `integra`**

1. Si scrive la funzione `f(t, y)` che riceve il tempo e lo stato (array) e restituisce l'array delle derivate; per un'equazione del secondo ordine è `return np.array([v, F(x, v, t)])`.
2. Si chiama `t, Y = integra(f, t0, [x0, v0], h, n_passi, metodo="rk4")`.
3. `Y[:, 0]` è la storia di $x$, `Y[:, 1]` quella di $v$; `t` contiene gli istanti.
4. L'argomento facoltativo `stop(t, y)` arresta la marcia quando una condizione diventa vera (ad esempio il proiettile tocca il suolo).

Nei programmi che seguono questa struttura verrà talvolta "aperta" per mostrare esplicitamente gli otto incrementi (Programma 1.1), e talvolta usata in forma compatta.

## 1.8 Passo adattivo: `solve_ivp` come analogo di `ODE45`

Nei testi che usano MATLAB il risolutore di riferimento è `ODE45`, basato su una coppia di formule Runge–Kutta del quarto e del quinto ordine (schema di Dormand–Prince): la differenza fra le due stime fornisce, a ogni passo e quasi gratuitamente, una stima dell'errore locale, che viene confrontata con una tolleranza; se l'errore è troppo grande il passo si ripete dimezzato, se è molto piccolo il passo successivo si allunga [3, §25.5]. Il passo si adatta quindi automaticamente alla rapidità con cui varia la soluzione: piccolo nel transitorio iniziale della pallina da ping-pong, grande quando la velocità terminale è raggiunta.

In Python l'equivalente è `scipy.integrate.solve_ivp`, con `method="RK45"` come impostazione predefinita:

```
from scipy.integrate import solve_ivp

def f(t, s):                          # s = [x, v]
    x, v = s
    return [v, F(x, v, t)]

sol = solve_ivp(f, [t0, tmax], [x0, v0], method="RK45",
                rtol=1e-6, atol=1e-9,          # tolleranze relativa e assoluta
                t_eval=np.arange(t0, tmax, h)) # istanti in cui si vuole l'output
x, v = sol.y                          # righe = variabili, colonne = istanti
```

Listato 1.3: Uso di `solve_ivp` (RK45 adattivo) per un sistema del secondo ordine.

Un evento (ad esempio $y = 0$ per il proiettile) si gestisce con l'argomento `events`, come nel programma del razzo (Cap. 5). Nei programmi di questa monografia si è scelto di scrivere esplicitamente RK4 a passo costante perché l'algoritmo resti completamente visibile; `solve_ivp` viene usato come *verifica indipendente* dei risultati.

## 1.9 Sintesi del capitolo

**Da ricordare**

- Un problema ai valori iniziali si risolve *marciando* nel tempo a passi $h$: $x_{i+1} = x_i + \Delta x_i$.
- L'errore di troncamento locale di un metodo di ordine $p$ è $\mathcal{O}(h^{p+1})$; quello globale $\mathcal{O}(h^p)$. Eulero: $p = 1$; RK2: $p = 2$; RK4: $p = 4$.
- I metodi di Runge–Kutta sostituiscono le derivate di $f$ con valutazioni di $f$ in punti intermedi; RK4 richiede quattro valutazioni per passo e la media pesata $(1, 2, 2, 1)/6$.
- Il passo deve rispettare anche un limite di *stabilità*: $h\lambda \leq 2$ (Eulero, RK2) o $h\lambda \leq 2{,}785$ (RK4), con $\lambda$ l'inverso della costante di tempo più rapida del sistema.
- Un'equazione di ordine $n$ equivale a un sistema di $n$ equazioni del primo ordine; in forma vettoriale le formule RK sono identiche al caso scalare.

- La verifica di un risultato numerico si fa ripetendo il calcolo con passi diversi ($h = 0{,}02$, 0,1, 0,2 s per la sfera del Cap. 2) o con un integratore indipendente.

# Capitolo 2

# Caduta libera di una sfera in un fluido

## 2.1 L'esperimento di Galileo rifatto al computer

Si racconta che Galileo abbia lasciato cadere dalla Torre di Pisa due corpi di massa diversa, osservando che toccavano terra nello stesso istante. Nel vuoto la conclusione è certamente vera: l'accelerazione $g$ non dipende dalla massa. Nell'atmosfera, però, l'aria esercita sul corpo forze che dipendono dalla sua dimensione e dalla sua velocità, e la conclusione diventa dubbia. Se non è esatta, quale dei due corpi arriva prima, il più grande o il più piccolo?

Invece di ripetere l'esperimento, lo *simuliamo*: formuliamo le equazioni del moto includendo tutte le forze esercitate dal fluido e le integriamo numericamente. Scegliamo una sfera perché è l'unica forma per cui si dispone di dati sperimentali di resistenza completi su tutto l'intervallo di numeri di Reynolds. È il primo esempio del capitolo in cui una risposta quantitativa *non può* essere ottenuta senza un metodo numerico.

## 2.2 Le forze su un corpo che si muove in un fluido

Scegliamo l'asse $z$ diretto come l'accelerazione di gravità $g$, con origine nel centro della sfera all'istante $t = 0$. All'istante $t > 0$ la sfera, di diametro $d$ e massa $m$, si trova alla quota $z$ e ha velocità $v = \mathrm{d}z/\mathrm{d}t$ (positiva verso il basso). Il fluido circostante ha densità $\rho_f$ e viscosità cinematica $\nu$. Nel vuoto l'unica forza è il peso $mg$, nel verso positivo di $z$; nel fluido si aggiungono le forze seguenti.

**1. La spinta di galleggiamento.** Per il principio di Archimede è pari al peso del fluido spostato, $-m_f g$, con

$$m_f = \tfrac{1}{6}\pi d^3 \rho_f, \tag{2.1}$$

e il segno negativo indica che è diretta verso l'alto.

**2. La forza dovuta all'accelerazione (massa aggiunta).** Quando un corpo immerso in un fluido in quiete viene messo in moto, trascina con sé parte del fluido circostante, che acquista energia cinetica. Questa energia deve essere fornita dal corpo, che quindi avverte una resistenza *anche se il fluido è privo di viscosità*. Per la sfera la teoria del moto potenziale [9] dà una forza pari a $-\frac{1}{2} m_f \, \mathrm{d}v/\mathrm{d}t$.

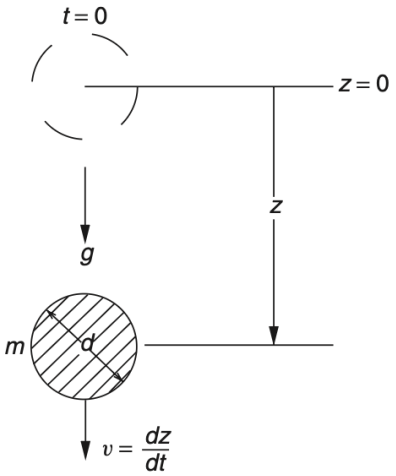



Figura 2.1: Schema della sfera in caduta libera e sistema di riferimento.

**Da dove viene il fattore $\frac{1}{2}$: la massa aggiunta della sfera**

Il campo di velocità potenziale attorno a una sfera di raggio $R$ che si muove con velocità $v$ in un fluido indefinito in quiete ha, in coordinate sferiche centrate sulla sfera, il potenziale di dipolo $\phi = -\frac{1}{2} v R^3 \cos\theta / r^2$. L'energia cinetica del fluido è

$$E_f = \tfrac{1}{2}\rho_f \int_{r>R} |\nabla\phi|^2 \, \mathrm{d}V = \tfrac{1}{2}\left(\tfrac{1}{2} m_f\right) v^2.$$

Il fluido si comporta quindi come una massa $m' = \frac{1}{2} m_f$ che si muove con la velocità della sfera. Per il teorema dell'energia cinetica, la potenza spesa dal corpo per accelerare il fluido è $\mathrm{d}E_f/\mathrm{d}t = m' v \, \mathrm{d}v/\mathrm{d}t$, cioè la forza aggiuntiva che agisce sul corpo è $-m' \, \mathrm{d}v/\mathrm{d}t$. In generale $m' = k\, m_f$ con $k$ dipendente dalla forma: $k = \frac{1}{2}$ per la sfera, $k = 1$ per il cilindro circolare in moto trasversale, $k \to \infty$ per una lastra piana in moto normale al proprio piano (che trascina teoricamente una massa infinita di fluido). La massa aggiunta è essenziale in idrodinamica navale e nelle strutture offshore; in aria, per corpi densi, è trascurabile, ma il programma la conserva per generalità.

**3. Le forze viscose.** Attorno a un corpo che si muove in un fluido reale esiste uno strato sottile, lo *strato limite*, in cui la velocità passa rapidamente dal valore della parete a quello della corrente esterna. Il gradiente di velocità produce uno sforzo tangenziale sulla superficie; la risultante degli sforzi tangenziali è l'*attrito di superficie* (*skin friction*). Inoltre, in un fluido viscoso la pressione sulla parte posteriore del corpo è minore di quella sulla parte anteriore (lo strato limite si separa e lascia una scia a bassa pressione): ne risulta la *resistenza di pressione* o *di forma*. La somma delle due è la resistenza viscosa totale, espressa in forma adimensionale dal coefficiente di resistenza

$$c_d = \frac{\text{resistenza}}{\frac{1}{2}\rho_f v^2 \, \frac{1}{4}\pi d^2}, \qquad Re = \frac{v\, d}{\nu}, \tag{2.2}$$

funzione della forma del corpo e del numero di Reynolds. Salvo poche forme semplici a bassissimo $Re$, questa funzione non è calcolabile analiticamente e si ricava dall'esperimento (Fig. 2.2).

**4. La resistenza d'onda.** Quando la velocità del corpo è confrontabile con quella del suono si formano onde d'urto sul corpo o davanti a esso, con una resistenza aggiuntiva. Limitandoci a velocità subsoniche basse, la trascuriamo.

## 2.3 La curva $c_d(Re)$ della sfera e la sua approssimazione

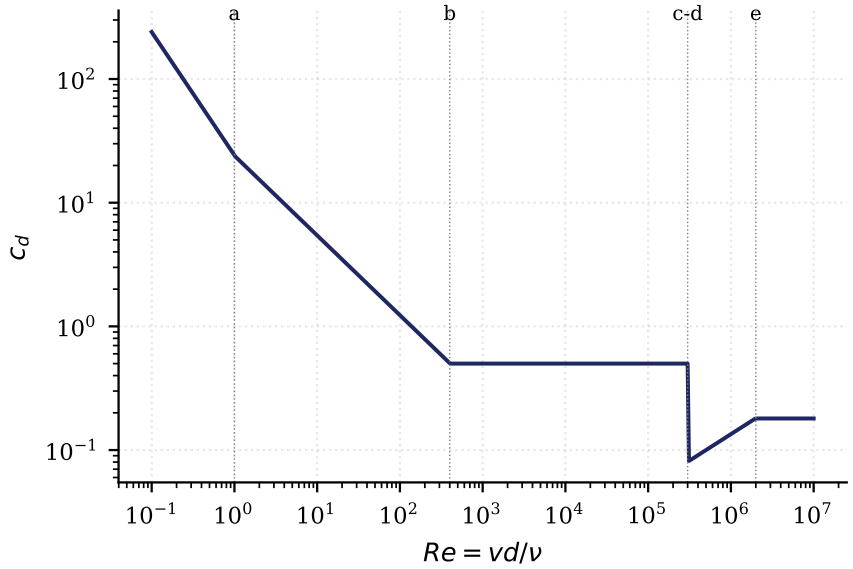



Figura 2.2: Approssimazione a tratti della curva sperimentale $c_d(Re)$ per sfere lisce [10], come implementata nella funzione `cd_sfera`. Le linee verticali indicano i punti di raccordo $a$, $b$, $c$–$d$, $e$ della spezzata.

Per usare la curva sperimentale nel programma la si sostituisce con una *spezzata* nel piano bilogaritmico (Fig. 2.2):

$$c_d = \frac{24}{Re} \qquad Re \le 1 \quad \text{(formula di Stokes)}, \tag{2.3}$$

$$c_d = \frac{24}{Re^{0,646}} \qquad 1 < Re \le 400, \tag{2.4}$$

$$c_d = 0{,}5 \qquad 400 < Re \le 3 \cdot 10^5, \tag{2.5}$$

$$c_d = 0{,}000366\, Re^{0,4275} \qquad 3 \cdot 10^5 < Re \le 2 \cdot 10^6, \tag{2.6}$$

$$c_d = 0{,}18 \qquad Re > 2 \cdot 10^6. \tag{2.7}$$

**Da dove vengono gli esponenti: rette nel piano log-log**

Una relazione $c_d = K\, Re^n$ è una retta nel piano $(\log Re, \log c_d)$ con pendenza $n$. La retta che congiunge $a = (1,\ 24)$ e $b = (400,\ 0{,}5)$ ha pendenza

$$n = \frac{\log(0{,}5) - \log(24)}{\log(400) - \log(1)} = -\frac{\log 48}{\log 400} = -0{,}646,$$

e passando per $a$ ha $K = 24$: è la (2.4). Analogamente fra $d = (3 \cdot 10^5,\ 0{,}08)$ ed $e = (2 \cdot 10^6,\ 0{,}18)$:

$$n = \frac{\log(0{,}18/0{,}08)}{\log(2 \cdot 10^6/3 \cdot 10^5)} = \frac{\log 2{,}25}{\log 6{,}667} = 0{,}4275, \qquad K = \frac{0{,}08}{(3 \cdot 10^5)^{0,4275}} = 3{,}66 \cdot 10^{-4},$$

che è la (2.6). Il lettore può verificare questi numeri con tre righe di Python. L'approssimazione è grossolana attorno ai punti di raccordo (a $Re = 1$ la formula di Stokes dà $c_d = 24$ contro un valore sperimentale di circa 27); una tabella interpolata darebbe risultati migliori, al prezzo di un codice un po' più lungo.

**La crisi della resistenza e la pallina da golf**

Il crollo di $c_d$ da 0,5 a circa 0,08 attorno a $Re \simeq 3 \cdot 10^5$ (punto $c$–$d$) è la *crisi della resistenza*: lo strato limite diventa turbolento *prima* del punto di separazione; uno strato limite turbolento, più energico, resiste meglio al gradiente di pressione avverso e si separa più a valle, la scia si restringe e la resistenza di forma crolla [8, §17.11]. È lo stesso fenomeno sfruttato dalle fossette della pallina da golf, che provocano la transizione a $Re$ più bassi e riducono la resistenza; e lo stesso che vedremo produrre una brusca accelerazione nella sfera da 7 cm di Fig. 2.4 e un'anomalia nell'angolo di tiro ottimo nel Cap. 4.

## 2.4 L'equazione del moto

Con tutte le forze del §2.2, la seconda legge di Newton per la sfera si scrive

$$m\frac{\mathrm{d}v}{\mathrm{d}t} = mg - m_f g - \tfrac{1}{2}m_f\frac{\mathrm{d}v}{\mathrm{d}t} - \tfrac{1}{2}\rho_f\, v|v|\,\tfrac{1}{4}\pi d^2\, c_d(v). \tag{2.8}$$

Si usa $v|v|$ anziché $v^2$ perché la resistenza deve essere sempre opposta alla velocità, anche quando il corpo risale ($v < 0$). Riordinando,

$$\left(m + \tfrac{1}{2}m_f\right)\frac{\mathrm{d}v}{\mathrm{d}t} = (m - m_f)\, g - \tfrac{1}{8}\pi\rho_f\, d^2\, v|v|\, c_d(v). \tag{2.9}$$

Il primo membro mostra che in un moto accelerato il corpo si comporta come se la sua massa fosse aumentata di $\frac{1}{2}m_f$: la massa aggiunta. Sostituendo $m = \frac{1}{6}\pi d^3\rho$ ($\rho$ densità del corpo) e la (2.1), e dividendo per $\frac{1}{6}\pi d^3\rho$, si ottiene il sistema del primo ordine nella forma (1.18):

$$\frac{\mathrm{d}v}{\mathrm{d}t} = \frac{1}{A}\left[B - C\, v|v|\, c_d(v)\right] \equiv F(v), \qquad \frac{\mathrm{d}z}{\mathrm{d}t} = v, \tag{2.10}$$

con

$$A = 1 + \tfrac{1}{2}\bar\rho, \qquad B = (1 - \bar\rho)\, g, \qquad C = \frac{3\bar\rho}{4d}, \qquad \bar\rho = \frac{\rho_f}{\rho}. \tag{2.11}$$

**Il significato fisico di $A$, $B$, $C$**

Tutto il fluido entra attraverso il solo rapporto di densità $\bar\rho$ (oltre che attraverso $c_d$). $A$ è il fattore di massa apparente; $B$ è l'accelerazione di gravità ridotta dal galleggiamento, che si annulla per $\bar\rho = 1$ (corpo neutro) e cambia segno per $\bar\rho > 1$ (corpo più leggero del fluido: risale); $C$ ha le dimensioni dell'inverso di una lunghezza ed è il coefficiente della resistenza per unità di massa: tanto più grande quanto più piccolo è il diametro. Per una sfera d'acciaio in aria $\bar\rho = 1{,}5 \cdot 10^{-4}$: galleggiamento e massa aggiunta sono del tutto trascurabili ($A \simeq 1$, $B \simeq g$), ma non lo sarebbero in acqua.

Poiché il secondo membro della (2.10) non contiene esplicitamente né $z$ né $t$, lo chiamiamo semplicemente $F(v)$; le formule RK4 (1.20) si semplificano negli otto incrementi

$$\begin{aligned} \Delta_1 z_i &= h v_i, & \Delta_1 v_i &= hF(v_i), \\ \Delta_2 z_i &= h\left(v_i + \tfrac{1}{2}\Delta_1 v_i\right), & \Delta_2 v_i &= hF\left(v_i + \tfrac{1}{2}\Delta_1 v_i\right), \\ \Delta_3 z_i &= h\left(v_i + \tfrac{1}{2}\Delta_2 v_i\right), & \Delta_3 v_i &= hF\left(v_i + \tfrac{1}{2}\Delta_2 v_i\right), \\ \Delta_4 z_i &= h\left(v_i + \Delta_3 v_i\right), & \Delta_4 v_i &= hF\left(v_i + \Delta_3 v_i\right), \end{aligned} \tag{2.12}$$

seguiti dalle medie pesate (1.19). Nel vuoto ($\rho_f = 0$) le equazioni si integrano direttamente: $v_v = v_0 + gt$, $z_v = z_0 + v_0 t + \frac{1}{2} g t^2$; il programma le calcola per confronto.

## 2.5 Programma 1.1

**Dati dell'esperimento numerico**

Sfere d'acciaio in aria in condizioni standard al livello del mare: $\rho = 8000$ kg/m$^3$, $\rho_f = 1{,}22$ kg/m$^3$, $\nu = 1{,}49 \cdot 10^{-5}$ m$^2$/s, $g = 9{,}8$ m/s$^2$; $z_0 = 0$, $v_0 = 0$; passo $h = 0{,}1$ s; tempo massimo 10 s (poi 12 s per le figure). Diametro di riferimento $d = 0{,}01$ m.

Nel programma le variabili $t_i$, $z_i$, $v_i$ non vengono conservate come array con un elemento per passo, nello stile dei programmi classici in FORTRAN: si stampano i valori a ogni passo e si riutilizzano gli stessi nomi per i valori all'istante successivo (gli indici $i$, $i+1$ scompaiono). Per tracciare i grafici, tuttavia, i valori vengono anche accumulati in liste. Il listato 2.1 riproduce fedelmente la struttura del Programma 1.1: la funzione `F(v)`, gli otto incrementi, la media pesata. Il numero di Reynolds istantaneo è stampato per riferimento.

prog1_1_caduta_sfera.py

```
"""
prog1_1_caduta_sfera.py  --  Programma 1.1
Caduta libera di una sfera in un fluido in quiete (Runge-Kutta 4).

    dz/dt = v
    dv/dt = F(v) = [ B - C v|v| c_d(Re) ] / A

con  A = 1 + rho_bar/2,  B = (1 - rho_bar) g,  C = 3 rho_bar/(4 d),
rho_bar = rho_f / rho  (rapporto fra densita' del fluido e del corpo).
"""
import numpy as np
import matplotlib.pyplot as plt
from ode_solvers import cd_sfera
from stile_grafici import salva, PALETTE

# ------------------------------------------------------------ dati
RHO, RHOF, NU, G = 8000.0, 1.22, 1.49e-5, 9.8   # acciaio in aria (livello del mare)


def costanti(d, rho=RHO, rhof=RHOF):
    rb = rhof / rho
    return 1.0 + 0.5 * rb, (1.0 - rb) * G, 3.0 * rb / (4.0 * d)


def caduta(d, h=0.1, tmax=10.0, z0=0.0, v0=0.0, rho=RHO, rhof=RHOF, nu=NU):
    """Integra il moto con RK4 e restituisce gli array t, z, v, Re."""
    A, B, C = costanti(d, rho, rhof)

```

```
    def F(v):                                          # accelerazione dv/dt
        return (B - C * v * abs(v) * cd_sfera(v * d / nu)) / A

    t, z, v = 0.0, z0, v0
    T, Z, V = [t], [z], [v]
    while t < tmax - 1e-12:
        d1z = h * v;              d1v = h * F(v)
        d2z = h * (v + d1v / 2);  d2v = h * F(v + d1v / 2)
        d3z = h * (v + d2v / 2);  d3v = h * F(v + d2v / 2)
        d4z = h * (v + d3v);      d4v = h * F(v + d3v)
        z += (d1z + 2 * d2z + 2 * d3z + d4z) / 6
        v += (d1v + 2 * d2v + 2 * d3v + d4v) / 6
        t += h
        T.append(t); Z.append(z); V.append(v)
    T, Z, V = map(np.array, (T, Z, V))
    return T, Z, V, V * d / nu


if __name__ == "__main__":
    # ------------------------------------------- tabella per d = 0.01 m
    d = 0.01
    T, Z, V, RE = caduta(d, h=0.1, tmax=10.0)
    ZV, VV = 0.5 * G * T ** 2, G * T                  # soluzione nel vuoto
    print(f"Sfera d'acciaio, d = {d} m, h = 0.1 s")
    print(f"{'t [s]':>6} {'z_vuoto':>9} {'z_aria':>9} {'v_vuoto':>9} {'v_aria':>9}
    {'Re':>10}")
    for i in range(0, len(T), 10):
        print(f"{T[i]:6.1f} {ZV[i]:9.3f} {Z[i]:9.3f} {VV[i]:9.3f} {V[i]:9.3f}
    {RE[i]:10.0f}")

    # ------------------------------------------- verifica del passo
    print("\nVerifica della sensibilita' al passo (z, v a t = 10 s):")
    for h in (0.2, 0.1, 0.02):
        T_, Z_, V_, _ = caduta(d, h=h, tmax=10.0)
        print(f"  h = {h:4.2f}  z = {Z_[-1]:9.4f} m   v = {V_[-1]:8.4f} m/s")

    # ------------------------------------------- velocita' terminale
    A, B, C = costanti(d)
    vt = np.sqrt(B / (C * cd_sfera(RE[-1])))
    print(f"\nRe finale = {RE[-1]:.0f} -> c_d = {cd_sfera(RE[-1])}, "
          f"v_t = sqrt(B/(C c_d)) = {vt:.2f} m/s  (v a 10 s: {V[-1]:.2f} m/s)")

    # ------------------------------------------- figure per vari diametri
    diametri = (0.001, 0.005, 0.01, 0.02, 0.07)
    fig1, ax1 = plt.subplots(figsize=(4.4, 3.2))
    fig2, ax2 = plt.subplots(figsize=(4.4, 3.2))
    tt = np.linspace(0, 12, 200)
    for dd, c in zip(diametri, PALETTE):
        T, Z, V, RE = caduta(dd, h=0.1, tmax=12.0)
        ax1.plot(T, Z, color=c, label=f"$d={dd}$ m")
        ax2.plot(T, V, color=c, label=f"$d={dd}$ m")
        print(f"d = {dd:5.3f} m: v(12 s) = {V[-1]:6.2f} m/s,  Re = {RE[-1]:9.0f}, "
              f"t(z=56 m) = {np.interp(56.0, Z, T):.3f} s")
    ax1.plot(tt, 0.5 * G * tt ** 2, "k--", lw=1, label="nel vuoto")
    ax2.plot(tt, G * tt, "k--", lw=1, label="nel vuoto")
    ax1.set_xlabel("$t$ [s]"); ax1.set_ylabel("$z$ [m]"); ax1.set_ylim(0, 500)
    ax2.set_xlabel("$t$ [s]"); ax2.set_ylabel("$v$ [m/s]"); ax2.set_ylim(0, 100)
    ax1.legend(fontsize=7); ax2.legend(fontsize=7, loc="upper left")
    salva(fig1, "prog1_1_z")
    salva(fig2, "prog1_1_v")
```

```
87
88      # ---------------------------------------- curva c_d(Re)
89      Re = np.logspace(-1, 7, 500)
90      fig, ax = plt.subplots(figsize=(4.6, 3.0))
91      ax.loglog(Re, [cd_sfera(r) for r in Re], color=PALETTE[0])
92      for r, lab in ((1, "a"), (400, "b"), (3e5, "c-d"), (2e6, "e")):
93          ax.axvline(r, color="gray", lw=0.5, ls=":")
94          ax.text(r, 300, lab, ha="center", fontsize=7)
95      ax.set_xlabel("$Re = v d/\\nu$"); ax.set_ylabel("$c_d$")
96      salva(fig, "prog1_1_cd")
```

Listato 2.1: Programma 1.1 – caduta libera di una sfera (`prog1_1_caduta_sfera.py`).

Tabella 2.1: Uscita del Programma 1.1 per la sfera d'acciaio di 0,01 m ($h = 0{,}1$ s; stampata ogni secondo).

| $t$ [s] | $z_v$ [m] | $z$ [m] | $v_v$ [m/s] | $v$ [m/s] | $Re$ |
|---|---|---|---|---|---|
| 0,0 | 0,000 | 0,000 | 0,000 | 0,000 | 0 |
| 1,0 | 4,900 | 4,854 | 9,800 | 9,619 | 6456 |
| 2,0 | 19,600 | 18,905 | 19,600 | 18,252 | 12 250 |
| 3,0 | 44,100 | 40,817 | 29,400 | 25,280 | 16 967 |
| 4,0 | 78,400 | 68,879 | 39,200 | 30,562 | 20 511 |
| 5,0 | 122,500 | 101,424 | 49,000 | 34,296 | 23 018 |
| 6,0 | 176,400 | 137,070 | 58,800 | 36,825 | 24 715 |
| 7,0 | 240,100 | 174,786 | 68,600 | 38,487 | 25 830 |
| 8,0 | 313,600 | 213,849 | 78,400 | 39,559 | 26 550 |
| 9,0 | 396,900 | 253,775 | 88,200 | 40,241 | 27 007 |
| 10,0 | 490,000 | 294,248 | 98,000 | 40,672 | 27 296 |

**Verifica dell'accuratezza: il test del passo**

Come si fa a sapere se $h = 0{,}1$ s è adeguato, non avendo una soluzione esatta con cui confrontarsi? Si ripete il calcolo con passi diversi. Con $h = 0{,}2$ e $h = 0{,}02$ s si ottiene, a $t = 10$ s, $z = 294{,}2477$ m e $v = 40{,}6718$ m/s in tutti e tre i casi, identici alle quattro cifre decimali: il risultato è *indipendente dal passo* e quindi affidabile. Questo controllo elementare è la forma più semplice di *studio di convergenza* e andrebbe fatto in ogni simulazione numerica. Con il metodo RK4 l'errore si riduce di un fattore 16 a ogni dimezzamento di $h$, quindi se due passi che differiscono di un fattore 10 danno lo stesso risultato, l'errore di troncamento è davvero trascurabile.

**Che cosa dice l'esperimento numerico**

- **La risposta a Galileo.** Le curve di spostamento (Fig. 2.3) mostrano che, per percorrere una data distanza, un corpo più grande impiega meno tempo di uno più piccolo. Per i 56 m della Torre di Pisa il programma dà 3,38 s nel vuoto, 3,41 s per $d = 7$ cm, 3,47 s per 2 cm, 3,56 s per 1 cm, 3,75 s per 5 mm e 5,2 s per 1 mm. Per due sfere di dimensioni confrontabili la differenza è una piccola frazione di secondo, impercettibile senza strumenti: Galileo aveva ragione *nei limiti della sua precisione*.
- **La velocità terminale.** Un corpo che cade in un fluido raggiunge sempre una

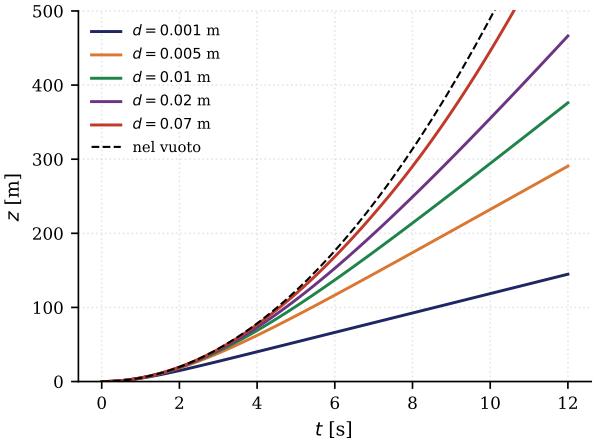



Figura 2.3: Spostamento di sfere d'acciaio in caduta nell'aria per vari diametri.

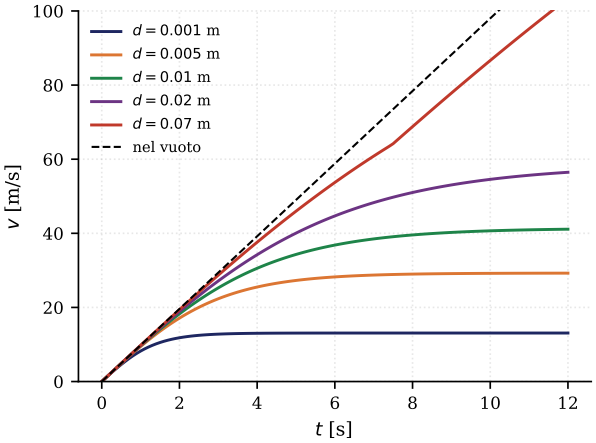



Figura 2.4: Velocità delle stesse sfere. Si noti il brusco cambio di pendenza per $d = 0{,}07$ m attorno a $t \simeq 7{,}5$ s.

velocità costante (Fig. 2.4), tanto maggiore quanto più grande è il diametro. Per $d = 1$ cm: 40,7 m/s a 10 s, con $Re \simeq 2{,}73 \cdot 10^4$.

- **La crisi della resistenza in azione.** Per $d = 7$ cm il numero di Reynolds supera $3 \cdot 10^5$ attorno a $t \simeq 7{,}5$ s: il crollo di $c_d$ produce una brusca *accelerazione* del corpo, visibile come un gomito nella curva della velocità. A 12 s la sfera ha raggiunto 103 m/s.
- **Il limite del vuoto.** Per una sfera molto grande l'effetto del fluido diventa trascurabile rispetto all'inerzia: la curva tende a quella del vuoto, la velocità cresce indefinitamente e non esiste velocità terminale ($C \to 0$ per $d \to \infty$).

## 2.6 La velocità terminale

Alla velocità terminale $v_t$ l'accelerazione è nulla: il peso è equilibrato da galleggiamento e resistenza. Ponendo $\mathrm{d}v/\mathrm{d}t = 0$ nella (2.9), con $m = \frac{1}{6}\pi d^3 \rho$:

$$(1-\bar\rho)\,\tfrac{1}{6}\pi d^3 \rho\, g = \tfrac{1}{8}\pi \rho_f d^2 v_t^2 c_d(v_t) \quad\Longrightarrow\quad v_t = \sqrt{\frac{4(1-\bar\rho)\,g\,d}{3\,\bar\rho\,c_d(v_t)}} = \sqrt{\frac{B}{C\,c_d(v_t)}}. \tag{2.13}$$

La formula è *implicita*: $c_d$ dipende da $v_t$ attraverso $Re$. Nel caso della sfera di 1 cm il calcolo numerico ha mostrato che il Reynolds terminale è circa $2{,}73 \cdot 10^4$, cioè nel tratto (2.5) in cui $c_d = 0{,}5$; allora $v_t = \sqrt{9{,}8/(0{,}01144 \cdot 0{,}5)} = 41{,}4$ m/s, in accordo con l'asintoto della Fig. 2.4. In generale $c_d(v_t)$ non è noto a priori e la velocità terminale non si può calcolare senza prima integrare il moto; ma in due casi limite la formula diventa esplicita.

**Due casi espliciti**

**Regime di Stokes** ($Re < 1$, particelle minuscole). Sostituendo $c_d = 24/Re = 24\nu/(v_t d)$ nella (2.13) e risolvendo:

$$v_t = \frac{(1-\bar\rho)\,g\,d^2}{18\,\bar\rho\,\nu}. \tag{2.14}$$

La velocità terminale cresce con il *quadrato* del diametro: è la legge con cui sedimentano polveri e aerosol (e con cui si misurava la viscosità nell'esperimento di Millikan).
**Regime turbolento** ($Re > 2 \cdot 10^6$). Per sfere d'acciaio in aria il calcolo mostra che il Reynolds terminale supera $2 \cdot 10^6$ se $d > 0{,}12$ m; allora $c_d = 0{,}18$ costante e $v_t \propto \sqrt{d}$. Un esercizio istruttivo: qual è il diametro minimo di una sfera d'acciaio la cui velocità terminale nell'aria supera la velocità del suono (340 m/s)? Dalla (2.13) con $v_t = 340$ m/s e $c_d = 0{,}18$:

$$d = \frac{3\,\bar{\rho}\,c_d\,v_t^2}{4(1-\bar{\rho})\,g} = \frac{3 \cdot 1{,}525 \cdot 10^{-4} \cdot 0{,}18 \cdot 340^2}{4 \cdot 9{,}8} = 0{,}243 \text{ m}.$$

Il valore è una stima per difetto: prima di raggiungere la velocità del suono compare una regione transonica con resistenza d'onda, e i dati di $c_d$ a $Re$ così alti sono scarsi.

**Atmosfera non uniforme**

Se il corpo percorre distanze tali che $\rho_f$ e $\nu$ variano apprezzabilmente con la quota (rientro atmosferico, lancio di sonde, caduta in oceano) la funzione $F(v)$ va sostituita con $F(v, z)$, specificando le leggi $\rho_f(z)$ e $\nu(z)$: la struttura del programma non cambia, perché RK4 per sistemi accetta una $F$ dipendente da tutte le variabili (si veda il Problema 4.3 nel Cap. 4).

## 2.7 Problemi svolti

### 2.7.1 Problema 2.1: la pallina da ping-pong in acqua

Una pallina da ping-pong di $d = 0{,}036$ m è rilasciata sul fondo di una vasca d'acqua ($\rho_f = 1000$ kg/m$^3$, $\nu = 10^{-6}$ m$^2$/s); il guscio è così sottile che la densità del corpo coincide con quella dell'aria interna, $\rho = 1{,}22$ kg/m$^3$. Il Programma 1.1 funziona senza modifiche, cambiando i dati: ora $\bar{\rho} = 820$, per cui $A = 411$ (la massa aggiunta è 410 volte la massa della pallina!), $B = -8023$ m/s$^2$ (negativo: la spinta di Archimede prevale) e $C = 17\,077$ m$^{-1}$. L'accelerazione iniziale è $B/A = -19{,}5$ m/s$^2$, verso l'alto, pari a circa $2g$.

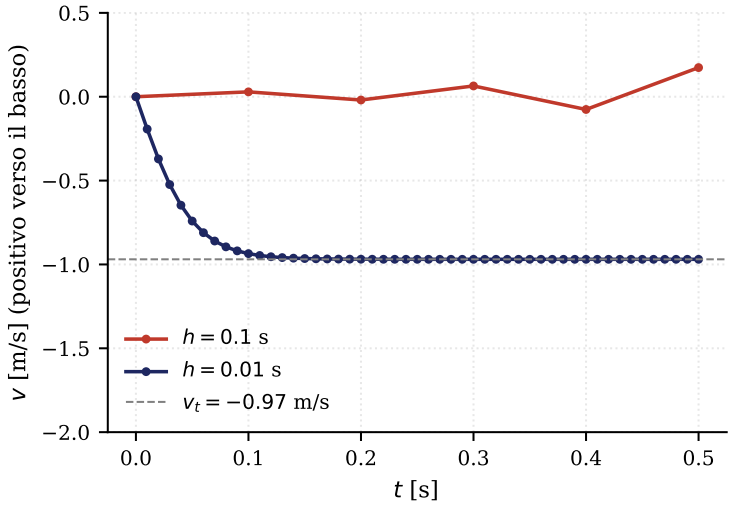



Figura 2.5: Pallina da ping-pong rilasciata in acqua: velocità (positiva verso il basso) con $h = 0{,}1$ s e $h = 0{,}01$ s. Con il passo grande il calcolo è instabile e privo di senso fisico.

Con $h = 0{,}1$ s, il passo che andava benissimo per la sfera d'acciaio, la soluzione è instabile (Fig. 2.5): la velocità oscilla attorno allo zero senza mai avvicinarsi al valore terminale; con $h = 0{,}01$ s la pallina raggiunge in meno di 0,1 s la velocità di risalita $v_t = -0{,}97$ m/s, in accordo con la (2.13) (che per $B < 0$ va letta con il segno meno: $v_t = -\sqrt{-B/(Cc_d)}$ e $c_d = 0{,}5$, essendo $Re \simeq 3{,}5 \cdot 10^4$).

**Perché $h = 0{,}1$ s non va: la costante di tempo del problema**

Linearizziamo la (2.10) attorno alla velocità terminale: posto $v = v_t + \delta$, con $c_d$ costante,

$$\frac{\mathrm{d}\delta}{\mathrm{d}t} \simeq -\frac{2Cc_d|v_t|}{A}\,\delta \equiv -\lambda\,\delta, \qquad \tau = \frac{1}{\lambda} = \frac{A}{2Cc_d|v_t|}.$$

Per la pallina $\tau = 411/(2 \cdot 17\,077 \cdot 0{,}5 \cdot 0{,}97) = 0{,}025$ s. Il limite di stabilità di RK4, $h\lambda \le 2{,}785$ (§1.6), richiede $h \le 2{,}785\,\tau = 0{,}07$ s: $h = 0{,}1$ s è fuori dalla regione di stabilità, $h = 0{,}01$ s ($h\lambda = 0{,}4$) è ben dentro, e l'accuratezza è anche buona. Per la sfera d'acciaio di 1 cm, invece, $\tau = 1/(2 \cdot 0{,}01144 \cdot 0{,}5 \cdot 41{,}4) = 2{,}1$ s e $h = 0{,}1$ s dà $h\lambda = 0{,}05$: nessun problema. La regola pratica è che *il passo deve essere una piccola frazione della costante di tempo più rapida del sistema*, e questa va stimata prima di lanciare il calcolo.

### 2.7.2 Problema 2.2: la velocità terminale non dipende dalla velocità iniziale

Per un dato corpo in un dato fluido la velocità terminale è fissata dall'equilibrio delle forze, indipendentemente da come il moto è iniziato. Assegnando a $v_0$ i valori 0, $-30$ m/s (lancio verso l'alto) e $+80$ m/s (lancio verso il basso, oltre la velocità terminale) il Programma 1.1 dà, a 12 s, rispettivamente 41,1, 40,4 e 41,5 m/s (Fig. 2.6): in tutti i casi la velocità converge a $v_t = 41{,}4$ m/s, da sotto o da sopra. Matematicamente $v_t$ è un *punto di equilibrio stabile* dell'equazione $\mathrm{d}v/\mathrm{d}t = F(v)$: $F(v_t) = 0$ e $F'(v_t) < 0$.

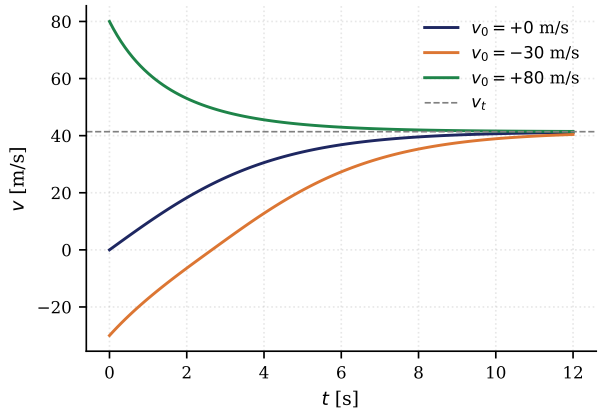



Figura 2.6: Sfera d'acciaio di 1 cm con tre velocità iniziali diverse: convergenza alla stessa $v_t$.

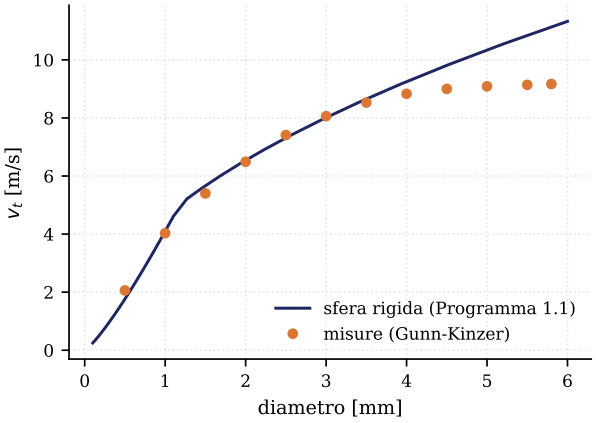



Figura 2.7: Velocità terminale di gocce d'acqua in aria: sfera rigida (Programma 1.1) e misure di Gunn e Kinzer [11].

### 2.7.3 Le gocce di pioggia

In un giorno di pioggia si osserva che le gocce più grandi cadono più in fretta delle piccole. Trattando la goccia come una sfera rigida d'acqua ($\rho = 1000$ kg/m$^3$ nel programma) si ottiene

la curva di Fig. 2.7, confrontata con le misure classiche di Gunn e Kinzer: l'accordo è buono fino a circa 3,5 mm di diametro, oltre il quale le gocce reali sono più *lente* della sfera rigida.

La differenza ha due cause opposte. A piccoli $Re$ la goccia è sferica, ma la sua superficie è mobile: il fluido interno circola, riducendo lo sforzo tangenziale e ritardando la separazione; la resistenza è quindi *minore* di quella della sfera rigida. Per $Re \ll 1$ la resistenza di una sfera liquida è [12]

$$D = 3\pi\mu_e\, d\, v\, \frac{1 + \dfrac{2\mu_e}{3\mu_i}}{1 + \dfrac{\mu_e}{\mu_i}}, \tag{2.15}$$

con $\mu_i$ e $\mu_e$ le viscosità dinamiche del fluido interno ed esterno; per acqua in aria ($\mu_i = 10^{-3}$, $\mu_e = 1{,}818 \cdot 10^{-5}$ kg/(m s)) il fattore correttivo vale 0,994 e la resistenza è appena inferiore ai $3\pi\mu_e dv$ di Stokes. A $Re$ grandi, invece, la goccia viene *schiacciata* dalla corrente esterna in una forma appiattita verticalmente, che provoca una separazione precoce e un forte aumento della resistenza di forma: la velocità terminale cresce sempre meno con il diametro (tratto piatto della curva sperimentale). Oltre circa 6 mm le instabilità frammentano la goccia, ed è per questo che gocce più grandi non si osservano.

**Una goccia che gela**

Se una goccia gela, la densità diminuisce e il diametro aumenta. A basso $Re$, dalla (2.14), $v_t \propto (1-\bar\rho)\, d^2/\bar\rho \simeq \rho\, d^2$: a parità di massa $\rho d^3 = \text{cost}$, quindi $v_t \propto d^{-1}$ e la goccia che gela *decelera*. Il lettore può verificarlo con il programma assegnando a `rho` e `d` i valori prima e dopo il congelamento.

## 2.8 Sintesi del capitolo

**Da ricordare**

- Su un corpo in moto in un fluido agiscono: galleggiamento ($-m_f g$), forza di massa aggiunta ($-k\, m_f\, \mathrm{d}v/\mathrm{d}t$, $k = \frac{1}{2}$ per la sfera), resistenza viscosa ($\frac{1}{2}\rho_f v|v| S c_d$), resistenza d'onda (trascurata a bassa velocità).
- $c_d(Re)$ della sfera: Stokes ($24/Re$) a $Re < 1$, plateau 0,5 fra 400 e $3 \cdot 10^5$, crisi della resistenza a $3 \cdot 10^5$, plateau 0,18 oltre $2 \cdot 10^6$.
- L'equazione del moto $\mathrm{d}v/\mathrm{d}t = [B - Cv|v|c_d]/A$ dipende dal fluido solo tramite $\bar\rho = \rho_f/\rho$.
- La velocità terminale $v_t = \sqrt{B/(Cc_d(v_t))}$ è un equilibrio stabile raggiunto da qualunque velocità iniziale; è esplicita solo nei regimi di Stokes e turbolento.
- Il passo di integrazione va confrontato con la costante di tempo $\tau = A/(2Cc_d|v_t|)$: grande per corpi densi e grandi, piccolissima per corpi leggeri in fluidi densi.
- L'affidabilità di un risultato numerico si verifica ripetendo il calcolo con passi diversi.

# Capitolo 3

# Simulazione di moti vincolati: pendolo e ala su molla

## 3.1 La simulazione numerica come esperimento

Nel capitolo precedente il sistema fisico è stato sostituito da un sistema di equazioni che ne descrive approssimativamente il moto, e l'algoritmo risolutivo è stato programmato per un insieme di condizioni assegnate. Variando i dati d'ingresso dello stesso programma si ottiene il moto di una sfera di qualunque materiale, in qualunque fluido, a partire da qualunque condizione iniziale: invece di *misurare* il moto reale (operazione spesso laboriosa, talvolta impossibile), lo si *simula* al computer. Fra moto reale e moto simulato esistono certamente discrepanze, dovute alle approssimazioni del modello e agli errori numerici; ma la simulazione fornisce un risultato approssimato *prima* che l'esperimento sia condotto, e può guidarne la progettazione e la scelta della strumentazione. È così, per esempio, che si prevede l'orbita di un veicolo spaziale o il punto di atterraggio di una capsula di rientro.

In questo capitolo simuliamo due moti *vincolati*: un pendolo la cui massa oscilla in un fluido e un'ala sospesa elasticamente in una galleria del vento.

## 3.2 Il pendolo semplice in un fluido

### 3.2.1 Il modello

Il pendolo è una sfera di diametro $d$ e massa $m$ sospesa a un filo inestensibile e privo di massa, con distanza $l$ fra il perno e il centro della sfera (Fig. 3.1). All'istante $t$ l'angolo con la verticale è $\theta$ e la velocità tangenziale è $v = l\,\mathrm{d}\theta/\mathrm{d}t$. Sulla sfera agiscono il peso ridotto dal galleggiamento $(m - m_f)g$, la tensione del filo (che non compie lavoro) e le forze fluidodinamiche: resistenza viscosa e forza di massa aggiunta, entrambe lungo la tangente. Proiettando la seconda legge di Newton sulla direzione tangenziale:

$$m\frac{\mathrm{d}v}{\mathrm{d}t} = -(m - m_f)\,g\sin\theta - \tfrac{1}{2}m_f\frac{\mathrm{d}v}{\mathrm{d}t} - \tfrac{1}{2}\rho_f\,v|v|\,\tfrac{1}{4}\pi d^2 c_d(v). \tag{3.1}$$

Ponendo $y = l\theta$ (spostamento lungo l'arco) e definendo $A$, $B$, $C$ come nella (2.11), dividendo per $\frac{1}{6}\pi d^3\rho$ si ottiene

$$\frac{\mathrm{d}y}{\mathrm{d}t} = v, \qquad \frac{\mathrm{d}v}{\mathrm{d}t} = \frac{1}{A}\left[-B\sin\frac{y}{l} - C\,v|v|\,c_d(v)\right], \tag{3.2}$$

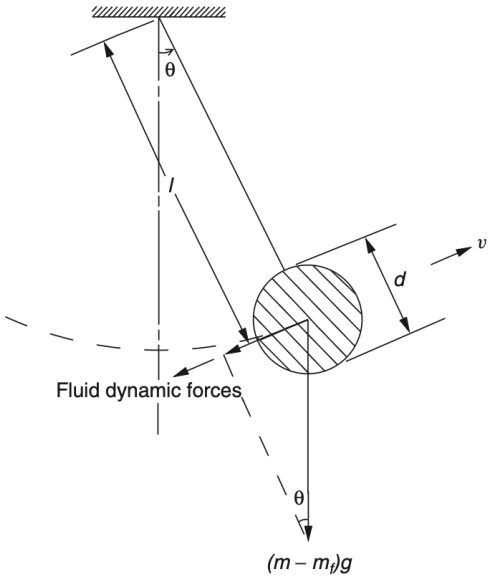



Figura 3.1: Pendolo semplice con massa sferica immersa in un fluido .

con condizioni iniziali $y = y_0 = l\theta_0$ e $v = v_0$ a $t = t_0$. Nel vuoto ($\rho_f = 0$) le equazioni si riducono a $\mathrm{d}v_v/\mathrm{d}t = -g\sin(y_v/l)$: anche in questo caso, se l'ampiezza è grande, la soluzione analitica non è elementare.

**Il periodo del pendolo a grande ampiezza (nel vuoto)**

La conservazione dell'energia, $\frac{1}{2}l^2\dot\theta^2 - gl\cos\theta = -gl\cos\theta_0$, dà $\dot\theta = \sqrt{2g/l}\,\sqrt{\cos\theta - \cos\theta_0}$. Il periodo è quattro volte il tempo per andare da 0 a $\theta_0$:

$$T = 4\sqrt{\frac{l}{2g}}\int_0^{\theta_0}\frac{\mathrm{d}\theta}{\sqrt{\cos\theta-\cos\theta_0}} = 4\sqrt{\frac{l}{g}}\,K(k), \qquad k = \sin\frac{\theta_0}{2},$$

dove $K(k) = \int_0^{\pi/2}\mathrm{d}\varphi/\sqrt{1-k^2\sin^2\varphi}$ è l'integrale ellittico completo di prima specie (la sostituzione è $\sin(\theta/2) = k\sin\varphi$). Sviluppando $K$ in serie: $T = 2\pi\sqrt{l/g}\,\left(1+\frac{1}{4}k^2+\frac{9}{64}k^4+\cdots\right)$. Per $\theta_0 = 45°$, $k = 0{,}3827$ e $T = 1{,}040\cdot T_{\mathrm{lin}}$: il periodo è del 4% più lungo di quello della teoria linearizzata $2\pi\sqrt{l/g}$, in accordo con il valore 2,95 s stimato dal grafico numerico (contro 2,838 s della formula linearizzata). In Python $K$ è `scipy.special.ellipk(k**2)` (attenzione: SciPy vuole come argomento $m = k^2$). La Fig. 3.3 (Problema 3.1) mostra $T/T_{\mathrm{lin}}$ in funzione dell'ampiezza: il periodo diverge per $\theta_0 \to 180°$, perché il pendolo tende a fermarsi nella posizione di equilibrio instabile.

### 3.2.2 Programma 1.2

**Dati**

Sfera di vetro, $\rho = 2500$ kg/m$^3$, $d = 0{,}01$ m, $l = 2$ m; rilascio da fermo con $\theta_0 = 45°$. Fluidi: vuoto ($\rho_f = 0$), aria ($\rho_f = 1{,}22$, $\nu = 1{,}49 \cdot 10^{-5}$), acqua ($\rho_f = 1000$, $\nu = 10^{-6}$). $h = 0{,}1$ s, $t_{\max} = 20$ s. L'angolo è calcolato in radianti e stampato in gradi.

Il programma usa la funzione `integra` del modulo `ode_solvers` in forma vettoriale sullo stato $(y, v)$: è l'equivalente di una subroutine `RUNGE` dei programmi classici, e implementa le (1.19)–(1.20).

prog1_2_pendolo.py

```
"""
prog1_2_pendolo.py  --  Programma 1.2
Pendolo semplice con sfera immersa in un fluido (vuoto, aria, acqua).

    dy/dt = v                       (y = l*theta, spostamento lungo l'arco)
    dv/dt = [ -B sin(y/l) - C v|v| c_d(Re) ] / A
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import ellipk
from ode_solvers import integra, cd_sfera
from stile_grafici import salva, BLU, ARANCIO, VERDE

G = 9.8
d, rho, l = 0.01, 2500.0, 2.0          # sfera di vetro, filo di 2 m
theta0 = 45.0                           # gradi
tmax, h = 20.0, 0.1


def pendolo(rhof, nu):
    """Restituisce t e theta(t) [gradi] per un fluido di densita' rhof e viscosita' nu."""
    rb = rhof / rho
    A, B, C = 1 + rb / 2, (1 - rb) * G, 3 * rb / (4 * d)

    def f(t, s):
        y, v = s
        cd = cd_sfera(v * d / nu) if nu > 0 else 0.0
        return np.array([v, (-B * np.sin(y / l) - C * v * abs(v) * cd) / A])

    y0 = l * np.radians(theta0)
    T, S = integra(f, 0.0, [y0, 0.0], h, int(tmax / h))
    return T, np.degrees(S[:, 0] / l), S[:, 1]


if __name__ == "__main__":
    casi = (("nel vuoto", 0.0, 0.0, BLU), ("in aria", 1.22, 1.49e-5, ARANCIO),
            ("in acqua", 1000.0, 1.0e-6, VERDE))
    fig, ax = plt.subplots(figsize=(5.2, 3.0))
    for nome, rhof, nu, c in casi:
        T, TH, V = pendolo(rhof, nu)
        ax.plot(T, TH, color=c, label=nome)
        # periodo stimato dagli attraversamenti di theta = 0 con v<0 -> v>0 ...
    usiamo i massimi
        idx = [i for i in range(1, len(TH) - 1) if TH[i] > TH[i - 1] and TH[i] >= TH[i + 1]]
        if len(idx) >= 2:
```

```
            print(f"{nome:10s}: primo periodo ~ {T[idx[1]] - T[idx[0]]:.2f} s, "
                  f"ampiezza 2o massimo {TH[idx[1]]:.2f} deg")
        # istante del primo passaggio per la verticale
        i0 = np.where(TH <= 0)[0][0]
        print(f"{'':10s}  primo passaggio per theta=0 a t ~ {T[i0]:.2f} s")
    ax.set_xlabel("$t$ [s]"); ax.set_ylabel("$\\theta$ [gradi]")
    ax.set_ylim(-50, 50); ax.legend(ncol=3, loc="upper right")
    salva(fig, "prog1_2_pendolo")

    # periodo esatto nel vuoto (integrale ellittico completo di prima specie)
    k = np.sin(np.radians(theta0) / 2)
    T_lin = 2 * np.pi * np.sqrt(l / G)
    T_ex = 4 * np.sqrt(l / G) * ellipk(k ** 2)
    print(f"\nPeriodo linearizzato 2 pi sqrt(l/g) = {T_lin:.3f} s;  "
          f"periodo esatto a {theta0} deg = {T_ex:.3f} s
    (+{(T_ex/T_lin-1)*100:.1f} %)")

    # Problema 3.1: periodo in funzione dell'ampiezza
    amp = np.linspace(5, 170, 34)
    fig, ax = plt.subplots(figsize=(4.2, 2.8))
    ax.plot(amp, [4 * np.sqrt(l / G) * ellipk(np.sin(np.radians(a) / 2) ** 2) /
    T_lin
                  for a in amp], color=BLU)
    ax.set_xlabel("ampiezza $\\theta_0$ [gradi]")
    ax.set_ylabel("$T/T_{\\rm lin}$")
    salva(fig, "prog1_2_periodo")
```

Listato 3.1: Programma 1.2 – pendolo in un fluido (`prog1_2_pendolo.py`).

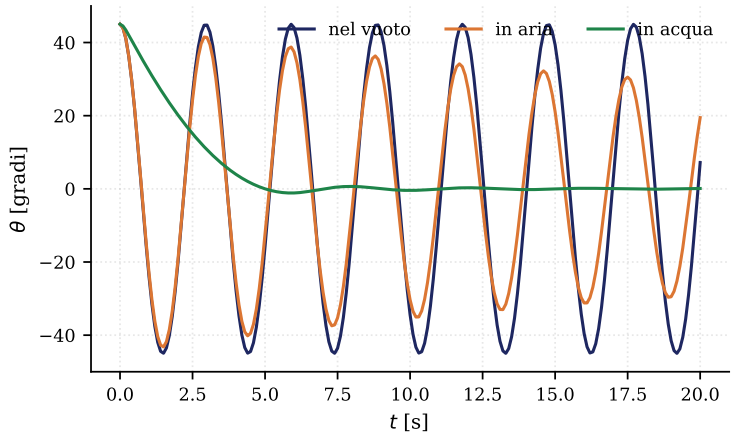



Figura 3.2: Spostamento angolare della sfera di vetro sospesa, in tre mezzi diversi.

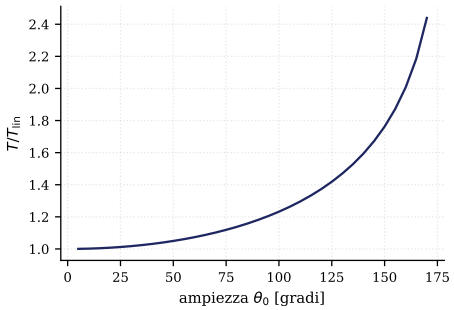



Figura 3.3: Problema 3.1: periodo in funzione dell'ampiezza, rapportato a $2\pi\sqrt{l/g}$.

**Lettura della Fig. 3.2**

- **Nel vuoto** il pendolo oscilla indefinitamente con ampiezza costante di 45° e periodo 2,95 s; la curva $\theta(t)$ non è una cosinusoide (si confrontino le "spalle" dei massimi con quelle di una sinusoide).
- **In aria** l'ampiezza si smorza lentamente per la resistenza aerodinamica (38,8° al secondo massimo, dopo un periodo) e, al diminuire dell'ampiezza, il periodo si accorcia verso il valore linearizzato.

- **In acqua** la densità del fluido è confrontabile con quella del vetro ($\bar\rho = 0{,}4$): $A = 1{,}2$, $B = 0{,}6g$, e soprattutto $C$ è 820 volte più grande che in aria. Il moto è dominato dalla resistenza: la sfera impiega 5 s a raggiungere la verticale (contro 0,74 s nel vuoto e in aria), poi oscilla con ampiezza piccolissima e decrescente e periodo di circa 4 s. È un moto quasi *sovrasmorzato*: il pendolo in acqua è, in pratica, un sedimentatore.

**Problema 3.2: il guscio pieno d'aria in acqua**

Se la sfera è un guscio sottile pieno d'aria e il filo è sostituito da un'asta rigida, priva di massa e di attrito, incernierata al perno, il programma vale ancora con $\rho \simeq 1{,}22$ kg/m$^3$ (il peso del guscio è trascurabile). Ora $\bar\rho = 820$ e $B < 0$: la "gravità efficace" è rivolta verso l'alto, e l'equilibrio stabile del sistema è nella posizione *superiore* ($\theta = 180°$). Partendo da fermo a 5° dalla posizione inferiore, che è ora instabile, il guscio risale lungo l'arco e si assesta oscillando attorno alla verticale superiore. Lo studente verifichi che, con $A = 411$ e $C = 17\,077$ m$^{-1}$ (gli stessi della pallina da ping-pong del Cap. 2), il passo deve essere ridotto a $h \simeq 0{,}01$ s per la stabilità, e stimi con la (2.13) la velocità massima lungo l'arco.

## 3.3 Un sistema aeroelastico elementare: l'ala su molla

### 3.3.1 Il modello

Un'ala elasticamente vincolata vibra in certe condizioni di volo; lo studio analitico passa di solito per teorie linearizzate. Qui simuliamo un sistema aeroelastico semplificato e osserviamo al computer la risposta dell'ala a diverse condizioni di vento.

Un'ala di massa $m$ è montata in galleria del vento (Fig. 3.4) su una molla di costante $k$, che rappresenta la rigidezza flessionale dell'ala di un velivolo; la sezione è un profilo simmetrico calettato all'angolo $\alpha_0$ rispetto all'orizzontale; il baricentro può muoversi solo verticalmente, con spostamento $z$ (positivo verso l'alto) misurato dalla posizione di equilibrio statico in assenza di vento. La galleria fornisce una corrente orizzontale uniforme di velocità $u$. Quando l'ala sale con velocità $v$, l'aria le scorre incontro, relativamente, con velocità verticale $-v$: l'*angolo d'attacco effettivo* è

$$\alpha = \alpha_0 - \Delta\alpha = \alpha_0 - \arctan\frac{v}{u}. \tag{3.3}$$

La portanza, normale alla velocità relativa $\sqrt{u^2+v^2}$, vale

$$L = \tfrac{1}{2}\rho_f\,(u^2+v^2)\,S\,c_l, \tag{3.4}$$

con $S$ la superficie alare, e per la teoria del profilo sottile [8, §5.5]

$$c_l = 2\pi\alpha \qquad \text{per} \quad -18° \le \alpha \le 18°, \qquad c_l = 0 \quad \text{altrimenti (stallo).} \tag{3.5}$$

Le equazioni del moto del baricentro sono

$$\frac{\mathrm{d}z}{\mathrm{d}t} = v, \qquad m\frac{\mathrm{d}v}{\mathrm{d}t} = -kz + L\cos\Delta\alpha, \tag{3.6}$$

dove $L\cos\Delta\alpha$ è la componente verticale della portanza (la componente orizzontale è assorbita dal vincolo). Con $\cos\Delta\alpha = u/\sqrt{u^2+v^2}$ e la (3.4):

$$m\frac{\mathrm{d}v}{\mathrm{d}t} = -kz + \tfrac{1}{2}\rho_f\,S\,c_l\,u\sqrt{u^2+v^2}. \tag{3.7}$$

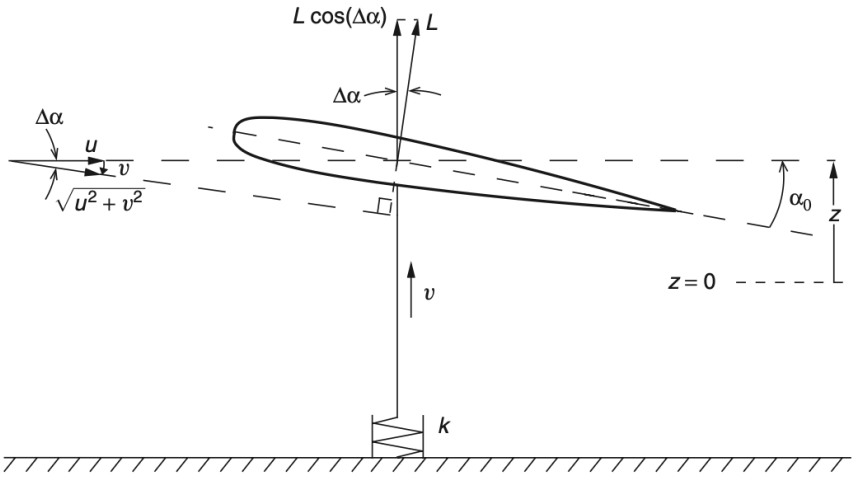



Figura 3.4: Ala vincolata a muoversi verticalmente su una molla in galleria del vento, e triangolo delle velocità.

Il peso non compare perché $z$ è misurato dalla posizione di equilibrio statico, in cui è già equilibrato dalla molla. Si noti che la resistenza aerodinamica non entra: è orizzontale (a meno di termini in $\sin\Delta\alpha$, trascurati) ed è assorbita dal vincolo.

### 3.3.2 Adimensionalizzazione

Il problema contiene sei parametri ($\rho_f$, $g$, $m$, $S$, $k$, $u$) oltre ad $\alpha_0$. Risolverlo per ogni combinazione sarebbe uno spreco: è più economico riscrivere le equazioni in forma adimensionale, in modo che una sola soluzione valga per un'intera classe di problemi. La procedura consiste nello scegliere le *grandezze di riferimento* (un tempo, una lunghezza, una velocità) naturali per il problema:

- tempo di riferimento: il periodo proprio del sistema massa-molla, $2\pi\sqrt{m/k}$;
- lunghezza di riferimento: l'allungamento statico della molla sotto il peso dell'ala, $mg/k$;
- velocità di riferimento: il rapporto fra le due, $(g/2\pi)\sqrt{m/k}$.

Si definiscono quindi le variabili adimensionali

$$T = \frac{t}{2\pi\sqrt{m/k}}, \qquad Z = \frac{z}{mg/k}, \qquad U = \frac{u}{(g/2\pi)\sqrt{m/k}}, \qquad V = \frac{v}{(g/2\pi)\sqrt{m/k}}. \tag{3.8}$$

**Derivazione delle equazioni adimensionali**

Indichiamo con $t_r = 2\pi\sqrt{m/k}$, $l_r = mg/k$ e $v_r = l_r/t_r$ le grandezze di riferimento. Allora $z = l_r Z$, $v = v_r V$, $u = v_r U$, $t = t_r T$ e $\mathrm{d}/\mathrm{d}t = (1/t_r)\,\mathrm{d}/\mathrm{d}T$. La prima delle (3.6) diventa $(l_r/t_r)\,\mathrm{d}Z/\mathrm{d}T = v_r V$, cioè $\mathrm{d}Z/\mathrm{d}T = V$. La (3.7), divisa per $m$:

$$\frac{v_r}{t_r}\frac{\mathrm{d}V}{\mathrm{d}T} = -\frac{k}{m}\,l_r Z + \frac{\rho_f S}{2m}\,c_l\,v_r^2\,U\sqrt{U^2+V^2}.$$

Moltiplicando per $t_r/v_r = t_r^2/l_r$: il coefficiente di $Z$ diventa $(k/m)\,t_r^2 = (2\pi)^2$; quello del termine aerodinamico, $(\rho_f S/2m)\,v_r t_r = (\rho_f S/2m)\,l_r = (\rho_f S/2m)(mg/k) = \rho_f g S/(2k)$.

Si ottiene quindi

$$\frac{\mathrm{d}Z}{\mathrm{d}T} = V, \qquad \frac{\mathrm{d}V}{\mathrm{d}T} = -(2\pi)^2 Z + \beta\, c_l\, U\sqrt{U^2+V^2}, \qquad \beta = \frac{\rho_f\, g\, S}{2k}. \tag{3.9}$$

I sei parametri dimensionali si sono ridotti a *due* gruppi adimensionali, $\beta$ e $U$ (più $\alpha_0$ e l'angolo di stallo). $\beta$ è il rapporto fra la pressione dinamica di riferimento per unità di superficie, $\frac{1}{2}\rho_f v_r^2 S$, e la forza elastica di riferimento $k\, l_r$, calcolato con $v_r$ e $l_r$ sopra definite; $U$ è la velocità del vento misurata in velocità di riferimento. Ogni coppia $(\beta, U)$ rappresenta infinite combinazioni dei parametri dimensionali: ad esempio moltiplicare $m$, $S$ e $k$ per uno stesso fattore non cambia nulla.

**Dati e grandezze di riferimento**

$\rho_f = 1{,}22$ kg/m$^3$, $g = 9{,}8$ m/s$^2$, $m = 3$ kg, $S = 0{,}3$ m$^2$, $k = 980$ kg/s$^2$, $\alpha_0 = 10°$. Ne seguono $\beta = 0{,}00183$, $t_r = 0{,}348$ s, $l_r = 0{,}030$ m, $v_r = 0{,}0862$ m/s; $U = 100$ corrisponde a un vento di 8,62 m/s. Condizioni iniziali: ala ferma nella posizione di equilibrio statico, $Z = V = 0$, quando il vento viene acceso. Passo adimensionale $h = 0{,}01$, $T_{\max} = 20$.

### 3.3.3 Programma 1.3 e risultati

prog1_3_ala_molla.py

```
"""
prog1_3_ala_molla.py  --  Programma 1.3
Moto verticale di un'ala sostenuta da una molla in galleria del vento
(sistema aeroelastico a un grado di liberta', forma adimensionale).

    dZ/dT = V
    dV/dT = -(2 pi)^2 Z + beta c_l(alpha) U sqrt(U^2 + V^2)

    alpha = alpha0 - atan(V/U);   c_l = 2 pi alpha  per |alpha| <= 18 deg, 0
    altrimenti
"""
import numpy as np
import matplotlib.pyplot as plt
from ode_solvers import integra
from stile_grafici import salva, PALETTE, BLU

# dati dimensionali e gruppi adimensionali
rhof, g, m, S, k = 1.22, 9.8, 3.0, 0.3, 980.0
alpha0 = np.radians(10.0)
beta = rhof * g * S / (2 * k)
t_rif = 2 * np.pi * np.sqrt(m / k)
l_rif = m * g / k
v_rif = l_rif / t_rif
print(f"beta = {beta:.5f}, t_rif = {t_rif:.3f} s, l_rif = {l_rif:.3f} m, v_rif =
    {v_rif:.4f} m/s")
print(f"U = 100  <->  u = {100*v_rif:.2f} m/s")

ALPHA_STALLO = np.radians(18.0)


def cl(alpha):
    return 2 * np.pi * alpha if abs(alpha) <= ALPHA_STALLO else 0.0


```

```
def sistema(U_di_T, V_raffica=lambda T: 0.0):
    """Restituisce f(T, s) per un vento orizzontale U(T) e un'eventuale raffica verticale."""
    def f(T, s):
        Z, V = s
        U = U_di_T(T)
        Vr = V + V_raffica(T)            # velocita' verticale relativa all'aria
        alpha = alpha0 - np.arctan2(Vr, U)
        return np.array([V, -(2 * np.pi) ** 2 * Z + beta * cl(alpha) * U * np.hypot(U, Vr)])
    return f


if __name__ == "__main__":
    h, Tmax = 0.01, 20.0
    n = int(Tmax / h)

    # ---------------- Fig. 1.3.4: cinque velocita' del vento
    fig, ax = plt.subplots(figsize=(5.2, 3.0))
    for U, c in zip((25, 50, 75, 100, 125), PALETTE):
        T, S = integra(sistema(lambda T, U=U: U), 0.0, [0.0, 0.0], h, n)
        ax.plot(T, S[:, 0], color=c, label=f"$U={U}$")
        # posizione di equilibrio: (2pi)^2 Z = beta cl(alpha0) U^2
        Zeq = beta * cl(alpha0) * U ** 2 / (2 * np.pi) ** 2
        print(f"U = {U:3d}: Z(20) = {S[-1,0]:.4f}, Z_eq = {Zeq:.4f}, Z_max = {S[:,0].max():.4f}")
    ax.set_xlabel("$T$"); ax.set_ylabel("$Z$"); ax.legend(ncol=5, fontsize=7, loc="upper right")
    ax.set_ylim(0, 1.5)
    salva(fig, "prog1_3_vento")

    # ---------------- Fig. 1.3.5: raffica orizzontale U(1 + a sin(omega T))
    U0, a = 100.0, 0.2
    fig, axs = plt.subplots(3, 1, figsize=(5.2, 4.6), sharex=True)
    for ax, om in zip(axs, (2.0, 1.0, 0.5)):
        T, S = integra(sistema(lambda T, om=om: U0 * (1 + a * np.sin(om * T))), 0.0, [0, 0], h, n)
        ax.plot(T, S[:, 0], color=BLU)
        ax.set_ylabel("$Z$"); ax.set_title(f"$\\omega = {om}$", fontsize=8, loc="right")
        ax.set_ylim(0, 1.6)
    axs[-1].set_xlabel("$T$")
    salva(fig, "prog1_3_raffica_h")

    # ---------------- Fig. 1.3.6: raffica verticale -a U sin(omega T)
    fig, axs = plt.subplots(3, 1, figsize=(5.2, 4.6), sharex=True)
    for ax, om in zip(axs, (2.0, 1.0, 0.5)):
        T, S = integra(sistema(lambda T: U0, lambda T, om=om: -a * U0 * np.sin(om * T)),
                       0.0, [0, 0], h, n)
        ax.plot(T, S[:, 0], color=BLU)
        ax.set_ylabel("$Z$"); ax.set_title(f"$\\omega = {om}$", fontsize=8, loc="right")
        ax.set_ylim(-5, 5)
    axs[-1].set_xlabel("$T$")
    salva(fig, "prog1_3_raffica_v")
```

Listato 3.2: Programma 1.3 – ala su molla in galleria del vento (`prog1_3_ala_molla.py`).

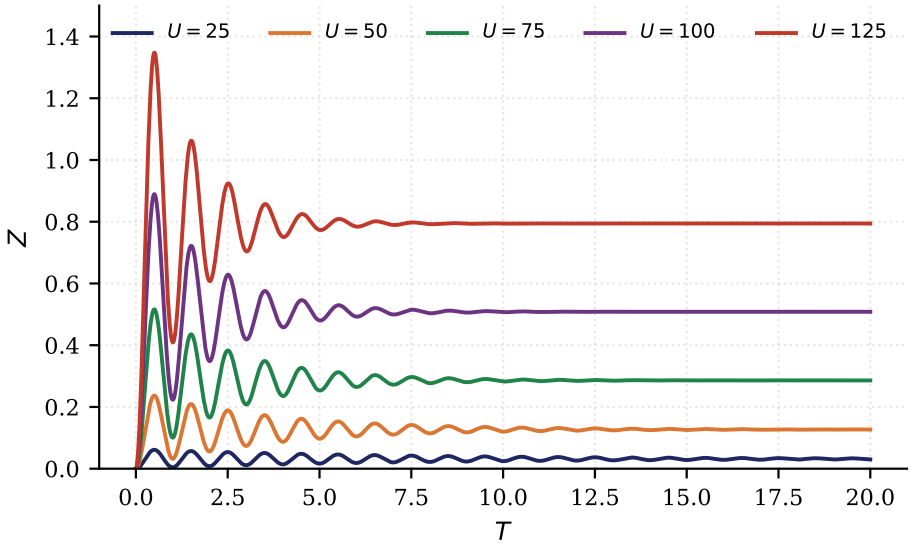



Figura 3.5: Spostamento adimensionale dell'ala per cinque velocità del vento.

**Lettura della Fig. 3.5**

Acceso il vento, la portanza solleva l'ala dalla posizione di equilibrio statico e questa comincia a oscillare; nel frattempo il moto dell'ala modifica l'angolo d'attacco e quindi la portanza. A bassa velocità l'ala oscilla alla frequenza propria del sistema massa-molla ($T = 1$ in unità adimensionali) e l'ampiezza si smorza lentamente; al crescere del vento il periodo aumenta leggermente e lo smorzamento è più rapido. In ogni caso l'ala si assesta su un nuovo equilibrio in cui la portanza è bilanciata dalla molla allungata: $Z_{\text{eq}} = \beta c_l(\alpha_0) U^2/(2\pi)^2$, cioè 0,032, 0,127, 0,286, 0,508 e 0,794 per $U = 25$, 50, 75, 100, 125, esattamente i valori a cui tendono le curve. Il primo massimo è circa il doppio dell'equilibrio ($Z_{\max} = 0{,}89$ per $U = 100$): è il classico *fattore 2 del carico applicato bruscamente*.

**Perché l'aria smorza: lo smorzamento aerodinamico**

Il modello non contiene alcun attrito, eppure le oscillazioni si smorzano. La ragione è nell'angolo d'attacco effettivo (3.3): quando l'ala sale ($V > 0$) l'angolo d'attacco diminuisce, la portanza cala e l'ala è frenata; quando scende, la portanza cresce e la sostiene. Per piccoli $V/U$, $\arctan(V/U) \simeq V/U$ e $U\sqrt{U^2+V^2} \simeq U^2$, per cui la (3.9) si linearizza in

$$\frac{\mathrm{d}^2 Z}{\mathrm{d}T^2} + 2\pi\beta U \frac{\mathrm{d}Z}{\mathrm{d}T} + (2\pi)^2 Z = 2\pi\beta\,\alpha_0\,U^2.$$

È un oscillatore armonico *smorzato* con pulsazione propria $\omega_n = 2\pi$, coefficiente di smorzamento $c = 2\pi\beta U$ e rapporto di smorzamento $\zeta = c/(2\omega_n) = \beta U/2$: per $U = 100$, $\zeta = 0{,}09$; per $U = 25$, $\zeta = 0{,}023$. Lo smorzamento aerodinamico cresce *linearmente con la velocità del vento*, il periodo smorzato $2\pi/(\omega_n\sqrt{1-\zeta^2})$ si allunga leggermente, e per $U \geq 2/\beta \simeq 1090$ ($\zeta \geq 1$) l'ala raggiungerebbe l'equilibrio senza oscillare, come si può verificare con il programma (Esercizio B.9). La posizione di equilibrio $Z_{\text{eq}} = \beta\alpha_0 U^2/(2\pi)$ coincide con quella calcolata sopra. Questo smorzamento aerodinamico del moto di flappeggio è lo stesso che rende stabile, in volo, l'oscillazione flessionale di un'ala rigida in torsione.

**I limiti del modello**

Il coefficiente di portanza è calcolato con la formula *stazionaria* (3.5) usando l'angolo istantaneo fra velocità relativa e corda (approssimazione *quasi-stazionaria*); la portanza reale di un'ala in moto non stazionario richiede la teoria di Theodorsen [14]. Inoltre il grado di libertà torsionale, decisivo nel *flutter*, è stato eliminato: con il solo grado di libertà verticale il sistema è sempre stabile e l'ampiezza non può crescere indefinitamente. Il modello è quindi utile per il *buffeting* (risposta a raffiche), non per il flutter.

### 3.3.4 Risposta alle raffiche

Il progettista vuole conoscere la risposta dell'ala a un vento a raffiche. Sovrapponiamo alla corrente uniforme una fluttuazione orizzontale, cosicché la velocità adimensionale totale sia $U(1 + a \sin \omega T)$: ampiezza $aU$, frequenza $\omega$ volte quella propria dell'ala. Nel programma basta sostituire $U$ con questa espressione (la funzione `sistema` accetta $U$ come funzione di $T$). La Fig. 3.6 mostra i risultati per $U = 100$, $a = 0{,}2$, $\omega = 0{,}5$, 1, 2: in risposta a una raffica di qualunque frequenza l'ala tenta dapprima di vibrare alla propria frequenza naturale, poi si assesta su un moto periodico (non sinusoidale) con frequenza pari a quella della raffica, la cui ampiezza *cresce* con $\omega$.

Per una raffica *verticale* di velocità adimensionale $-aU \sin \omega T$ si sostituisce $V$ con $V + aU \sin \omega T$ nella funzione $F$ (la velocità verticale relativa all'aria cambia, non quella dell'ala rispetto alla molla). La Fig. 3.7 mostra che in tutti e tre i casi l'ala vibra alla propria frequenza naturale con ampiezza modulata nel tempo; il moto, apparentemente irregolare, si ripete con un periodo che cresce al diminuire di $\omega$ (battimento fra la frequenza propria e quella della raffica), e l'ampiezza finale *decresce* con $\omega$.

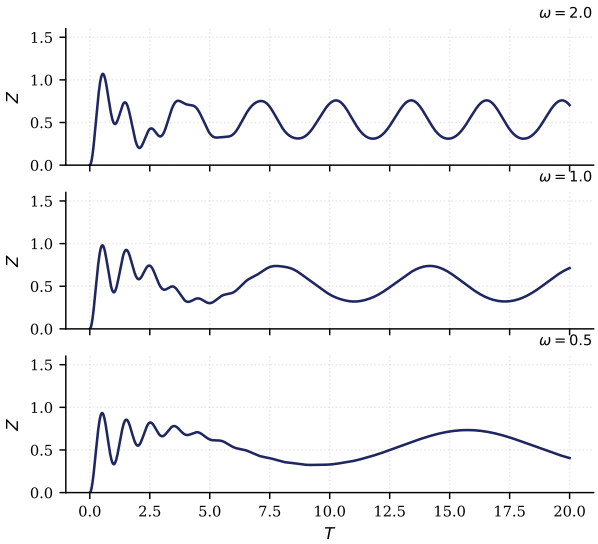



Figura 3.6: Risposta a un vento orizzontale $U(1 + a \sin \omega T)$, $U = 100$, $a = 0{,}2$.

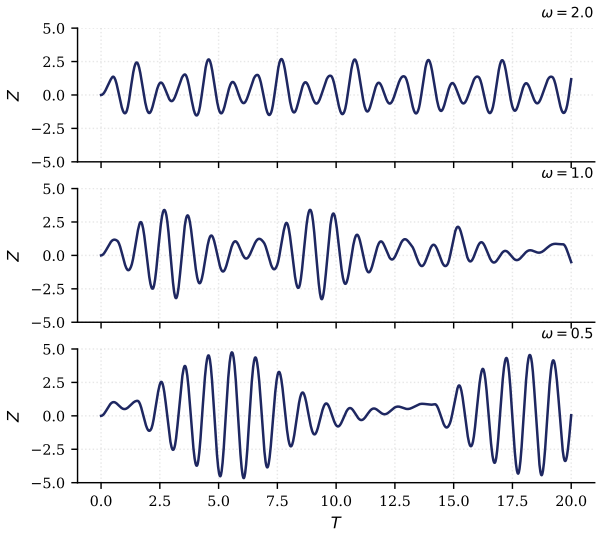



Figura 3.7: Risposta a un vento orizzontale uniforme $U = 100$ con raffica verticale $0{,}2U \sin \omega T$.

In nessuno dei due casi si ha *risonanza* quando la frequenza della raffica coincide con quella propria: l'ampiezza non diverge, grazie allo smorzamento aerodinamico e alla non linearità (stallo oltre 18°). Il programma si generalizza facilmente a un'atmosfera turbolenta con fluttuazioni in entrambe le direzioni, esprimibili come serie di Fourier.

**Problema 3.3: la linea elettrica nel vento**

Sostituendo nel pendolo la sfera con un cilindro orizzontale e aggiungendo un vento orizzontale oscillante $u = 30(1 + 0{,}2\sin\omega t)$ m/s si ottiene un modello grossolano del moto di un conduttore di una linea elettrica aerea (filo di rame, $\rho = 8950$ kg/m$^3$, $d = 5$ mm, $l = 0{,}3$ m, $c_d = 1{,}2$ costante nel campo $10^4 < Re < 1{,}5 \cdot 10^5$). Poiché il rame è molto più denso dell'aria, galleggiamento e massa aggiunta sono trascurabili; la componente tangenziale della forza viscosa è $\frac{1}{2}\rho_f d c_d\,(u\cos\theta - v)\sqrt{(u\sin\theta)^2 + (u\cos\theta - v)^2}$ per unità di lunghezza, con segno controllato da $(u\cos\theta - v)$. La struttura del Programma 1.2 resta valida: cambiano la funzione $F$ e le costanti. È un esercizio proposto al lettore, con $\omega = 0,\ 0{,}5,\ 1,\ 2$.

## 3.4 Sintesi del capitolo

**Da ricordare**

- Le equazioni di un moto vincolato in un fluido si ottengono proiettando la seconda legge di Newton sulla direzione libera; le forze fluidodinamiche (resistenza, massa aggiunta, portanza) dipendono dalla velocità *relativa* fra corpo e fluido.
- Il periodo del pendolo cresce con l'ampiezza: $T = 4\sqrt{l/g}\,K(\sin\theta_0/2)$; $+4\%$ a $45^\circ$.
- Adimensionalizzare significa scegliere tempo, lunghezza e velocità di riferimento intrinseci al problema; il numero di parametri si riduce ai gruppi adimensionali indipendenti ($\beta$, $U$).
- Il moto verticale dell'ala è smorzato aerodinamicamente con $\zeta = \beta U/2$: la variazione di angolo d'attacco indotta dalla velocità dell'ala si oppone al moto.
- Un modello a un solo grado di libertà non può prevedere il flutter, ma descrive la risposta alle raffiche.

# Capitolo 4

# Moti bidimensionali e balistica di un proiettile sferico

## 4.1 Runge–Kutta per il moto piano di un corpo in un fluido

Consideriamo il moto di traslazione di un corpo nel piano $x$-$y$, con $y$ diretto verso l'alto (opposto a $g$), Fig. 4.1. Il corpo ha velocità $\mathbf{w} = (u, v)$ rispetto al sistema fisso; il fluido, in generale in moto, ha nella posizione del corpo velocità $\mathbf{w}_f = (u_f, v_f)$. Le forze fluidodinamiche sono determinate dalla velocità del fluido *relativa* al corpo, $\mathbf{w}_r = \mathbf{w}_f - \mathbf{w}$: è questa, non $\mathbf{w}$, che compare nel numero di Reynolds e nella pressione dinamica.

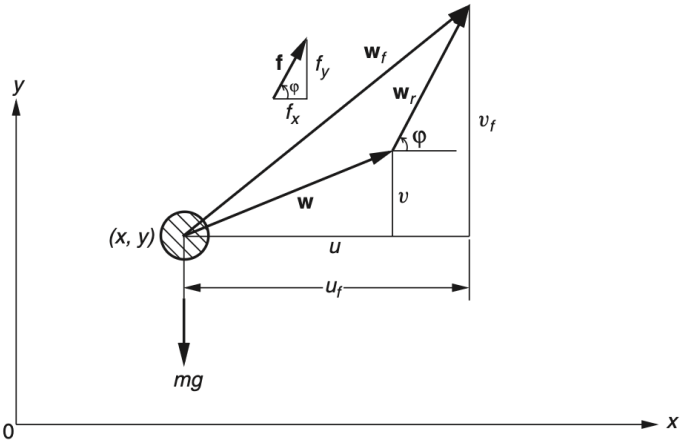



Figura 4.1: Moto bidimensionale di un corpo in un fluido in moto. La forza fluidodinamica $\mathbf{f}$ è allineata con la velocità relativa $\mathbf{w}_r$ per un corpo non rotante.

Se $\mathbf{f} = (f_x, f_y)$ è la risultante delle forze fluidodinamiche, esclusa quella di massa aggiunta $m'$, le equazioni del moto di un corpo di massa $m$ sono

$$(m + m')\frac{\mathrm{d}^2 x}{\mathrm{d}t^2} = f_x, \qquad (m + m')\frac{\mathrm{d}^2 y}{\mathrm{d}t^2} = -(m - m_f)\, g + f_y, \tag{4.1}$$

con $m_f$ la massa di fluido spostata. Poiché $\mathbf{f}$ dipende in generale da posizione, velocità e tempo, il sistema si scrive nella forma di quattro equazioni del primo ordine:

$$\frac{\mathrm{d}x}{\mathrm{d}t} = u, \quad \frac{\mathrm{d}u}{\mathrm{d}t} = F_1(x, y, u, v, t), \qquad \frac{\mathrm{d}y}{\mathrm{d}t} = v, \quad \frac{\mathrm{d}v}{\mathrm{d}t} = F_2(x, y, u, v, t). \tag{4.2}$$

Date posizione $(x_0, y_0)$ e velocità $(u_0, v_0)$ all'istante $t_0$, si cercano traiettoria e moto per $t > t_0$. Le formule RK4 sono l'estensione diretta delle (1.20): sedici incrementi, quattro per variabile, calcolati a gruppi di quattro:

$$\begin{array}{llll} \Delta_1 x = hu, & \Delta_1 y = hv, & \Delta_1 u = hF_1(x,y,u,v,t), & \Delta_1 v = hF_2(x,y,u,v,t), \\ \Delta_2 x = h\left(u + \tfrac{1}{2}\Delta_1 u\right), & \Delta_2 y = h\left(v + \tfrac{1}{2}\Delta_1 v\right), & \Delta_2 u = hF_1(\ldots)_{1/2}, & \Delta_2 v = hF_2(\ldots)_{1/2}, \\ \Delta_3 x = h\left(u + \tfrac{1}{2}\Delta_2 u\right), & \Delta_3 y = h\left(v + \tfrac{1}{2}\Delta_2 v\right), & \Delta_3 u = hF_1(\ldots)'_{1/2}, & \Delta_3 v = hF_2(\ldots)'_{1/2}, \\ \Delta_4 x = h\left(u + \Delta_3 u\right), & \Delta_4 y = h\left(v + \Delta_3 v\right), & \Delta_4 u = hF_1(\ldots)_1, & \Delta_4 v = hF_2(\ldots)_1, \end{array} \tag{4.3}$$

dove $(\ldots)_{1/2}$ indica gli argomenti $(x + \frac{1}{2}\Delta_1 x,\ y + \frac{1}{2}\Delta_1 y,\ u + \frac{1}{2}\Delta_1 u,\ v + \frac{1}{2}\Delta_1 v,\ t + \frac{1}{2}h)$, $(\ldots)'_{1/2}$ gli stessi con gli incrementi $\Delta_2$, e $(\ldots)_1$ gli argomenti $(x + \Delta_3 x, \ldots, t + h)$; infine $x \leftarrow x + \frac{1}{6}(\Delta_1 x + 2\Delta_2 x + 2\Delta_3 x + \Delta_4 x)$ e analogamente per $y$, $u$, $v$.

**Dalla subroutine `KUTTA` alla forma vettoriale**

Nei programmi classici in FORTRAN o MATLAB la (4.3) è una subroutine del tipo `KUTTA(X,Y,U,V,T,DT,FX,FY)`, a cui si allegano due funzioni `FX` e `FY` che definiscono $F_1$ e $F_2$. In Python l'intero sistema (4.2) è un'unica funzione `f(t, s)` che riceve lo stato $\mathbf{s} = (x, y, u, v)$ e restituisce $(u,\ v,\ F_1,\ F_2)$: la funzione `passo_rk4` del Listato 1.2 esegue *esattamente* i sedici incrementi della (4.3), perché le operazioni `y + 0.5 * k1` agiscono su tutte le componenti insieme. Il vantaggio è che lo stesso integratore serve per due, quattro o ottanta variabili (Cap. 6).

## 4.2 Balistica di un proiettile sferico

È noto che un proiettile lanciato verso l'alto in direzione non verticale descrive una parabola; ma la conclusione vale nel vuoto. Per studiare l'effetto del fluido torniamo alle (4.1). Se la sfera non ruota (niente effetto Magnus), $\mathbf{f}$ è la sola resistenza viscosa, diretta come $\mathbf{w}_r$, che forma con l'asse $x$ l'angolo $\varphi$ (Fig. 4.1): $\cos\varphi = (u_f - u)/w_r$, $\sin\varphi = (v_f - v)/w_r$, con $w_r = \sqrt{(u_f - u)^2 + (v_f - v)^2}$. Il modulo della resistenza è $\frac{1}{2}\rho_f w_r^2 \frac{1}{4}\pi d^2 c_d$, quindi

$$f_x = \tfrac{1}{8}\pi\rho_f d^2 c_d\,(u_f - u)\,w_r, \qquad f_y = \tfrac{1}{8}\pi\rho_f d^2 c_d\,(v_f - v)\,w_r, \tag{4.4}$$

dove i segni sono automaticamente quelli di $(u_f - u)$ e $(v_f - v)$, e $c_d$ è il coefficiente della sfera alla velocità $w_r$ (funzione `cd_sfera` del Cap. 2). Con $m' = \frac{1}{2}m_f$ e le densità, le (4.1) diventano

$$\frac{\mathrm{d}^2 x}{\mathrm{d}t^2} = \frac{C\,c_d\,(u_f - u)\,w_r}{A} \equiv F_X, \qquad \frac{\mathrm{d}^2 y}{\mathrm{d}t^2} = \frac{-B + C\,c_d\,(v_f - v)\,w_r}{A} \equiv F_Y, \tag{4.5}$$

con $A$, $B$, $C$ dati dalla (2.11). Si noti la perfetta analogia con la (2.10): la caduta verticale è il caso particolare $u = u_f = 0$.

**Dati del Programma 1.4**

Sfera d'acciaio $d = 0{,}05$ m in aria ($\rho$, $\rho_f$, $\nu$, $g$ come nel Cap. 2); lancio dall'origine con $w_0 = 50$ m/s ed elevazione $\theta_0 = 30^\circ, 45^\circ, 60^\circ$; fluido in quiete ($u_f = v_f = 0$); $h = 0{,}1$ s. Il calcolo si arresta quando il proiettile ricade alla quota iniziale, cioè al primo passo con $y \le 0$ (e $v < 0$).

prog1_4_proiettile.py

```
"""
prog1_4_proiettile.py  --  Programma 1.4
Moto bidimensionale di un proiettile sferico in un fluido (RK4 per sistemi).

Stato s = [x, y, u, v];  vento (uf, vf) assegnato dalla funzione vento(x, y, t).

    d2x/dt2 = FX = C c_d (uf - u) w_r / A
    d2y/dt2 = FY = [ -B + C c_d (vf - v) w_r ] / A ,   w_r = |w_f - w|
"""
import numpy as np
import matplotlib.pyplot as plt
from ode_solvers import passo_rk4, cd_sfera
from stile_grafici import salva, PALETTE, BLU, ARANCIO, VERDE

RHO, RHOF, NU, G = 8000.0, 1.22, 1.49e-5, 9.8


def modello(d, rhof=RHOF, vento=lambda x, y, t: (0.0, 0.0)):
    """Costruisce f(t, s) per una sfera di diametro d in un fluido di densita'
    rhof."""
    rb = rhof / RHO
    A, B, C = 1 + rb / 2, (1 - rb) * G, 3 * rb / (4 * d)

    def f(t, s):
        x, y, u, v = s
        uf, vf = vento(x, y, t)
        wr = np.hypot(uf - u, vf - v)
        cd = cd_sfera(wr * d / NU)
        return np.array([u, v,
                         C * cd * (uf - u) * wr / A,
                         (-B + C * cd * (vf - v) * wr) / A])
    return f


def traiettoria(f, w0, theta0_deg, h=0.1):
    """Integra finche' il proiettile non ricade a y <= 0; restituisce la storia e i
    punti P, Q."""
    th = np.radians(theta0_deg)
    t, s = 0.0, np.array([0.0, 0.0, w0 * np.cos(th), w0 * np.sin(th)])
    storia = [(t, *s)]
    while s[1] >= 0.0:
        s = passo_rk4(f, t, s, h)
        t += h
        storia.append((t, *s))
    sQ = s
    sP = passo_rk4(f, t, s, -h)            # un passo all'indietro: punto P
    return np.array(storia), sP, sQ


def gittata(sP, sQ):
    """Eq. (1.5.3): intersezione della corda PQ con l'asse x."""
    xP, yP, xQ, yQ = sP[0], sP[1], sQ[0], sQ[1]
    return (xQ * yP - xP * yQ) / (yP - yQ)


if __name__ == "__main__":
    d, w0 = 0.05, 50.0
    fig, ax = plt.subplots(figsize=(5.2, 3.0))
    print(f"Sfera d'acciaio d = {d} m, w0 = {w0} m/s, h = 0.1 s")
    for th, c in zip((30, 45, 60), (BLU, ARANCIO, VERDE)):
```

```
        S, sP, sQ = traiettoria(modello(d), w0, th)
        Sv, sPv, sQv = traiettoria(modello(d, rhof=0.0), w0, th)
        ax.plot(S[:, 1], S[:, 2], color=c, label=f"$\\theta_0 = {th}^\\circ$")
        ax.plot(Sv[:, 1], Sv[:, 2], "--", color=c, lw=0.9)
        xr_teor = w0 ** 2 * np.sin(2 * np.radians(th)) / G
        print(f"  theta0 = {th:2d}: in aria  x_r = {gittata(sP, sQ):7.2f} m, "
              f"y_max = {S[:,2].max():6.2f} m, t_volo = {S[-1,0]:5.2f} s | "
              f"nel vuoto x_r = {gittata(sPv, sQv):7.2f} m (teorico {xr_teor:.2f})")
    ax.plot([], [], "k--", lw=0.9, label="nel vuoto")
    ax.set_xlabel("$x$ [m]"); ax.set_ylabel("$y$ [m]")
    ax.set_xlim(0, 300); ax.set_ylim(0, 100); ax.set_aspect("equal")
    ax.legend(fontsize=7)
    salva(fig, "prog1_4_traiettorie")

    # Problema 4.1: vento di 20 m/s a favore e contro (theta0 = 45)
    print("\nProblema 4.1 -- vento orizzontale, theta0 = 45 deg:")
    fig, ax = plt.subplots(figsize=(5.2, 2.8))
    for uf, c in ((20.0, VERDE), (0.0, BLU), (-20.0, ARANCIO)):
        S, sP, sQ = traiettoria(modello(d, vento=lambda x, y, t, uf=uf: (uf, 0.0)),
    w0, 45)
        ax.plot(S[:, 1], S[:, 2], color=c, label=f"$u_f = {uf:+.0f}$ m/s")
        print(f"  u_f = {uf:+5.1f} m/s -> x_r = {gittata(sP, sQ):7.2f} m")
    ax.set_xlabel("$x$ [m]"); ax.set_ylabel("$y$ [m]"); ax.set_aspect("equal")
    ax.legend(fontsize=7)
    salva(fig, "prog1_4_vento")

    # verifica di reversibilita': integrazione all'indietro fino a t = 0
    S, sP, sQ = traiettoria(modello(d), w0, 45)
    f = modello(d)
    t, s = S[-1, 0], S[-1, 1:].copy()
    n = len(S) - 1
    for _ in range(n):
        s = passo_rk4(f, t, s, -0.1); t -= 0.1
    print(f"\nIntegrazione all'indietro di {n} passi: t = {t:.2e}, "
          f"x = {s[0]:.2e} m, y = {s[1]:.2e} m, u = {s[2]:.6f}, v = {s[3]:.6f}")
```

Listato 4.1: Programma 1.4 – traiettoria di un proiettile sferico (`prog1_4_proiettile.py`).

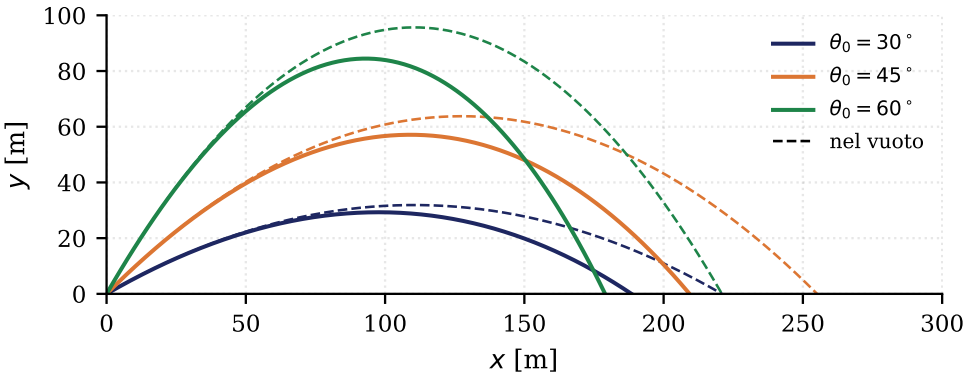



Figura 4.2: Traiettorie di una sfera d'acciaio di 5 cm lanciata a 50 m/s in aria (linee continue) e nel vuoto (tratteggiate).

Tabella 4.1: Gittata $x_r$, quota massima e tempo di volo per $d = 5$ cm, $w_0 = 50$ m/s. Nel vuoto $x_r = w_0^2 \sin 2\theta_0 / g$.

| $\theta_0$ [°] | $x_r$ aria [m] | $y_{\max}$ aria [m] | $t_{\text{volo}}$ aria [s] | $x_r$ vuoto [m] |
|---|---|---|---|---|
| 30 | 188,45 | 29,30 | 4,9 | 220,92 |
| 45 | 209,29 | 57,11 | 6,9 | 255,10 |
| 60 | 178,99 | 84,46 | 8,4 | 220,92 |

**Lettura della Fig. 4.2**

La resistenza dell'aria riduce la gittata, la quota massima e il tempo di volo. Nel vuoto le gittate a 30° e 60° sono uguali (angoli complementari); in aria la traiettoria a 60°, che resta in volo più a lungo, subisce più a lungo la resistenza e ha gittata minore (179 contro 188 m). Le traiettorie in aria non sono simmetriche: il ramo discendente è più ripido di quello ascendente, perché il proiettile arriva al vertice con velocità ridotta.

**Problema 4.1: l'effetto del vento**

Le componenti $u_f, v_f$ sono definite nella funzione `vento(x, y, t)` passata a `modello`, e possono descrivere qualsiasi campo di vento stazionario o no. Con un vento orizzontale di $\pm 20$ m/s e $\theta_0 = 45°$ il programma dà $x_r = 235{,}9$ m (vento a favore), 209,3 m (calma), 171,6 m (vento contrario): un vento di 20 m/s sposta la gittata di circa $\pm 15\%$. Si osservi che l'effetto del vento *non* è semplicemente "trasporto" ($u_f t_{\text{volo}} = \pm 138$ m): il vento cambia la velocità relativa e quindi la resistenza, e l'effetto netto è molto minore.

## 4.3 Il calcolo della gittata

Il Programma 1.4 calcola posizione e velocità a passi di tempo costanti, per cui la gittata non compare esplicitamente nell'uscita: l'ultimo punto calcolato, $Q$, è il primo che cade *sotto* l'orizzonte. Il punto precedente, $P$, è ancora sopra. Il tratto $PQ$ è in generale curvo, ma per $h$ piccolo si può approssimare con un segmento (Fig. 4.3): la gittata $x_r$ è l'ascissa in cui il segmento $PQ$ interseca l'asse $x$. Dalla similitudine dei triangoli,

$$\frac{y_P}{x_r - x_P} = \frac{-y_Q}{x_Q - x_r} \quad \Longrightarrow \quad x_r = \frac{x_Q y_P - x_P y_Q}{y_P - y_Q}, \tag{4.6}$$

dove $y_Q \leq 0$. Si tratta di un'interpolazione lineare inversa, con errore $\mathcal{O}(h^2)$, più che sufficiente rispetto all'errore $\mathcal{O}(h^4)$ dell'integrazione.

Quando il programma raggiunge $Q$, i dati del punto $P$ sono già stati sovrascritti (le formule RK non conservano la storia). Il modo più elegante per recuperarli è chiamare l'integratore con passo $-h$: *integrare all'indietro di un passo* (`passo_rk4(f, t, s, -h)` nel Listato 4.1). Le formule RK4 valgono per $h$ di qualunque segno.

**Un test di accuratezza: andare e tornare**

Se si continua a integrare all'indietro da $P$ fino a riportare il tempo al valore iniziale, lo scarto fra le condizioni calcolate e quelle assegnate a $t = 0$ misura l'accuratezza del metodo. Il Listato 4.1 esegue questo test per $\theta_0 = 45°$: dopo 69 passi avanti e 69 indietro, $x = -3 \cdot 10^{-10}$ m, $y = -8 \cdot 10^{-10}$ m, e le velocità coincidono con quelle iniziali

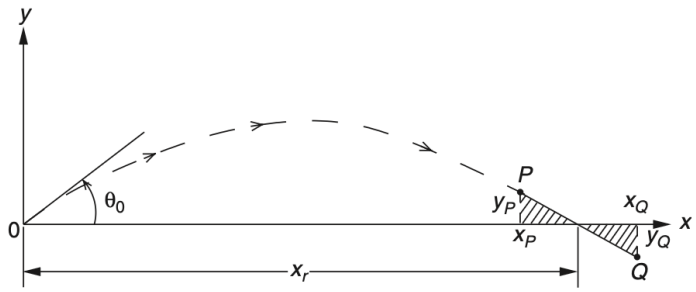



Figura 4.3: Determinazione della gittata: $P$ e $Q$ sono gli ultimi due punti calcolati; i triangoli ombreggiati sono simili.

(35,355339 m/s) a sei decimali. Si noti che la reversibilità del calcolo *non* dimostra che la soluzione sia esatta (un metodo potrebbe essere reversibile e inaccurato), ma è un controllo prezioso della correttezza del programma e dell'assenza di perdite di cifre significative.

## 4.4 La gittata massima: il metodo del dimezzamento dell'intervallo

Nel vuoto la gittata è massima per $\theta_0 = 45°$. È ancora vero nel fluido? La gittata (4.6) è una funzione $x_r(\theta_0)$ che possiamo calcolare punto per punto, ma non derivare analiticamente: serve un algoritmo che ne trovi il massimo *automaticamente*. Usiamo un metodo di ricerca elementare e robusto, il **dimezzamento dell'intervallo** (*half-interval method*), illustrato in Fig. 4.4.

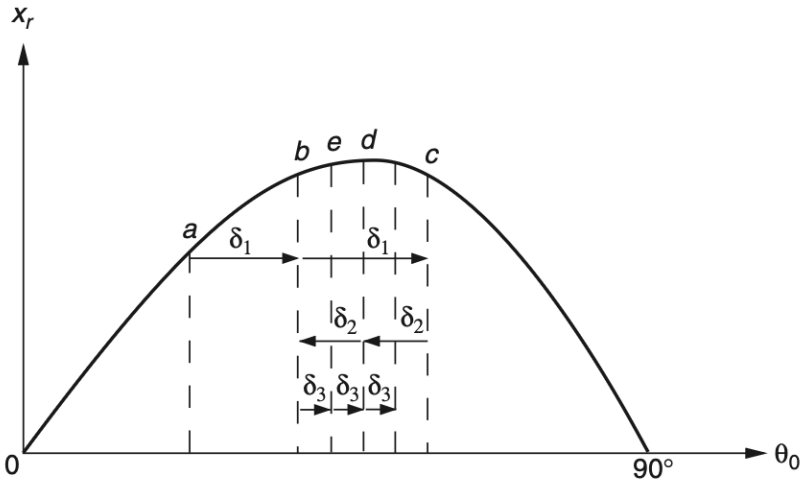



Figura 4.4: Ricerca del massimo della gittata con il metodo del dimezzamento dell'intervallo.

Si parte da un punto $a$ e si calcola un secondo punto $b$ con $(\theta_0)_b = (\theta_0)_a + \delta_1$. Se $(x_r)_a < (x_r)_b$ il massimo è a destra, e si procede con lo stesso passo a un punto $c$; se $(x_r)_b < (x_r)_c$ si continua. Appena la gittata *non* cresce più, $(x_r)_b \geq (x_r)_c$, il massimo è stato superato: si cambia l'incremento in $\delta_2 = -\delta_1/2$ e si torna indietro di mezzo passo al punto $d$. Da $d$, con lo stesso criterio, si prosegue o si inverte di nuovo dimezzando. A ogni inversione l'intervallo che contiene il massimo si dimezza, e il processo si arresta quando $|\delta|$ scende sotto una tolleranza $\varepsilon$.

### Algoritmo del dimezzamento dell'intervallo

1. Si parte da un punto $(\theta_{\text{old}}, x_{\text{old}})$ e da un incremento $\delta$ (qui $\theta_{\text{old}} = 0$, $\delta = 10°$).
2. Si calcola $\theta_{\text{new}} = \theta_{\text{old}} + \delta$ e, integrando la traiettoria, la gittata $x_{\text{new}}$.
3. Se $x_{\text{new}} > x_{\text{old}}$ si accetta il nuovo punto: $\theta_{\text{old}} \leftarrow \theta_{\text{new}}$, $x_{\text{old}} \leftarrow x_{\text{new}}$, e $\delta$ resta invariato. Altrimenti si sostituisce $\delta \leftarrow -\delta/2$ (il vecchio punto resta).
4. Si ripete dal passo 2 finché $|\delta| < \varepsilon$.
5. Fra gli ultimi due punti si sceglie quello con gittata maggiore: è la gittata massima, e il corrispondente $\theta_0$ è l'elevazione ottima. L'accuratezza è controllata da $\varepsilon$.

Con modifiche minime il metodo trova il minimo o gli zeri di una funzione (si veda il progetto in fondo al capitolo): è una forma di ricerca per *bisezione* che non richiede derivate, costa una valutazione di funzione per passo e converge sempre, anche se lentamente (un bit di accuratezza per inversione).

```python
def gittata_massima(f, w0, h=0.1, theta_old=0.0, xr_old=0.0, delta=10.0, eps=1e-3,
                    verbose=False):
    n = 0
    while abs(delta) >= eps:
        theta_new = theta_old + delta
        n += 1
        _, sP, sQ = traiettoria(f, w0, theta_new, h)    # integra fino a y <= 0
        xr_new = gittata(sP, sQ)                         # eq. (1.5.3)
        if xr_old >= xr_new:
            delta = -delta / 2.0                         # massimo superato: inverti
    e dimezza
        else:
            theta_old, xr_old = theta_new, xr_new        # accetta il nuovo punto
    if xr_old >= xr_new:
        return theta_old, xr_old, n
    return theta_new, xr_new, n
```

prog1_5_gittata_max.py

Listato 4.2: La funzione `gittata_massima` del Programma 1.5 (`prog1_5_gittata_max.py`).

### Dati del Programma 1.5

Per esaltare l'effetto dell'aria si considera una sfera d'acciaio più piccola, $d = 0{,}01$ m, con $w_0 = 50$ m/s; tre condizioni di vento orizzontale, $u_f = 20, 0, -20$ m/s; $h = 0{,}1$ s, $\delta = 10°$, $\varepsilon = 0{,}001°$. Poiché il vento varia da caso a caso, non è più definito dentro le funzioni $F_X$, $F_Y$ (come nel Programma 1.4) ma passato dall'esterno: in Python tramite la funzione `vento` catturata nella chiusura di `modello`, l'equivalente delle variabili globali dei linguaggi procedurali.

### Interpretazione

Per il proiettile di 1 cm gli angoli ottimi sono *tutti inferiori a* 45°: il più alto con vento a favore, il più basso con vento contrario. La spiegazione è energetica. Nel vuoto il tiro a 45° dà la gittata massima ma anche un tempo di volo lungo; in aria, un angolo più basso riduce il tempo di esposizione alla resistenza, dissipa meno energia cinetica e, in condizioni opportune, la perdita di distanza orizzontale è minore (si ricordi la

Tabella 4.2: Traccia del metodo del dimezzamento per $u_f = 0$: a ogni riga una traiettoria integrata. Il massimo è superato a $n = 5$; da lì in poi $\delta$ cambia segno e si dimezza a ogni inversione.

| $n$ | $\theta_{\text{new}}$ [°] | $x_r$ [m] | $\delta$ [°] |
|---|---|---|---|
| 1 | 10,0000 | 66,4370 | 10,0000 |
| 2 | 20,0000 | 104,2470 | 10,0000 |
| 3 | 30,0000 | 124,1647 | 10,0000 |
| 4 | 40,0000 | 130,1997 | 10,0000 |
| 5 | 50,0000 | 124,1985 | 10,0000 |
| 6 | 35,0000 | 128,7676 | −5,0000 |
| 7 | 42,5000 | 129,7892 | 2,5000 |
| 8 | 38,7500 | 130,1298 | −1,2500 |
| 9 | 40,6250 | 130,1677 | 0,6250 |
| 10 | 39,6875 | 130,2013 | −0,3125 |
| ⋮ | ⋮ | ⋮ | ⋮ |
| 23 | 39,7437 | 130,2014 | −0,0012 |

Tabella 4.3: Uscita del Programma 1.5: angolo ottimo, gittata massima e numero $N$ di traiettorie integrate.

| $u_f$ [m/s] | $\theta_{\text{ott}}$ [°] | $x_{r,\max}$ [m] | $N$ |
|---|---|---|---|
| 20 | 44,611 | 189,76 | 24 |
| 0 | 39,745 | 130,20 | 23 |
| −20 | 27,371 | 75,61 | 26 |

coppia 30°/60° di Fig. 4.2). Un vento a favore *trasporta* il proiettile: alzare il tiro aumenta il tempo di contatto con l'aria e quindi la gittata. Con vento contrario, al contrario, conviene abbassare il tiro per ridurre l'azione frenante. È l'esperienza di ogni golfista: contro vento, palla bassa.

### 4.4.1 L'angolo ottimo in funzione del diametro

Gli angoli ottimi non sono sempre inferiori a 45°: dipendono dalle dimensioni del proiettile. Facendo variare $d$ nel Programma 1.5 si ottiene la Fig. 4.5.

Per proiettili piccoli con vento a favore l'angolo ottimo supera 45° (fino a 46,6° per $d =$ 2,5 mm): il vento sostiene il proiettile tanto a lungo quanto più questo resta in aria. Al crescere del diametro l'influenza dell'aria diminuisce e tutte le curve tendono a 45°. Sulla curva del vento contrario compare una *brusca depressione* nella zona in cui il numero di Reynolds, calcolato con la velocità relativa $w_0 + |u_f| = 70$ m/s, comincia a superare $3 \cdot 10^5$ (cioè $d \gtrsim 6$ cm): è ancora la crisi della resistenza di Fig. 2.2, che con vento contrario viene raggiunta per diametri minori perché la velocità relativa è maggiore.

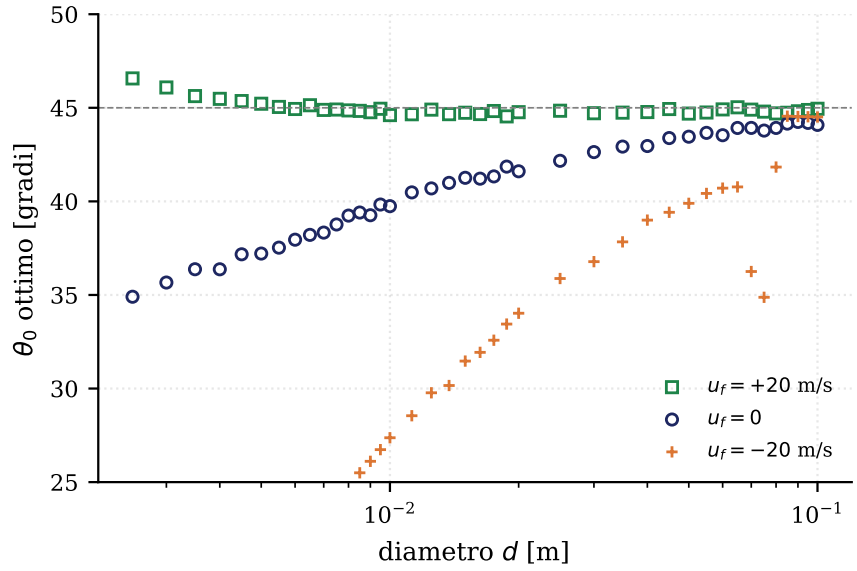



Figura 4.5: Angolo di tiro ottimo per una sfera d'acciaio lanciata a 50 m/s, in funzione del diametro, per tre condizioni di vento.

**Problema 4.2: quando conviene tirare controvento**

Contro il senso comune, in certe condizioni la gittata massima è *maggiore* tirando controvento. Per una sfera di 9 cm il programma dà: $x_{r,\max} = 243{,}8$ m con vento a favore ($\theta_{\text{ott}} = 44{,}8°$), 227,3 m in calma (44,3°), 244,2 m controvento (44,5°). Il numero di Reynolds iniziale spiega il paradosso: $0{,}93 \cdot 10^5$ con vento a favore e $2{,}16 \cdot 10^5$ in calma (entrambi nel plateau $c_d = 0{,}5$), contro $3{,}36 \cdot 10^5$ controvento, *oltre* la crisi della resistenza: nella prima parte del volo il proiettile controvento ha $c_d \simeq 0{,}08$–$0{,}09$, sei volte minore, e il vantaggio compensa abbondantemente il trasporto sfavorevole del vento.

### 4.4.2 Problema 4.3: il proietto d'artiglieria e l'atmosfera stratificata

Durante la prima guerra mondiale si scoprì per caso che, puntando un cannone più in alto di quanto si ritenesse ottimale, la gittata aumentava notevolmente: il proietto raggiunge quote dove l'aria è rarefatta e la resistenza minore. Per verificarlo si considera un proietto equivalente a una sfera d'acciaio di $d = 0{,}3$ m, con velocità alla bocca $w_0 = 800$ m/s e, per semplicità, $c_d = 0{,}4$ costante (a velocità supersonica $c_d$ dipende dal numero di Mach, ma qui interessa solo l'effetto della densità). Si confrontano un'atmosfera a densità costante 1,22 kg/m$^3$ e un'atmosfera esponenziale $\rho_f = 1{,}22\, e^{-0{,}000118\, y}$ kg/m$^3$ ($y$ in metri): la densità si dimezza ogni 5,9 km.

```
def modello_cannone(densita):
    """f(t, s) con c_d costante e densita' dell'aria funzione della quota y."""
    def f(t, s):
        x, y, u, v = s
        rb = densita(y) / RHO                          # rho_bar dipende dalla quota
        A, B, C = 1 + rb / 2, (1 - rb) * G, 3 * rb / (4 * d)
        w = np.hypot(u, v)
        return np.array([u, v, -C * CD * u * w / A, (-B - C * CD * v * w) / A])
    return f

atmosfere = (("densita' costante", lambda y: 1.22),
```

prog1_8_cannone.py

```
            ("atmosfera esponenziale", lambda y: 1.22 * np.exp(-0.000118 * y)))
for nome, dens in atmosfere:
    th, xr, n = gittata_massima(modello_cannone(dens), w0, h=0.05, delta=10.0,
    eps=1e-2)
```

Listato 4.3: Modello del proietto con $c_d$ costante e densità variabile con la quota (`prog1_8_cannone.py`).

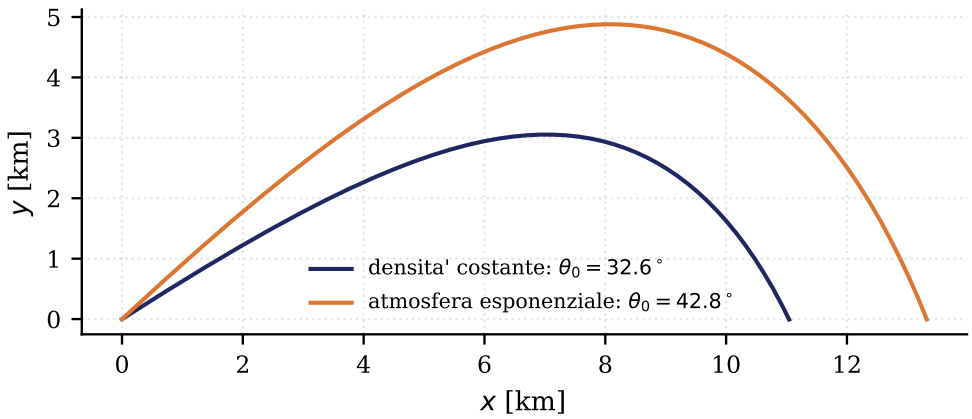



Figura 4.6: Traiettorie di gittata massima del proietto di 0,3 m a 800 m/s nelle due atmosfere.

**Risultati del Problema 4.3**

Atmosfera a densità costante: $\theta_{\text{ott}} = 32{,}6°$, gittata 11,0 km, quota massima 3,1 km. Atmosfera esponenziale: $\theta_{\text{ott}} = 42{,}8°$, gittata 13,3 km, quota massima 4,9 km. Come previsto, l'atmosfera stratificata dà un angolo ottimo più alto e una gittata più lunga: a 45° nell'atmosfera esponenziale si ottengono 13,3 km contro 10,5 km a densità costante. Il famoso cannone di Parigi del 1918 sfruttava esattamente questo principio, lanciando i proietti a quote stratosferiche.

**Progetto: colpire un bersaglio a distanza assegnata**

Con la stessa logica del dimezzamento si può trovare l'angolo $\theta_t$ per cui $x_r(\theta_t) = x_t$, dato un bersaglio a distanza $x_t$. In generale esistono due soluzioni (tiro teso e tiro curvo), se $x_t$ è entro la gittata massima. Si parte dall'estremo sinistro con due punti distanti $\delta > 0$ e si esamina a ogni passo il prodotto $[x_t - (x_r)_{\text{old}}]\,[x_t - (x_r)_{\text{new}}]$: se è positivo i due punti sono dalla stessa parte di $\theta_t$ e $\delta$ resta invariato; se è negativo la soluzione è stata scavalcata e si pone $\delta \leftarrow -\delta/2$. Trovata la prima soluzione, si riparte da lì con il $\delta$ iniziale per trovare la seconda. Se non si trova alcuna soluzione entro 90° il bersaglio è fuori portata, e il programma deve dirlo. Si suggerisce di provarlo con il proietto del Problema 4.3 per $x_t = 8000$ m e 15 000 m (quest'ultimo è raggiungibile?).

## 4.5 Sintesi del capitolo

**Da ricordare**

- Nel moto piano lo stato è $(x, y, u, v)$ e le forze fluidodinamiche dipendono dalla velocità *relativa* $\mathbf{w}_f - \mathbf{w}$; RK4 in forma vettoriale integra il sistema senza modifiche.
- In aria la gittata, la quota e il tempo di volo diminuiscono; la traiettoria perde la simmetria e la gittata dipende dal tempo di esposizione alla resistenza.
- La gittata si calcola per interpolazione lineare fra gli ultimi due punti, recuperando il penultimo con un passo all'indietro ($h \to -h$).
- Il metodo del dimezzamento dell'intervallo trova il massimo di una funzione calcolabile solo numericamente, senza derivate, dimezzando $\delta$ a ogni scavalcamento.
- L'angolo ottimo è $< 45^\circ$ per proiettili piccoli, tende a $45^\circ$ per quelli grandi, e risente della crisi della resistenza e della stratificazione dell'atmosfera.

# Capitolo 5

# La traiettoria di un aliante

## 5.1 Le equazioni del volo planato

Un aliante di massa $m$ vola con velocità **w** inclinata dell'angolo $\theta$ sull'orizzontale (Fig. 5.1). Le forze aerodinamiche in direzione normale e parallela alla traiettoria sono la portanza $L$ e la resistenza $D$. Trascurando la massa aggiunta (irrilevante per un velivolo in aria) e il moto di beccheggio attorno al baricentro, le equazioni del moto del baricentro sono

$$m\frac{\mathrm{d}u}{\mathrm{d}t} = -L\sin\theta - D\cos\theta, \qquad m\frac{\mathrm{d}v}{\mathrm{d}t} = -mg + L\cos\theta - D\sin\theta, \tag{5.1}$$

con $\sin\theta = v/w$, $\cos\theta = u/w$, $w = \sqrt{u^2+v^2}$. Portanza e resistenza dell'intero velivolo si esprimono con i coefficienti $c_l$ e $c_d$:

$$L = c_l\,\tfrac{1}{2}\rho w^2 S, \qquad D = c_d\,\tfrac{1}{2}\rho w^2 S, \tag{5.2}$$

con $\rho$ la densità dell'aria e $S$ la superficie alare.

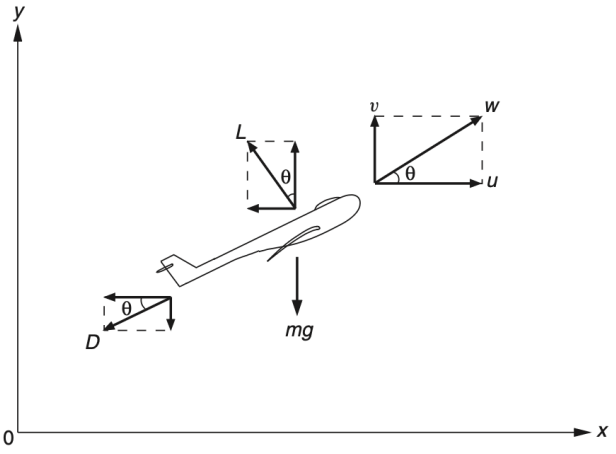



Figura 5.1: Forze su un aliante: portanza normale e resistenza parallela alla traiettoria, peso verticale.

### 5.1.1 Adimensionalizzazione

All'istante $t = 0$ la velocità è $w_0$ e l'inclinazione $\theta_0$. Usiamo $w_0$ come velocità di riferimento, $w_0/g$ come tempo di riferimento e $w_0^2/g$ come lunghezza di riferimento (è la lunghezza che

il velivolo percorrerebbe in caduta libera nel tempo di riferimento): $U = u/w_0$, $V = v/w_0$, $T = tg/w_0$, $X = xg/w_0^2$, $Y = yg/w_0^2$.

**Derivazione delle equazioni adimensionali**

Sostituendo le (5.2) e le relazioni trigonometriche nella prima delle (5.1):

$$\frac{\mathrm{d}u}{\mathrm{d}t} = -\frac{\rho S}{2m}\, w^2 \left( c_l \frac{v}{w} + c_d \frac{u}{w} \right) = -\frac{\rho S}{2m}\, w\,(c_l v + c_d u).$$

Con $u = w_0 U$, $w = w_0 W$ e $t = (w_0/g)T$, cioè $\mathrm{d}u/\mathrm{d}t = g\,\mathrm{d}U/\mathrm{d}T$:

$$\frac{\mathrm{d}U}{\mathrm{d}T} = -\frac{\rho S w_0^2}{2mg}\, W\,(c_l V + c_d U) = -\underbrace{\frac{c_l \,\frac{1}{2}\rho w_0^2 S}{mg}}_{A}\, W \left( \underbrace{\frac{c_d}{c_l}}_{B}\, U + V \right).$$

Analogamente per la componente verticale, $\mathrm{d}v/\mathrm{d}t = -g + (\rho S/2m)\, w\,(c_l u - c_d v)$ dà $\mathrm{d}V/\mathrm{d}T = -1 + A\, W\,(U - BV)$. Le equazioni adimensionali sono dunque

$$\frac{\mathrm{d}^2 X}{\mathrm{d}T^2} = -A\,(U^2 + V^2)^{1/2}\,(BU + V), \qquad \frac{\mathrm{d}^2 Y}{\mathrm{d}T^2} = -1 + A\,(U^2 + V^2)^{1/2}\,(U - BV), \quad (5.3)$$

con $U = \mathrm{d}X/\mathrm{d}T$, $V = \mathrm{d}Y/\mathrm{d}T$. I due parametri sono $A$, rapporto fra la *portanza alla velocità iniziale* e il peso, e $B = c_d/c_l$, l'inverso dell'efficienza aerodinamica $E$.

Poiché $c_l$ e $c_d$ dipendono dall'angolo d'attacco, controllato dal pilota, $A$ e $B$ sono in generale variabili. Qui li assumiamo *costanti*: è il caso di un aliante modello con ali fissate a un angolo d'attacco costante, o di un aeroplanino di carta.

**Dati del Programma 1.6**

$A = 1{,}5$ (alla velocità iniziale la portanza è una volta e mezzo il peso), $B = 0{,}06$ (efficienza $E = 16{,}7$, tipica di un aliante modello). Partenza dall'origine con $W_0 = 1$ e $\theta_0 = -90°$ (picchiata verticale) oppure $\theta_0 = 180°$ (volo rovescio verso le $x$ negative). Passo $h = 0{,}05$, 200 passi ($T_{\max} = 10$).

prog1_6_aliante.py

```
"""
prog1_6_aliante.py  --  Programma 1.6
Traiettoria di un aliante a portanza e resistenza costanti (forma adimensionale).

    dX/dT = U,  dU/dT = -A (U^2+V^2)^{1/2} (B U + V)
    dY/dT = V,  dV/dT = -1 + A (U^2+V^2)^{1/2} (U - B V)

A = c_l (1/2 rho w0^2 S)/(m g)  (portanza iniziale / peso),   B = c_d / c_l.
"""
import numpy as np
import matplotlib.pyplot as plt
from ode_solvers import integra
from stile_grafici import salva, BLU, ARANCIO, VERDE, VIOLA


def aliante(A, B):
    def f(T, s):
        X, Y, U, V = s
        W = np.hypot(U, V)
```

```
        return np.array([U, V, -A * W * (B * U + V), -1.0 + A * W * (U - B * V)])
    return f


if __name__ == "__main__":
    A, B, h, W0 = 1.5, 0.06, 0.05, 1.0
    fig, ax = plt.subplots(figsize=(5.4, 3.2))
    for th0, c, ls, lab in ((-90.0, BLU, "-", "$\\theta_0 = -90^\\circ$"),
                            (180.0, ARANCIO, "-.", "$\\theta_0 = 180^\\circ$")):
        th = np.radians(th0)
        T, S = integra(aliante(A, B), 0.0, [0, 0, W0 * np.cos(th), W0 *
    np.sin(th)], h, 200)
        ax.plot(S[:, 0], S[:, 1], ls, color=c, label=lab)
        W = np.hypot(S[:, 2], S[:, 3])
        print(f"theta0 = {th0:5.0f}: X(T=10) = {S[-1,0]:.3f}, Y = {S[-1,1]:.3f}; "
              f"W min = {W.min():.3f}, W max = {W.max():.3f}")
    ax.set_xlabel("$X$"); ax.set_ylabel("$Y$"); ax.set_aspect("equal")
    ax.set_xlim(-1, 5); ax.set_ylim(-2, 1.2)
    ax.legend(loc="lower right")
    salva(fig, "prog1_6_aliante")

    # Caso "fisico" di riferimento: volo planato rettilineo di equilibrio
    # a velocita' costante: la traiettoria tende all'angolo di planata gamma con
    tan(gamma) = B
    gamma = np.degrees(np.arctan(B))
    print(f"Angolo di planata di equilibrio: tan(gamma) = B -> gamma = {gamma:.2f}
    deg; "
          f"efficienza E = 1/B = {1/B:.1f}")

    # effetto di B: senza resistenza (B = 0) il moto non si smorza
    fig, ax = plt.subplots(figsize=(5.4, 3.2))
    for Bv, c in ((0.0, VERDE), (0.03, VIOLA), (0.06, BLU), (0.12, ARANCIO)):
        th = np.radians(-90.0)
        T, S = integra(aliante(A, Bv), 0.0, [0, 0, 0.0, -W0], h, 400)
        ax.plot(S[:, 0], S[:, 1], color=c, label=f"$B = {Bv}$")
    ax.set_xlabel("$X$"); ax.set_ylabel("$Y$"); ax.set_aspect("equal")
    ax.set_xlim(-0.5, 10.5)
    ax.legend(fontsize=7, loc="lower left")
    salva(fig, "prog1_6_effettoB")
```

Listato 5.1: Programma 1.6 – traiettorie di un aliante a portanza e resistenza costanti (`prog1_6_aliante.py`).

**Lettura della Fig. 5.2**

- **Il caso ideale.** Senza resistenza e senza peso la portanza sarebbe sempre perpendicolare alla direzione del moto e di modulo costante: la traiettoria sarebbe una *circonferenza*. Il peso trasforma la circonferenza in una curva a spirale ("a molla") che deriva lungo $x$ con ampiezza non smorzata; la resistenza dissipa energia cinetica e la quota media decresce nel tempo. L'analogia è con una particella carica in un campo magnetico uniforme, con e senza una forza di massa normale alle linee del campo.
- $\theta_0 = -90°$**.** L'aliante picchia, poi richiama e sale. Quando raggiunge una quota di poco superiore a quella iniziale ha speso quasi tutta l'energia cinetica contro attrito e gravità ($W_{\min} = 0{,}08$): la velocità è troppo bassa per generare portanza, l'aliante *stalla* e si rimette in picchiata per riprendere velocità. È la traiettoria che chiunque

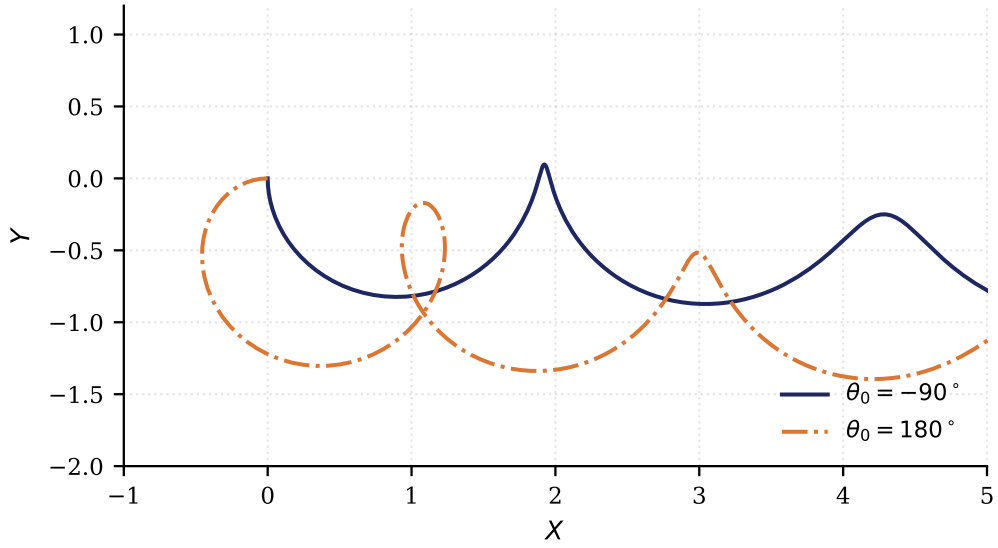



Figura 5.2: Traiettorie dell'aliante con $A = 1{,}5$, $B = 0{,}06$, $W_0 = 1$ per $\theta_0 = -90^\circ$ (linea continua) e $\theta_0 = 180^\circ$ (tratto-punto).

ha visto fare a un aeroplanino di carta lanciato troppo forte.

- $\theta_0 = 180^\circ$. In volo rovescio la portanza iniziale è diretta come il peso e accelera l'aliante verso il basso; quando raggiunge la prima posizione orizzontale ha una velocità così alta ($W_{\max} = 1{,}7$) da disporre dell'energia per compiere un *loop* completo, dopo il quale prosegue con oscillazioni smorzate.

**Il moto fugoide e la planata di equilibrio**

Le oscillazioni di lungo periodo della traiettoria a $\theta_0 = -90^\circ$ sono il *moto fugoide* descritto da Lanchester [13]: uno scambio periodico fra energia cinetica e potenziale a incidenza (quasi) costante. Dalle (5.3) si ricava il volo rettilineo uniforme di equilibrio ponendo $\mathrm{d}U/\mathrm{d}T = \mathrm{d}V/\mathrm{d}T = 0$: dalla prima, $V = -BU$, cioè $\tan\gamma = -V/U = B$: *l'angolo di planata di equilibrio vale* $\arctan B = 3{,}4^\circ$, e il rapporto di planata è $1/B = E = 16{,}7$ (per ogni metro di quota perduta se ne percorrono 16,7 in orizzontale). Dalla seconda, $A\,W_e\,U_e(1 + B^2) = 1$, da cui, per $B \ll 1$, $W_e \simeq A^{-1/2} = 0{,}816$: la velocità di equilibrio è quella a cui la portanza eguaglia il peso. Linearizzando le equazioni attorno a questo stato (e trascurando la resistenza) si trova che le perturbazioni oscillano con pulsazione $\omega = \sqrt{2}\,g/w_e$, cioè con periodo adimensionale $T_{\mathrm{ph}} = \pi\sqrt{2}\,W_e \simeq 3{,}6$ (dimensionale: $T_{\mathrm{ph}} = \pi\sqrt{2}\,w_e/g$, che per un aliante reale a 25 m/s vale 11 s). Il periodo dipende solo dalla velocità, non dalle caratteristiche del velivolo: è una delle proprietà più sorprendenti del moto fugoide, e la Fig. 5.2 la conferma qualitativamente (circa 2,5 oscillazioni in $T = 10$, con periodo un po' maggiore di 3,6 a causa della grande ampiezza). La Fig. 5.3 mostra l'effetto di $B$: per $B = 0$ (aliante ideale senza resistenza) le oscillazioni non si smorzano; al crescere di $B$ lo smorzamento aumenta e la planata di equilibrio è raggiunta prima, ma con pendenza maggiore.

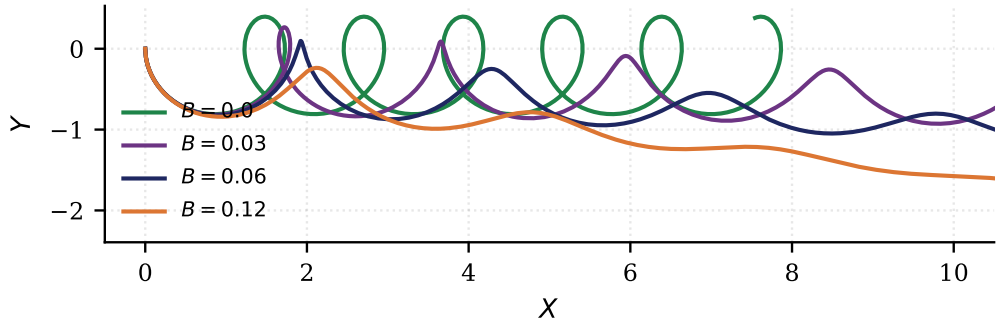



Figura 5.3: Traiettorie per $\theta_0 = -90°$ e $A = 1{,}5$ al variare di $B = c_d/c_l$: senza resistenza il moto fugoide non si smorza.

**Dall'aliante all'aeroplano**

Aggiungendo alle (5.1) una forza di spinta $F$ lungo la traiettoria (termini $+F\cos\theta$ e $+F\sin\theta$) si ottengono le equazioni del moto di un aeroplano nel piano verticale; con $c_l$ e $c_d$ funzioni dell'angolo d'attacco (comandato) e della velocità, il Programma 1.6 simula decollo, salita e manovre bidimensionali. È il nucleo di qualunque simulatore di meccanica del volo a tre gradi di libertà. Ciò che manca, qui come nel Cap. 3, è la dinamica di beccheggio: il modello suppone che l'assetto si adegui istantaneamente alla traiettoria.

## 5.2 Problema 5.1: volo verticale di un razzo

Come complemento, consideriamo il moto verticale di un razzo a *massa variabile*:

$$\frac{\mathrm{d}z}{\mathrm{d}t} = v, \qquad (m_c + m_p)\frac{\mathrm{d}v}{\mathrm{d}t} = -(m_c + m_p)\,g + \dot{m}_p\,v_e - \tfrac{1}{2}\rho\,v|v|\,A_s\,C_D, \tag{5.4}$$

dove $m_c$ è la massa dell'involucro (peso 500 N), $m_p(t)$ la massa istantanea del propellente (peso iniziale 1000 N), $\dot{m}_p$ la portata di combustione, $v_e = 360$ m/s la velocità di efflusso, $A_s = 0{,}1$ m$^2$ la sezione maestra, $C_D = 0{,}15$, $\rho = 1{,}23$ kg/m$^3$. La spinta $\dot{m}_p v_e$ è il flusso di quantità di moto espulso dall'ugello.

**Legge di combustione**

Assumiamo una portata di combustione costante pari a $m_{p0}/4$ per $0 \le t \le 4$ s, nulla dopo: il propellente si esaurisce in 4 s, $m_p(t) = m_{p0}(1 - t/4)$ per $t \le 4$ s e $m_p = 0$ dopo. La spinta vale allora $\dot{m}_p v_e = (1000/9{,}8)/4 \cdot 360 = 9184$ N, contro un peso iniziale di 1500 N: rapporto spinta/peso 6,1. Integriamo con un metodo RK del *secondo* ordine e $h = 0{,}1$ s, e confrontiamo con un risolutore adattivo (`solve_ivp`).

prog1_9_razzo.py

```python
"""
prog1_9_razzo.py  --  Problema 5.1
Volo verticale di un razzo con massa variabile (RK2 a passo 0.1 s, confronto con solve_ivp).

    dz/dt = v
    (m_c + m_p) dv/dt = -(m_c + m_p) g + mdot_p v_e - 1/2 rho v|v| A_s C_D

Ipotesi sulla legge di combustione: mdot_p = m_p0/4 costante per 0 <= t <= 4 s,
nulla dopo (il propellente, pari a 1000 N di peso, si esaurisce in 4 s).
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from ode_solvers import integra
from stile_grafici import salva, BLU, ARANCIO

g, rho, As, ve, CD = 9.8, 1.23, 0.1, 360.0, 0.15
mc, mp0 = 500.0 / g, 1000.0 / g
t_burn = 4.0


def mdot(t):
    return mp0 / t_burn if t <= t_burn else 0.0


def mp(t):
    return max(mp0 - mp0 * min(t, t_burn) / t_burn, 0.0)


def f(t, s):
    z, v = s
    m = mc + mp(t)
    spinta = mdot(t) * ve
    drag = 0.5 * rho * v * abs(v) * As * CD
    return np.array([v, (-m * g + spinta - drag) / m])


if __name__ == "__main__":
    h = 0.1
    tocca_terra = lambda t, s: s[0] < 0.0 and t > 1.0
    T, S = integra(f, 0.0, [0.0, 0.0], h, 2000, metodo="rk2", stop=tocca_terra)
    z, v = S[:, 0], S[:, 1]
    i_vmax, i_zmax = np.argmax(v), np.argmax(z)
    print(f"Spinta iniziale = {mdot(0)*ve:.1f} N, peso iniziale = {(mc+mp0)*g:.0f} N "
          f"(rapporto spinta/peso = {mdot(0)*ve/((mc+mp0)*g):.2f})")
    print(f"a) v_max = {v[i_vmax]:.2f} m/s a t = {T[i_vmax]:.1f} s, z = {z[i_vmax]:.1f} m")
    print(f"b) z_max = {z[i_zmax]:.1f} m a t = {T[i_zmax]:.1f} s")
    print(f"c) impatto a t ~ {T[-1]:.1f} s con v = {v[-1]:.2f} m/s")

    # confronto con solve_ivp (RK45 adattivo) -- analogo di ODE45
    ev = lambda t, s: s[0]
    ev.terminal, ev.direction = True, -1
    sol = solve_ivp(f, [0.5, 200.0], S[5], method="RK45", events=ev,
                    max_step=0.1, rtol=1e-8, atol=1e-10, dense_output=True)
    print(f"d) solve_ivp: impatto a t = {sol.t_events[0][0]:.3f} s, "
          f"v = {sol.y_events[0][0][1]:.3f} m/s, z_max = {sol.y[0].max():.2f} m")

```

```
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(6.0, 2.6))
    a1.plot(T, z, color=BLU, label="RK2, $h=0.1$ s")
    a1.plot(sol.t, sol.y[0], "--", color=ARANCIO, label="solve\\_ivp (RK45)")
    a2.plot(T, v, color=BLU); a2.plot(sol.t, sol.y[1], "--", color=ARANCIO)
    a1.set_xlabel("$t$ [s]"); a1.set_ylabel("$z$ [m]"); a1.legend(fontsize=7)
    a2.set_xlabel("$t$ [s]"); a2.set_ylabel("$v$ [m/s]")
    salva(fig, "prog1_9_razzo")
```

Listato 5.2: Problema 5.1 – razzo a massa variabile, RK2 e confronto con `solve_ivp` (`prog1_9_razzo.py`).

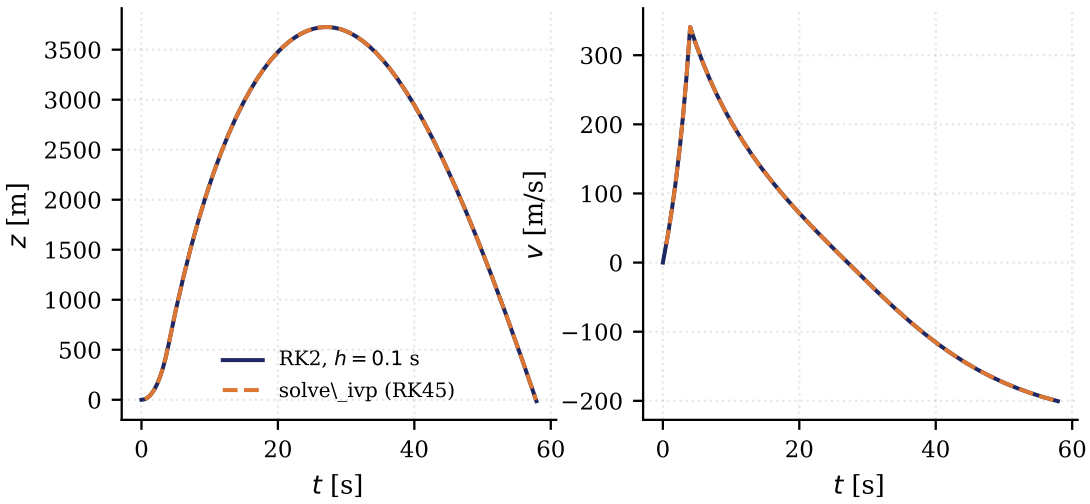



Figura 5.4: Quota e velocità del razzo: RK2 a passo 0,1 s e RK45 adattivo.

**Risultati del Problema 5.1**

a) Velocità massima 340,8 m/s, raggiunta allo spegnimento ($t = 4$ s) a 560 m di quota.

b) Quota massima 3726 m a $t = 27{,}1$ s.

c) Impatto al suolo a $t \simeq 57{,}9$ s con $v = -200{,}5$ m/s; la velocità terminale in discesa sarebbe $\sqrt{2m_c g/(\rho A_s C_D)} = 233$ m/s, non ancora raggiunta.

d) `solve_ivp` (RK45, evento $z = 0$) dà impatto a 57,83 s con $-200{,}3$ m/s e $z_{\max} = 3726{,}7$ m. Il "57,9 s" di RK2 è semplicemente il primo nodo sotto il suolo: interpolando linearmente fra gli ultimi due nodi si ottiene 57,831 s, a 2 millesimi di secondo dal riferimento. La *discontinuità* della spinta a $t = 4$ s non disturba perché con $h = 0{,}1$ s un nodo cade esattamente sull'istante di spegnimento; l'Esercizio B.14 mostra che cosa succede quando non è così.

## 5.3 Sintesi del capitolo

**Da ricordare**

- Le equazioni del volo planato nel piano verticale dipendono da due soli parametri adimensionali: $A$ (portanza iniziale/peso) e $B = c_d/c_l = 1/E$.
- Planata di equilibrio: $\tan\gamma = B$, $W_e = A^{-1/2}$; il moto fugoide ha periodo $\pi\sqrt{2}\,w_e/g$, indipendente dal velivolo.

- A incidenza costante l'aliante alterna stalli e picchiate, o compie un loop se parte in volo rovescio: la traiettoria è governata dallo scambio fra energia cinetica e potenziale, dissipato dalla resistenza.
- I sistemi a massa variabile (razzo) rientrano nello stesso schema: $f(t, \mathbf{s})$ dipende esplicitamente dal tempo attraverso $m_p(t)$ e $\dot{m}_p(t)$.
- Discontinuità nel secondo membro (accensione/spegnimento) riducono l'ordine effettivo del metodo: conviene far cadere un nodo esattamente sugli istanti di discontinuità (Esercizio B.14).

# Capitolo 6

# Avvolgimento della scia vorticosa dietro un'ala finita

## 6.1 Dal profilo all'ala finita: il sistema di vortici

Nel calcolo della portanza di un'ala di apertura infinita (profilo bidimensionale) l'ala può essere sostituita da un *foglio vorticoso* coincidente con la linea media. Se $\Gamma$ è la circolazione totale del foglio e l'ala vola a velocità costante $U$ in aria di densità $\rho$, la portanza per unità di apertura è data dal teorema di Kutta–Joukowski,

$$L = \rho\, U\, \Gamma, \qquad \Gamma = \oint \mathbf{V} \cdot \mathrm{d}\mathbf{s}, \tag{6.1}$$

dove l'integrale di linea è esteso a una curva chiusa qualsiasi che racchiuda il foglio, percorsa in senso orario. Ai fini della portanza di sezione, l'ala equivale quindi a un *vortice legato* di circolazione $\Gamma$, con asse parallelo all'apertura: "legato" perché, a differenza di un vortice libero, non si muove con il fluido.

Per un'ala finita il vortice legato non può terminare alle estremità (per i teoremi di Helmholtz un filamento vorticoso non può avere estremi nel fluido): i suoi estremi vengono trascinati a valle dalla corrente e formano una coppia di vortici controrotanti che si estendono all'infinito dietro l'ala, il *vortice a ferro di cavallo*. Poiché la portanza per unità di apertura varia lungo l'apertura, la circolazione del vortice legato è una funzione $\Gamma(x)$ della posizione in apertura; un'ala simmetrica si costruisce sovrapponendo infiniti ferri di cavallo di intensità diversa (Fig. 6.1). Le infinite linee vorticose trascinate formano, nel piano $x$-$z$, un *foglio vorticoso di scia* di intensità

$$\gamma(x) = \frac{\mathrm{d}\Gamma}{\mathrm{d}x}, \tag{6.2}$$

definita come circolazione contenuta nell'unità di lunghezza in apertura del foglio. I vortici di scia inducono sull'ala una velocità verso il basso, il *downwash*, responsabile della resistenza indotta, caratteristica dell'ala finita [8, Cap. 6].

## 6.2 Il foglio di scia non resta piano

Nella teoria dell'ala finita il foglio di scia si suppone, per comodità, fermo nel piano $x$-$z$. In realtà ogni linea vorticosa si muove nel campo di velocità indotto dalle altre, e il moto differisce da linea a linea secondo la geometria istantanea del foglio: il foglio *si avvolge*, formando

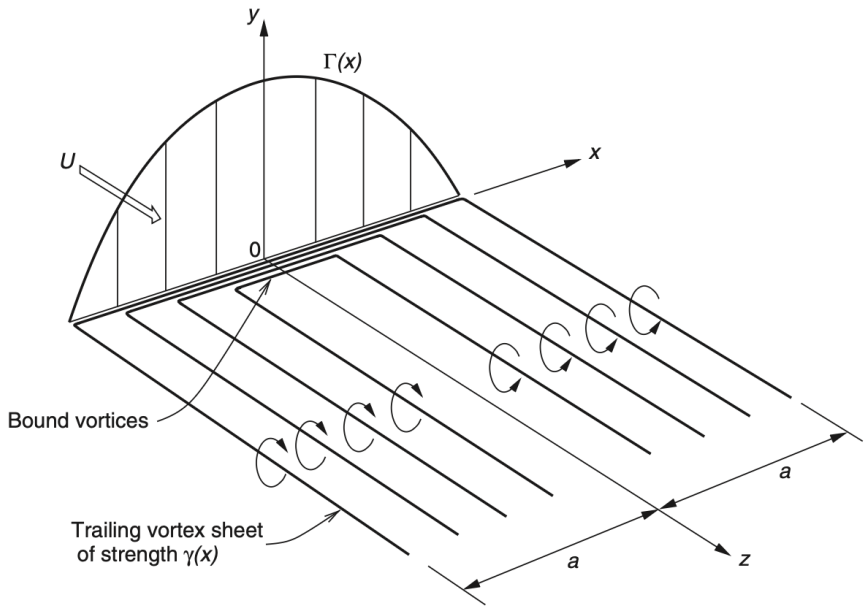



Figura 6.1: Sistema di vortici di un'ala finita: vortici legati di intensità crescente verso la mezzeria e foglio di scia trascinato a valle.

le due colonne vorticose ben definite che si osservano dietro un aereo (visibili nelle scie di condensazione d'estremità, o nelle fotografie con fumogeni).

L'avvolgimento è un fenomeno tridimensionale. Lo semplifichiamo in un problema bidimensionale: un foglio vorticoso di larghezza $2a$ che si estende all'infinito in entrambi i versi lungo $z$. Si rimuove così il vincolo che un estremo del foglio sia attaccato al vortice legato, ma le configurazioni del foglio bidimensionale a istanti successivi somigliano da vicino a quelle del foglio tridimensionale in sezioni sempre più a valle (sostituendo $t$ con $z/U$).

**Il foglio piano iniziale per un'ala a carico ellittico**

Per un'ala a carico ellittico, con circolazione $\Gamma_0$ in mezzeria,

$$\Gamma(x) = \frac{\Gamma_0}{a}\,(a^2 - x^2)^{1/2} \quad \Longrightarrow \quad \gamma(x) = \frac{\mathrm{d}\Gamma}{\mathrm{d}x} = -\frac{\Gamma_0\, x}{a\,(a^2 - x^2)^{1/2}}. \tag{6.3}$$

Il foglio è antisimmetrico: $\gamma > 0$ (vortici orari, nella convenzione della (6.1)) per $x < 0$ e $\gamma < 0$ per $x > 0$, con una singolarità integrabile alle estremità. La circolazione totale della metà sinistra è $\int_{-a}^{0} \gamma\,\mathrm{d}x = \Gamma(0) - \Gamma(-a) = \Gamma_0$, quella della metà destra $-\Gamma_0$: la circolazione totale del foglio è nulla, come deve essere (ogni ferro di cavallo contribuisce con due gambe opposte). Un'ala con circolazione $\Gamma_0$ in mezzeria rilascia dunque in scia due vortici di intensità $\pm\Gamma_0$, distribuiti sul foglio.

## 6.3 Discretizzazione in vortici puntiformi

Per studiare l'evoluzione di questa distribuzione continua, il foglio viene suddiviso in intervalli in apertura, e ciascuno è sostituito da un *vortice puntiforme* di circolazione pari a quella contenuta nell'intervallo. La velocità di ciascun vortice si calcola in funzione della posizione

degli altri; da essa si ottiene lo spostamento dopo un breve intervallo di tempo, e ripetendo la procedura si segue la deformazione del foglio. È, a tutti gli effetti, un *problema a N corpi*: $m$ "particelle" che interagiscono a coppie con una forza (qui una velocità) che decresce con l'inverso della distanza.

All'istante $t = 0$ il foglio piano è diviso in $m$ intervalli uguali ($m$ pari, ragionevolmente grande), ciascuno sostituito da un vortice nel punto medio (Fig. 6.2): la distanza fra vortici adiacenti è $2a/m$ e i due vortici estremi distano $a/m$ dall'estremità. I vortici sono numerati da sinistra. Integrando la (6.3) sull'intervallo $i$-esimo, l'intensità del vortice posto in $(x_i, 0)$ è

$$g_i = \int_{x_i - a/m}^{x_i + a/m} \gamma \, \mathrm{d}x = \Gamma_0 \left[ \sqrt{1 - \left( \frac{x_i}{a} + \frac{1}{m} \right)^2} - \sqrt{1 - \left( \frac{x_i}{a} - \frac{1}{m} \right)^2} \right]. \tag{6.4}$$

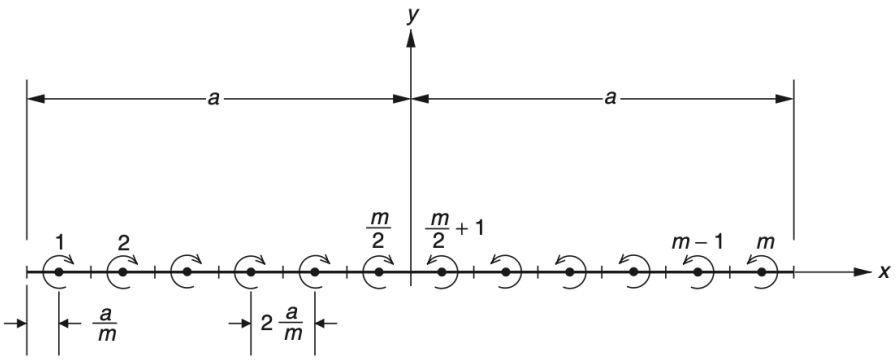



Figura 6.2: Sostituzione del foglio vorticoso con $m$ vortici puntiformi equispaziati.

**Velocità indotta: la legge di Biot–Savart in due dimensioni**

Un vortice puntiforme di circolazione $g_j$ in $(x_j, y_j)$ induce nel punto $(x_i, y_i)$, a distanza $R = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$, una velocità di modulo

$$W_i = \frac{g_j}{2\pi R},$$

tangente alla circonferenza di raggio $R$ centrata sul vortice (il campo di un filamento rettilineo infinito, caso particolare della legge di Biot–Savart). Con $\theta$ l'angolo che il raggio vettore forma con l'asse $y$, le componenti sono, per una circolazione positiva in senso orario,

$$u_i = W_i \sin\theta = \frac{y_i - y_j}{R} W_i, \qquad v_i = -W_i \cos\theta = -\frac{x_i - x_j}{R} W_i.$$

Il vortice non induce velocità su se stesso (il campo è singolare in $R = 0$, ma per simmetria la velocità al centro è nulla). Sommando i contributi di tutti i vortici $j \neq i$ e identificando $u$ con $\mathrm{d}x/\mathrm{d}t$ e $v$ con $\mathrm{d}y/\mathrm{d}t$:

$$\frac{\mathrm{d}x_i}{\mathrm{d}t} = \sum_{j \neq i}^{m} \frac{g_j}{2\pi} \frac{y_i - y_j}{(x_i - x_j)^2 + (y_i - y_j)^2}, \qquad \frac{\mathrm{d}y_i}{\mathrm{d}t} = -\sum_{j \neq i}^{m} \frac{g_j}{2\pi} \frac{x_i - x_j}{(x_i - x_j)^2 + (y_i - y_j)^2}. \tag{6.5}$$

Sono $2m$ equazioni differenziali del primo ordine accoppiate: ciascun vortice è un "corpo" il cui moto dipende da tutti gli altri.

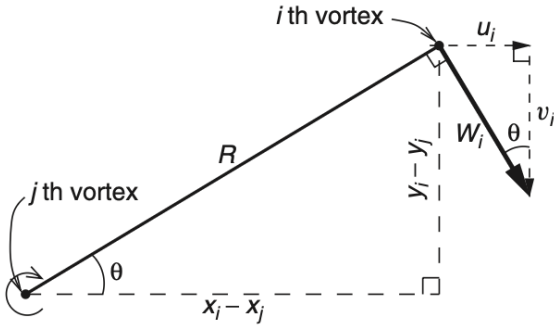



Figura 6.3: Velocità indotta sul vortice $i$-esimo dal vortice $j$-esimo.

### 6.3.1 Forma adimensionale e condizione sul passo

Con le variabili adimensionali

$$X = \frac{x}{a}, \qquad Y = \frac{y}{a}, \qquad G = \frac{g}{\Gamma_0}, \qquad T = \frac{t}{2\pi a^2/\Gamma_0}, \tag{6.6}$$

le (6.4) e (6.5) diventano

$$G_i = \sqrt{1 - \left(X_i + \tfrac{1}{m}\right)^2} - \sqrt{1 - \left(X_i - \tfrac{1}{m}\right)^2}, \tag{6.7}$$

$$\frac{\mathrm{d}X_i}{\mathrm{d}T} = \sum_{j \neq i} G_j \frac{Y_i - Y_j}{(X_i - X_j)^2 + (Y_i - Y_j)^2}, \qquad \frac{\mathrm{d}Y_i}{\mathrm{d}T} = -\sum_{j \neq i} G_j \frac{X_i - X_j}{(X_i - X_j)^2 + (Y_i - Y_j)^2}. \tag{6.8}$$

Il tempo di riferimento $2\pi a^2/\Gamma_0$ è il tempo che un vortice di circolazione $\Gamma_0$ impiega a far compiere un giro completo a una particella a distanza $a$; la distanza iniziale fra vortici adiacenti è $2/m$. Il sistema ha la forma $\mathrm{d}\mathbf{x}/\mathrm{d}t = \mathbf{F}_1$, $\mathrm{d}\mathbf{y}/\mathrm{d}t = \mathbf{F}_2$ e si integra con l'RK4 per sistemi del primo ordine, che è esattamente la `passo_rk4` del Listato 1.2 applicata al vettore di stato $(X_1, \ldots, X_m, Y_1, \ldots, Y_m)$.

**Il passo di tempo e l'instabilità numerica**

Due vortici vicini si inducono reciprocamente velocità grandi, inversamente proporzionali alla distanza: la "costante di tempo" locale del sistema è dell'ordine di $R^2/G$, e per vortici adiacenti ($R = 2/m$, $G \sim 1/m$ in media) vale $\sim 4/m$ in unità di $T$; vicino alle estremità, dove $G_i$ è massimo, è molto minore. Moore [15] ha trovato la restrizione, riscritta nelle nostre variabili,

$$\mathrm{d}T \ll \frac{8\pi}{m^2}. \tag{6.9}$$

Con $m = 40$ il limite vale 0,0157; scegliamo $\mathrm{d}T = 1/(25m) = 0{,}001$, quindici volte più piccolo, e si verifica che passi ancora più piccoli cambiano i risultati solo marginalmente vicino alle estremità. È la stessa logica del §1.6: il passo deve essere piccolo rispetto alla scala di tempo più rapida, che qui cresce come $m^2$ perché infittire i vortici li avvicina.

**Sfruttare la simmetria**

Il foglio resta simmetrico rispetto all'asse $Y$ in ogni istante (le intensità sono antisimmetriche, le posizioni simmetriche: il campo di velocità indotto è a sua volta simmetrico).

Basta quindi calcolare il moto dei vortici della metà sinistra, $i = 1, \dots, m/2$; il vortice $k = m + 1 - i$ sulla destra è l'immagine speculare dell'$i$-esimo: $X_k = -X_i$, $Y_k = Y_i$. Nel programma la velocità è calcolata con tutte le $m$ coppie (in forma vettoriale è immediato), e dopo ogni passo la funzione `simmetrizza` impone la simmetria, eliminando l'accumulo di errori di arrotondamento asimmetrici. Il costo del calcolo cresce come $m^2$ (tutte le coppie) per il numero dei passi, a sua volta $\propto m^2$ per la (6.9): raddoppiare $m$ costa sedici volte tanto.

## 6.4 Programma 1.7 e risultati

**Dati**

$m = 40$ vortici, $\mathrm{d}T = 1/(25m) = 0{,}001$; istanti di uscita $T = 0{,}005$, 0,025, 0,05, 0,075, 0,1, 0,15, 0,2, 0,35 (350 passi). Si disegna la metà destra del foglio, $0 \le X \le 1$.

Il listato sfrutta appieno NumPy: le differenze $X_i - X_j$ per tutte le coppie sono calcolate in un colpo solo come matrice $m \times m$ (*broadcasting*), la diagonale è esclusa ponendo $R^2 = \infty$, e le somme (6.8) sono somme per righe. Non c'è alcun ciclo esplicito sui vortici.

prog1_7_scia_vorticosa.py

```
"""
prog1_7_scia_vorticosa.py  --  Programma 1.7
Avvolgimento della scia vorticosa dietro un'ala a carico ellittico:
m vortici puntiformi che si muovono con la velocita' indotta dagli altri
(Biot-Savart bidimensionale), integrati nel tempo con RK4.

    dX_i/dT =  sum_{j != i} G_j (Y_i - Y_j) / R_ij^2
    dY_i/dT = -sum_{j != i} G_j (X_i - X_j) / R_ij^2
"""
import numpy as np
import matplotlib.pyplot as plt
from ode_solvers import passo_rk4
from stile_grafici import salva, BLU, ARANCIO

m = 40                          # numero di vortici (pari)
dT = 1.0 / (25 * m)             # passo: soddisfa dT << 8 pi/m^2 (Moore, 1974)
istanti = [0.005, 0.025, 0.05, 0.075, 0.10, 0.15, 0.20, 0.35]

# posizioni iniziali e intensita' (eq. 1.7.9)
dX = 1.0 / m
X0 = -1.0 + dX + 2.0 * dX * np.arange(m)
radice = lambda x: np.sqrt(np.clip(1.0 - x ** 2, 0.0, None))   # evita 1 - X^2 < 0
    per arrotondamento
G = radice(X0 + dX) - radice(X0 - dX)
print(f"Verifica: somma delle G sul semipiano sinistro = {G[:m//2].sum():.6f}
    (atteso 1)")
print(f"dT = {dT:.4f}; limite di Moore 8 pi/m^2 = {8*np.pi/m**2:.4f}")


def velocita(T, s):
    """s = [X_1..X_m, Y_1..Y_m]: velocita' indotta su ciascun vortice da tutti gli
    altri."""
    X, Y = s[:m], s[m:]
    DX = X[:, None] - X[None, :]
    DY = Y[:, None] - Y[None, :]
    R2 = DX ** 2 + DY ** 2
```

```
    np.fill_diagonal(R2, np.inf)                        # esclude l'autoinduzione
    U = (G[None, :] * DY / R2).sum(axis=1)
    V = -(G[None, :] * DX / R2).sum(axis=1)
    return np.concatenate([U, V])


def simmetrizza(s):
    """Impone la simmetria rispetto all'asse Y: il vortice k = m+1-i e' l'immagine dell'i-esimo."""
    X, Y = s[:m].copy(), s[m:].copy()
    X[m // 2:] = -X[: m // 2][::-1]
    Y[m // 2:] = Y[: m // 2][::-1]
    return np.concatenate([X, Y])


if __name__ == "__main__":
    s = np.concatenate([X0, np.zeros(m)])
    T, k = 0.0, 0
    fig, axs = plt.subplots(4, 2, figsize=(6.2, 7.0))
    axs = axs.T.ravel()
    y_mid = []
    while k < len(istanti):
        s = simmetrizza(passo_rk4(velocita, T, s, dT))
        T += dT
        y_mid.append((T, s[m + m // 2 - 1]))
        if T >= istanti[k] - 1e-12:
            ax = axs[k]
            ax.plot(s[m // 2:m], s[m + m // 2:], ".", color=BLU, ms=3)
            ax.set_title(f"$T = {istanti[k]}$", fontsize=8, loc="right")
            ax.set_xlim(0, 1.05); ax.set_aspect("equal")
            ax.set_ylim(min(-0.12, s[m:].min() - 0.03), 0.12)
            ax.set_xlabel("$X$"); ax.set_ylabel("$Y$")
            print(f"T = {istanti[k]:5.3f}: Y vortice centrale = {s[m + m//2 - 1]:+.4f}, "
                  f"Y vortice di estremita' = {s[2*m - 1]:+.4f}")
            k += 1
    fig.tight_layout()
    salva(fig, "prog1_7_scia")

    # velocita' di discesa del punto medio del foglio
    y_mid = np.array(y_mid)
    vel = np.gradient(y_mid[:, 1], y_mid[:, 0])
    fig, ax = plt.subplots(figsize=(4.4, 2.6))
    ax.plot(y_mid[:, 0], -vel, color=BLU)
    # in forma dimensionale dy/dt = Gamma_0/(2 pi a) dY/dT: il valore Gamma_0/(2a) corrisponde a dY/dT = pi
    ax.axhline(np.pi, color=ARANCIO, ls="--", lw=0.8, label="$\\pi$  (cioe' $\\Gamma_0/2a$)")
    ax.axhline(np.pi / 2, color="gray", ls=":", lw=0.8, label="$\\pi/2$")
    ax.set_xlabel("$T$"); ax.set_ylabel("$-\\mathrm{d}Y/\\mathrm{d}T$ del punto medio")
    ax.set_ylim(0, 3.5)
    ax.legend(fontsize=7)
    salva(fig, "prog1_7_discesa")
    print(f"Velocita' di discesa adimensionale del centro: iniziale {-vel[0]:.3f} (teorico pi = {np.pi:.3f}), "
          f"a T = 0.35: {-vel[-1]:.3f}")

    # applicazione: Boeing 747
    a, W, Uinf, Gmed = 30.0, 4.0e6, 67.0, 815.0
```

```
88      G0 = 4 * Gmed / np.pi
89      t_car = 2 * np.pi * a ** 2 / G0
90      print(f"Boeing 747: Gamma_0 = {G0:.0f} m^2/s, tempo caratteristico =
        {t_car:.2f} s, "
91            f"T = 0.35 -> t = {0.35*t_car:.2f} s -> {0.35*t_car*Uinf:.0f} m dietro
        l'aereo")
```

Listato 6.1: Programma 1.7 – avvolgimento di un foglio vorticoso (`prog1_7_scia_vorticosa.py`).

**Lettura della Fig. 6.4**

- Inizialmente il vortice d'estremità si muove *verso l'alto*, per la sua peculiarità di avere vortici da un lato solo; tutti gli altri scendono con velocità quasi uguali (il foglio piano si trasla verso il basso come un corpo rigido, salvo le estremità).
- Le interazioni nella regione d'estremità incurvano il foglio verso l'alto e lo arrotolano in una *spirale*; sempre più vortici vengono attratti nella spirale, che si espande, e il foglio si *allunga*, lasciando distanze crescenti fra vortici adiacenti lungo il tratto piano.
- Il foglio tende a evolvere in due spirali strettamente avvolte e controrotanti: è la coppia di vortici di scia osservata dietro un aereo. I centri delle due spirali distano meno dell'apertura $2a$ (per il carico ellittico la teoria dà $b_0 = \pi a/2 = 0{,}785 \cdot 2a$ fra i baricentri delle due metà del foglio).
- La regione d'estremità scende più lentamente della regione centrale. Il punto medio del foglio scende inizialmente con velocità $\Gamma_0/(2a)$ (cioè $\pi$ in unità adimensionali: la Fig. 6.5 dà 3,10 con $m = 40$ contro $\pi = 3{,}14$) e rallenta verso un valore circa metà.

**I limiti del calcolo: il caos della spirale**

Con 40 vortici i risultati non sono accurati a $T$ grandi, perché i vortici costituenti si allontanano troppo. A $T = 0{,}35$ compaiono irregolarità nella spirale; a tempi maggiori le traiettorie di alcuni vortici si contorcono e parti del foglio possono *incrociarsi*, cosa fisicamente impossibile per un foglio vorticoso continuo. Il calcolo va arrestato prima. Questo moto caotico della spirale è stato trovato da diversi autori (si veda [15] per una rassegna): Moore lo eliminò rappresentando la parte strettamente avvolta con un unico vortice concentrato d'estremità, che ingloba progressivamente i vortici catturati. Un'alternativa oggi diffusa è il metodo dei *vortex blob* di Krasny [16]: si sostituisce $R^2$ con $R^2 + \delta^2$ nelle (6.8), regolarizzando la singolarità con un nucleo di raggio $\delta$; il lettore può provarlo aggiungendo una riga a `velocita`. Risultati più accurati si ottengono con $m$ maggiore, ma il tempo di calcolo cresce circa come $m^2$ per passo.

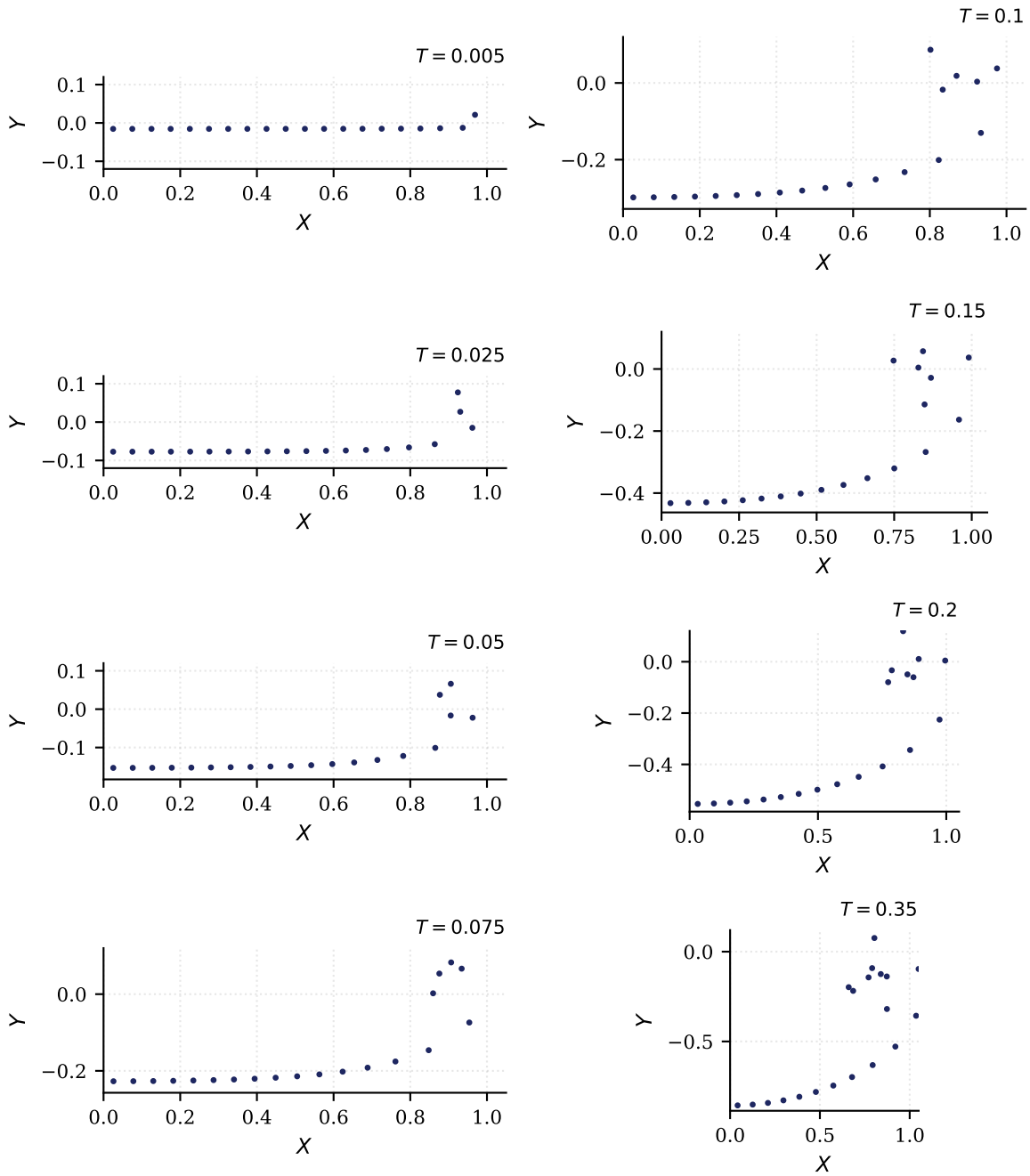



Figura 6.4: Evoluzione del foglio vorticoso inizialmente piano (metà destra, $m = 40$) a otto istanti adimensionali.

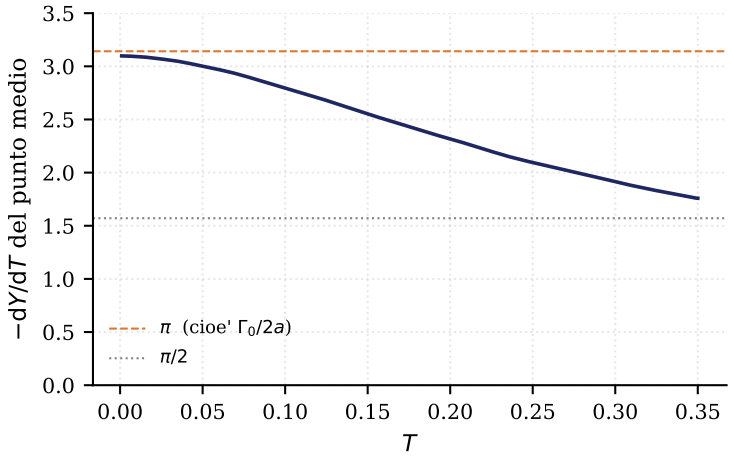



Figura 6.5: Velocità di discesa adimensionale del punto medio del foglio ($-\mathrm{d}Y/\mathrm{d}T$ dei due vortici centrali).

## 6.5 Applicazione: la scia di un Boeing 747

**Dalla forma adimensionale al caso reale**

Per un Boeing 747 con semiapertura $a = 30$ m e peso $W = 4 \cdot 10^6$ N, la circolazione media lungo l'apertura durante il decollo al livello del mare a $U = 67$ m/s è $\bar{\Gamma} = W/(\rho U \cdot 2a) = 815$ m$^2$/s. Supponendo la distribuzione ellittica (6.3),

$$2a\,\bar{\Gamma} = \int_{-a}^{a} \frac{\Gamma_0}{a}(a^2 - x^2)^{1/2}\,\mathrm{d}x = \tfrac{1}{2}\pi a\,\Gamma_0 \quad \Longrightarrow \quad \Gamma_0 = \frac{4\bar{\Gamma}}{\pi} = 1038 \text{ m}^2/\text{s}.$$

Il tempo caratteristico è $2\pi a^2/\Gamma_0 = 5{,}45$ s: l'ultimo riquadro della Fig. 6.4, $T = 0{,}35$, corrisponde a $t = 1{,}91$ s, cioè, a 67 m/s, alla configurazione della scia tridimensionale circa 128 m dietro l'aereo. La velocità iniziale di discesa del centro del foglio, $\Gamma_0/(2a) = 17$ m/s, si riduce rapidamente; la coppia di vortici completamente avvolta scende con la velocità reciprocamente indotta $\Gamma_0/(2\pi b_0)$, con $b_0 = \pi a/2 = 47$ m, cioè circa 3,5 m/s. Sono questi vortici, persistenti per minuti e capaci di rovesciare un velivolo leggero, a imporre le *separazioni per turbolenza di scia* fra aerei in decollo e atterraggio; la loro discesa e il loro trasporto da parte del vento sono oggetto di modelli numerici operativi derivati esattamente da questo tipo di calcolo.

## 6.6 Possibili miglioramenti

L'analisi del Programma 1.7 può essere raffinata. Poiché l'intensità $\gamma$ non è uniforme, i vortici contenuti in ciascun intervallo inducono in generale una velocità non nulla nel punto medio dell'intervallo stesso: il vortice discreto posto lì si muove anche sotto l'influenza *propria*, moto che la (6.5) trascura. Per essere realistici, il vortice discreto andrebbe collocato nel punto del segmento in cui la velocità autoindotta è nulla; il calcolo è semplice per un segmento rettilineo, ma dopo la deformazione il punto si sposta e la sua determinazione su una curva arbitraria diventa impraticabile. In alternativa il foglio si può sostituire con vortici *di uguale intensità*, non equispaziati: Moore ha mostrato che le due discretizzazioni concordano notevolmente.

**Progetto: vortici di uguale intensità**

Sostituire il foglio con 40 vortici di uguale intensità $G = 1/20$ sulla metà sinistra (e $-1/20$ sulla destra), posizionati in modo che ogni intervallo contenga la stessa circolazione: le ascisse si ottengono invertendo $\Gamma(x)/\Gamma_0 = \sqrt{1-X^2}$ nei valori $1-(k-\frac{1}{2})/20$, $k = 1,\dots,20$. Disegnare la configurazione iniziale (i vortici si addensano alle estremità, dove $\gamma$ è singolare) e confrontare le forme del foglio con quelle di Fig. 6.4 agli stessi istanti.

## 6.7 Sintesi del capitolo

**Da ricordare**

- Un'ala finita rilascia in scia un foglio vorticoso di intensità $\gamma = \mathrm{d}\Gamma/\mathrm{d}x$; per il carico ellittico le due metà portano circolazione $\pm\Gamma_0$.
- Discretizzando il foglio in vortici puntiformi, la sua evoluzione è un problema a $N$ corpi governato dalla legge di Biot–Savart bidimensionale: $2m$ ODE del primo ordine, integrabili con RK4 in forma vettoriale.
- Il passo deve soddisfare $\mathrm{d}T \ll 8\pi/m^2$: infittire i vortici riduce la scala di tempo più rapida.
- Il foglio si arrotola dalle estremità in due spirali controrotanti; il centro scende inizialmente a $\Gamma_0/(2a)$ e rallenta verso metà di tale valore. A tempi lunghi la discretizzazione puntiforme diventa caotica: servono vortici concentrati, nuclei regolarizzati o $m$ maggiore.
- Il risultato adimensionale si trasferisce a qualunque ala con il tempo caratteristico $2\pi a^2/\Gamma_0$: per un 747 al decollo, 5,45 s.

# Appendice A

# Simboli, listati di supporto e istruzioni

## A.1 Corrispondenza fra simboli matematici e variabili dei programmi

## A.2 Listati di supporto

I listati seguenti completano quelli riportati nei capitoli.



stile_grafici.py

```python
"""
stile_grafici.py -- impostazioni grafiche comuni (palette "della casa").
Importato da tutti gli script della monografia.
"""
import os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Palette coerente con raucci-aero.sty
BLU      = "#1E2761"
ARANCIO  = "#DC7633"
ROSSO    = "#C0392B"
VERDE    = "#1E8449"
GIALLO   = "#B7950B"
VIOLA    = "#6C3483"
GRIGIO   = "#566573"
PALETTE  = [BLU, ARANCIO, VERDE, VIOLA, ROSSO, GIALLO, GRIGIO]

plt.rcParams.update({
    "font.family": "serif",
    "font.size": 9,
    "axes.labelsize": 9,
    "axes.titlesize": 9,
    "legend.fontsize": 8,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "axes.prop_cycle": matplotlib.cycler(color=PALETTE),
    "axes.grid": True,
    "grid.alpha": 0.3,
    "grid.linestyle": ":",
    "lines.linewidth": 1.4,
    "figure.dpi": 120,
    "savefig.bbox": "tight",
    "savefig.pad_inches": 0.02,
```

Tabella A.1: Principali variabili dei programmi (nomi Python) e loro significato. I nomi ricalcano, in minuscolo, quelli tradizionali dei programmi FORTRAN della letteratura.

| Programma | Simbolo | Significato |
|---|---|---|
| *Comuni (ode_solvers, prog1_1, 1_2, 1_4, 1_5)* | | |
| `A`, `B`, `C` | $A$, $B$, $C$ | $1+\frac{1}{2}\bar{\rho}$; $(1-\bar{\rho})g$; $3\bar{\rho}/(4d)$, eq. (2.11) |
| `rb`, `RHOBAR` | $\bar{\rho}$ | rapporto di densità $\rho_f/\rho$ |
| `RHO`, `RHOF` | $\rho$, $\rho_f$ | densità del corpo e del fluido [kg/m$^3$] |
| `NU`, `G` | $\nu$, $g$ | viscosità cinematica [m$^2$/s], gravità [m/s$^2$] |
| `d`, `D` | $d$ | diametro della sfera [m] |
| `h`, `dT` | $h$, d$T$ | passo di integrazione (dimensionale o no) |
| `cd_sfera(Re)` | $c_d(Re)$ | coefficiente di resistenza della sfera, eq. (2.3)–(2.7) |
| `d1z`, `d1v`, ... | $\Delta_1 z_i$, $\Delta_1 v_i$, ... | incrementi RK4, eq. (2.12) |
| `k1`, ..., `k4` | $\Delta_1\mathbf{y}$, ... | incrementi RK4 in forma vettoriale |
| *Programma 1.1 (caduta)* | | |
| `T, Z, V, RE` | $t_i$, $z_i$, $v_i$, $Re_i$ | storie temporali; `ZV`, `VV`: soluzione nel vuoto |
| `tmax`, `z0`, `v0` | $t_{\max}$, $z_0$, $v_0$ | tempo massimo e condizioni iniziali |
| *Programma 1.2 (pendolo)* | | |
| `l`, `theta0` | $l$, $\theta_0$ | lunghezza perno-centro [m], angolo iniziale [gradi] |
| `y`, `v` | $y = l\theta$, $v$ | spostamento lungo l'arco [m] e velocità tangenziale [m/s] |
| *Programma 1.3 (ala su molla)* | | |
| `beta` | $\beta$ | $\rho_f g S/(2k)$, eq. (3.9) |
| `t_rif`, `l_rif`, `v_rif` | $t_r$, $l_r$, $v_r$ | grandezze di riferimento, eq. (3.8) |
| `U`, `Z`, `V`, `T` | $U$, $Z$, $V$, $T$ | vento, spostamento, velocità e tempo adimensionali |
| `alpha0`, `ALPHA_STALLO` | $\alpha_0$, $18^\circ$ | calettamento e limite di stallo [rad] |
| `cl(alpha)` | $c_l$ | coefficiente di portanza, eq. (3.5) |
| *Programmi 1.4, 1.5, 1.8 (proiettile)* | | |
| `s = [x, y, u, v]` | $(x, y, u, v)$ | stato: posizione [m] e velocità [m/s] |
| `vento(x, y, t)` | $(u_f, v_f)$ | componenti della velocità del fluido |
| `w0`, `theta0_deg` | $w_0$, $\theta_0$ | velocità iniziale e elevazione |
| `sP`, `sQ` | $P$, $Q$ | ultimi due stati sopra/sotto l'orizzonte |
| `theta_old`, `xr_old` | $(\theta_0)_{\text{old}}$, $(x_r)_{\text{old}}$ | punto corrente nel dimezzamento |
| `delta`, `eps` | $\delta$, $\varepsilon$ | incremento e tolleranza [gradi] |
| *Programma 1.6 (aliante)* | | |
| `A`, `B` | $A$, $B$ | portanza iniziale/peso; $c_d/c_l$, eq. (5.3) |
| `s = [X, Y, U, V]` | $(X, Y, U, V)$ | stato adimensionale |
| *Programma 1.7 (scia)* | | |
| `m`, `dX` | $m$, $1/m$ | numero di vortici; mezza spaziatura iniziale |
| `X0`, `G` | $X_i(0)$, $G_i$ | ascisse iniziali e intensità, eq. (6.7) |
| `s = [X..., Y...]` | $(X_i, Y_i)$ | stato: $2m$ coordinate |
| `DX`, `DY`, `R2` | $X_i - X_j$, $Y_i - Y_j$, $R_{ij}^2$ | matrici $m \times m$ delle differenze |

```
    "axes.spines.top": False,
    "axes.spines.right": False,
    "legend.frameon": False,
})

# Le figure vengono salvate nella cartella "figure" accanto a quella degli script
# (creata se manca): cosi' ogni programma funziona anche se scaricato da solo.
FIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "figure")


def salva(fig, nome):
    """Salva la figura in PDF nella cartella delle figure del progetto LaTeX."""
    os.makedirs(FIG_DIR, exist_ok=True)
    fig.savefig(os.path.join(FIG_DIR, f"{nome}.pdf"))
    plt.close(fig)
    print(f"  -> figura salvata: {nome}.pdf")
```

Listato A.1: `stile_grafici.py`: impostazioni comuni di Matplotlib.

es01_ordine_stabilita.py

```
"""
es01_ordine_stabilita.py
Esperimenti numerici del Capitolo 1 (metodi a un passo):
  (a) verifica dell'ordine di accuratezza di Eulero, RK2, RK4
      sul problema test  dx/dt = x - t^2 + 1,  x(0) = 0.5
      la cui soluzione esatta e'  x(t) = (t+1)^2 - 0.5 e^t
  (b) stabilita' assoluta di Eulero sul problema modello dx/dt = -lambda x
"""
import numpy as np
import matplotlib.pyplot as plt
from ode_solvers import integra
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO


# ---------------------------------------------------------------- (a)
def f_test(t, x):
    return x - t ** 2 + 1.0


def x_esatta(t):
    return (t + 1.0) ** 2 - 0.5 * np.exp(t)


print("=== (a) Verifica dell'ordine: errore globale in t = 2 ===")
passi = [0.4, 0.2, 0.1, 0.05, 0.025]
errori = {m: [] for m in ("eulero", "rk2", "rk4")}
for h in passi:
    n = int(round(2.0 / h))
    for m in errori:
        t, X = integra(f_test, 0.0, 0.5, h, n, metodo=m)
        errori[m].append(abs(X[-1, 0] - x_esatta(2.0)))

print(f"{'h':>7} {'Eulero':>12} {'RK2':>12} {'RK4':>12}")
for i, h in enumerate(passi):
    print(f"{h:7.3f} {errori['eulero'][i]:12.3e} {errori['rk2'][i]:12.3e} "
          f"{errori['rk4'][i]:12.3e}")

print("\nOrdine osservato p = log2(E(h)/E(h/2)):")
for m in errori:
    e = np.array(errori[m])
    p = np.log2(e[:-1] / e[1:])
    print(f"  {m:7s}: " + "  ".join(f"{v:5.2f}" for v in p))
```

```
43
44 fig, ax = plt.subplots(figsize=(4.2, 3.0))
45 for m, lab, c in (("eulero", "Eulero (ordine 1)", BLU),
46                   ("rk2", "RK2 (ordine 2)", ARANCIO),
47                   ("rk4", "RK4 (ordine 4)", VERDE)):
48     ax.loglog(passi, errori[m], "o-", label=lab, color=c)
49 hh = np.array(passi)
50 ax.loglog(hh, 0.6 * hh ** 4, ":", color=VERDE, lw=0.8)
51 ax.loglog(hh, 0.5 * hh ** 2, ":", color=ARANCIO, lw=0.8)
52 ax.loglog(hh, 1.0 * hh, ":", color=BLU, lw=0.8)
53 ax.set_xlabel("passo $h$")
54 ax.set_ylabel("errore globale $|x_N - x(2)|$")
55 ax.legend()
56 salva(fig, "es01_ordine")
57
58 # ------------------------------------------------------------------ (b)
59 print("\n=== (b) Stabilita' di Eulero su dx/dt = -lambda x ===")
60 lam = 1.0
61 fig, ax = plt.subplots(figsize=(4.6, 3.0))
62 tt = np.linspace(0, 12, 400)
63 ax.plot(tt, np.exp(-lam * tt), "k-", lw=1.0, label="esatta $e^{-\\lambda t}$")
64 for hl, c in ((0.5, VERDE), (1.5, ARANCIO), (2.1, ROSSO)):
65     h = hl / lam
66     n = int(12 / h)
67     t, X = integra(lambda t, x: -lam * x, 0.0, 1.0, h, n, metodo="eulero")
68     ax.plot(t, X[:, 0], "o-", ms=3, color=c, label=f"Eulero, $h\\lambda={hl}$")
69     print(f"  h*lambda = {hl}: x_N = {X[-1,0]: .3e}  (fattore 1-h*lambda =
    {1-hl:+.2f})")
70 ax.set_ylim(-3, 3)
71 ax.set_xlabel("$t$")
72 ax.set_ylabel("$x$")
73 ax.legend(ncol=2)
74 salva(fig, "es01_stabilita")
75
76 # ------------------------------------------------------------------ (c)
77 # Regioni di stabilita' assoluta nel piano complesso z = h*lambda
78 print("\n=== (c) Regioni di stabilita' assoluta ===")
79 re = np.linspace(-4.0, 1.0, 400)
80 im = np.linspace(-3.5, 3.5, 400)
81 RE, IM = np.meshgrid(re, im)
82 z = RE + 1j * IM
83 R = {
84     "Eulero": 1 + z,
85     "RK2": 1 + z + z ** 2 / 2,
86     "RK4": 1 + z + z ** 2 / 2 + z ** 3 / 6 + z ** 4 / 24,
87 }
88 fig, ax = plt.subplots(figsize=(4.2, 3.6))
89 for (nome, Rz), c in zip(R.items(), (BLU, ARANCIO, VERDE)):
90     ax.contour(RE, IM, np.abs(Rz), levels=[1.0], colors=[c], linewidths=1.4)
91     ax.plot([], [], color=c, label=nome)
92 ax.axhline(0, color="k", lw=0.5)
93 ax.axvline(0, color="k", lw=0.5)
94 ax.set_aspect("equal")
95 ax.set_xlabel("Re$(h\\lambda)$")
96 ax.set_ylabel("Im$(h\\lambda)$")
97 ax.legend(loc="upper left")
98 salva(fig, "es01_regioni")
99 for nome in R:
100     # intersezione con l'asse reale negativo
101     rr = np.linspace(-3.0, 0, 30001)
```

```
    Rr = {"Eulero": 1 + rr, "RK2": 1 + rr + rr**2/2,
          "RK4": 1 + rr + rr**2/2 + rr**3/6 + rr**4/24}[nome]
    lim = rr[np.where(np.abs(Rr) <= 1.0)[0][0]]
    print(f"  {nome:7s}: stabile sull'asse reale per  -{abs(lim):.3f} <= h*lambda
    <= 0")
```

Listato A.2: `es01_ordine_stabilita.py`: esperimenti sull'ordine e sulla stabilità (Cap. 1), versione completa.

prog1_1b_problemi.py

```
"""
prog1_1b_problemi.py
Problemi collegati al Programma 1.1:
  (1) pallina da ping-pong rilasciata sul fondo di una vasca d'acqua
      (effetto del passo h sulla stabilita')
  (2) velocita' terminale: indipendenza dalla velocita' iniziale
  (3) velocita' terminale di gocce di pioggia (sfera rigida d'acqua)
"""
import numpy as np
import matplotlib.pyplot as plt
from prog1_1_caduta_sfera import caduta, costanti
from ode_solvers import cd_sfera
from stile_grafici import salva, PALETTE, BLU, ARANCIO, ROSSO

# ------------------------------------------------- (1) ping-pong in acqua
print("=== (1) Pallina da ping-pong in acqua ===")
d, rho, rhof, nu = 0.036, 1.22, 1000.0, 1.0e-6
A, B, C = costanti(d, rho, rhof)
print(f"A = {A:.2f}  B = {B:.1f} m/s^2  C = {C:.1f} 1/m   (accelerazione iniziale
    B/A = {B/A:.1f} m/s^2)")
fig, ax = plt.subplots(figsize=(4.4, 3.0))
for h, c in ((0.1, ROSSO), (0.01, BLU)):
    T, Z, V, RE = caduta(d, h=h, tmax=0.5, rho=rho, rhof=rhof, nu=nu)
    ax.plot(T, V, "o-", ms=2.5, color=c, label=f"$h = {h}$ s")
    print(f"  h = {h}: v(0.5 s) = {V[-1]:.3f} m/s   Re = {RE[-1]:.0f}")
# velocita' terminale teorica (c_d = 0.5 per 400 < Re < 3e5): v_t = sqrt(B/(C c_d))
    (verso l'alto)
vt = -np.sqrt(-B / (C * 0.5))
ax.axhline(vt, color="gray", ls="--", lw=0.8, label=f"$v_t = {vt:.2f}$ m/s")
ax.set_xlabel("$t$ [s]"); ax.set_ylabel("$v$ [m/s] (positivo verso il basso)")
ax.set_ylim(-2.0, 0.5)
ax.legend()
salva(fig, "prog1_1b_pingpong")
print(f"  v_t teorica = {vt:.3f} m/s;  tempo caratteristico tau = A/(2 C c_d |v_t|)
    = "
      f"{A/(2*C*0.5*abs(vt)):.3f} s")

# ------------------------------------------------- (2) effetto di v0
print("\n=== (2) Velocita' iniziale diversa, sfera d'acciaio d = 0.01 m ===")
fig, ax = plt.subplots(figsize=(4.4, 3.0))
for v0, c in ((0.0, PALETTE[0]), (-30.0, PALETTE[1]), (80.0, PALETTE[2])):
    T, Z, V, RE = caduta(0.01, h=0.1, tmax=12.0, v0=v0)
    ax.plot(T, V, color=c, label=f"$v_0 = {v0:+.0f}$ m/s")
    print(f"  v0 = {v0:+5.1f} m/s -> v(12 s) = {V[-1]:.2f} m/s")
ax.axhline(41.39, color="gray", ls="--", lw=0.8, label="$v_t$")
ax.set_xlabel("$t$ [s]"); ax.set_ylabel("$v$ [m/s]")
ax.legend()
salva(fig, "prog1_1b_v0")

# ------------------------------------------------- (3) gocce di pioggia
print("\n=== (3) Velocita' terminale di gocce d'acqua in aria (sfera rigida) ===")
# valori misurati (Gunn & Kinzer, 1949) usati come riferimento sperimentale
```

```
d_mis = np.array([0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 5.8]) * 1e-3
v_mis = np.array([2.06, 4.03, 5.40, 6.49, 7.41, 8.06, 8.53, 8.83, 9.00, 9.09, 9.14, 9.17])
d_calc = np.logspace(np.log10(0.1e-3), np.log10(6e-3), 30)
v_calc = []
for dd in d_calc:
    T, Z, V, RE = caduta(dd, h=0.005, tmax=5.0, rho=1000.0)
    v_calc.append(V[-1])
v_calc = np.array(v_calc)
for dd, vv in zip(d_calc[::6], v_calc[::6]):
    print(f"  d = {dd*1e3:5.2f} mm  v_t = {vv:6.2f} m/s  Re = {vv*dd/1.49e-5:8.0f}")
fig, ax = plt.subplots(figsize=(4.4, 3.0))
ax.plot(d_calc * 1e3, v_calc, color=BLU, label="sfera rigida (Programma 1.1)")
ax.plot(d_mis * 1e3, v_mis, "o", color=ARANCIO, ms=4, label="misure (Gunn-Kinzer)")
ax.set_xlabel("diametro [mm]"); ax.set_ylabel("$v_t$ [m/s]")
ax.legend(loc="lower right")
salva(fig, "prog1_1b_pioggia")
```

Listato A.3: `prog1_1b_problemi.py`: problemi collegati al Programma 1.1 (Cap. 2).

prog1_5_gittata_max.py

```
"""
prog1_5_gittata_max.py  --  Programma 1.5
Angolo di tiro ottimo (gittata massima) di un proiettile sferico
con il metodo del dimezzamento dell'intervallo.
"""
import numpy as np
import matplotlib.pyplot as plt
from prog1_4_proiettile import modello, traiettoria, gittata
from stile_grafici import salva, BLU, ARANCIO, VERDE


def gittata_massima(f, w0, h=0.1, theta_old=0.0, xr_old=0.0, delta=10.0, eps=1e-3,
                    verbose=False):
    """
    Metodo del dimezzamento dell'intervallo (Sez. 1.5):
    theta_new = theta_old + delta; se la gittata non cresce, delta <- -delta/2.
    Restituisce (theta_ottimo, x_r massima, numero di valutazioni).
    """
    n = 0
    while abs(delta) >= eps:
        theta_new = theta_old + delta
        n += 1
        _, sP, sQ = traiettoria(f, w0, theta_new, h)
        xr_new = gittata(sP, sQ)
        if verbose:
            print(f"    n={n:2d}  theta={theta_new:9.4f}  x_r={xr_new:9.4f}  delta={delta:+9.4f}")
        if xr_old >= xr_new:
            delta = -delta / 2.0
        else:
            theta_old, xr_old = theta_new, xr_new
    if xr_old >= xr_new:
        return theta_old, xr_old, n
    return theta_new, xr_new, n


if __name__ == "__main__":
    d, w0 = 0.01, 50.0
    print(f"Proiettile sferico d'acciaio d = {d} m, w0 = {w0} m/s")
    print(f"{'u_f [m/s]':>10} {'theta_ott [deg]':>16} {'x_r max [m]':>12} {'N':>4}")
```

```
    for uf in (20.0, 0.0, -20.0):
        f = modello(d, vento=lambda x, y, t, uf=uf: (uf, 0.0))
        th, xr, n = gittata_massima(f, w0, verbose=(uf == 0.0))
        print(f"{uf:10.1f} {th:16.3f} {xr:12.2f} {n:4d}")

    # ---------------- Fig. 1.5.4: angolo ottimo in funzione del diametro
    print("\nAngolo ottimo in funzione del diametro (puo' richiedere qualche secondo)...")
    diam = np.concatenate([np.arange(0.0025, 0.0101, 0.0005),
                           np.arange(0.01125, 0.0201, 0.00125),
                           np.arange(0.025, 0.1001, 0.005)])
    fig, ax = plt.subplots(figsize=(4.8, 3.0))
    for uf, c, mk, lab in ((20.0, VERDE, "s", "$u_f = +20$ m/s"), (0.0, BLU, "o", "$u_f = 0$"),
                           (-20.0, ARANCIO, "+", "$u_f = -20$ m/s")):
        angoli = []
        for dd in diam:
            f = modello(dd, vento=lambda x, y, t, uf=uf: (uf, 0.0))
            th, xr, n = gittata_massima(f, w0)
            angoli.append(th)
        ax.semilogx(diam, angoli, mk, color=c, ms=4, mfc="none", label=lab)
        print(f"  u_f = {uf:+5.1f}: theta_ott da {angoli[0]:.2f} (d={diam[0]}) a {angoli[-1]:.2f} (d={diam[-1]})")
    ax.axhline(45, color="gray", lw=0.6, ls="--")
    ax.set_xlabel("diametro $d$ [m]"); ax.set_ylabel("$\\theta_0$ ottimo [gradi]")
    ax.set_ylim(25, 50); ax.legend(fontsize=7, loc="lower right")
    salva(fig, "prog1_5_angolo_ottimo")

    # ---------------- Problema 4.2: sfera di 0.09 m
    print("\nProblema 4.2 -- sfera d = 0.09 m:")
    for uf in (20.0, 0.0, -20.0):
        f = modello(0.09, vento=lambda x, y, t, uf=uf: (uf, 0.0))
        th, xr, n = gittata_massima(f, w0)
        print(f"  u_f = {uf:+5.1f}: theta_ott = {th:6.2f} deg, x_r max = {xr:7.2f} m, "
              f"Re iniziale = {abs(w0*np.cos(np.radians(th)) - uf)*0.09/1.49e-5:.0f}")
```

Listato A.4: `prog1_5_gittata_max.py`: Programma 1.5 completo (Cap. 4).

```
"""
prog1_8_cannone.py  --  Problema 4.3
Proietto d'artiglieria (sfera d'acciaio equivalente, d = 0.3 m, c_d = 0.4 costante,
w0 = 800 m/s) in atmosfera a densita' costante e in atmosfera esponenziale
rho_f = 1.22 exp(-0.000118 y). Angolo ottimo e gittata massima con il metodo
del dimezzamento dell'intervallo.
"""
import numpy as np
import matplotlib.pyplot as plt
from ode_solvers import passo_rk4
from prog1_4_proiettile import traiettoria, gittata
from prog1_5_gittata_max import gittata_massima
from stile_grafici import salva, BLU, ARANCIO

RHO, G, d, CD, w0 = 8000.0, 9.8, 0.3, 0.4, 800.0


def modello_cannone(densita):
    """f(t, s) con c_d costante e densita' dell'aria funzione della quota y."""
    def f(t, s):
        x, y, u, v = s
```

prog1_8_cannone.py

```
        rb = densita(y) / RHO
        A, B, C = 1 + rb / 2, (1 - rb) * G, 3 * rb / (4 * d)
        w = np.hypot(u, v)
        return np.array([u, v, -C * CD * u * w / A, (-B - C * CD * v * w) / A])
    return f


if __name__ == "__main__":
    atmosfere = (("densita' costante", lambda y: 1.22, BLU),
                 ("atmosfera esponenziale", lambda y: 1.22 * np.exp(-0.000118 * y), ARANCIO))
    h = 0.05
    fig, ax = plt.subplots(figsize=(5.4, 2.8))
    for nome, dens, c in atmosfere:
        f = modello_cannone(dens)
        th, xr, n = gittata_massima(f, w0, h=h, delta=10.0, eps=1e-2)
        S, sP, sQ = traiettoria(f, w0, th, h)
        ax.plot(S[:, 1] / 1e3, S[:, 2] / 1e3, color=c, label=f"{nome}: $\\theta_0={th:.1f}^\\circ$")
        print(f"{nome:24s}: theta_ott = {th:6.2f} deg, gittata max = {xr/1e3:6.2f} km, "
              f"quota max = {S[:,2].max()/1e3:5.2f} km, tempo di volo ~ {S[-1,0]:5.1f} s")
    # confronto a parita' di angolo (45 gradi)
    for nome, dens, c in atmosfere:
        S, sP, sQ = traiettoria(modello_cannone(dens), w0, 45.0, h)
        print(f"  a 45 deg, {nome:22s}: gittata = {gittata(sP, sQ)/1e3:6.2f} km")
    ax.set_xlabel("$x$ [km]"); ax.set_ylabel("$y$ [km]"); ax.set_aspect("equal")
    ax.legend(fontsize=7)
    salva(fig, "prog1_8_cannone")
```

Listato A.5: `prog1_8_cannone.py`: Problema 4.3 completo (Cap. 4).

## A.3 Come riprodurre i risultati

**Istruzioni**

1. Requisiti: Python $\geq$ 3.9 con `numpy`, `scipy`, `matplotlib` (`pip install numpy scipy matplotlib`); una distribuzione TEX con `pdflatex` e i pacchetti `tcolorbox`, `pgfplots`, `siunitx`, `listings`, `babel-italian` (tutti inclusi in TEX Live / MacTeX completo).
2. Dalla cartella `codice/`: eseguire gli script nell'ordine `es01`, `prog1_1`, `prog1_1b`, `prog1_2`, `prog1_3`, `prog1_4`, `prog1_5`, `prog1_6`, `prog1_7`, `prog1_8`, `prog1_9` e infine `esercizi_svolti.py` (oppure `bash esegui_tutti.sh`). Le figure vengono salvate in `figure/`; `prog1_5`, `prog1_8` ed `esercizi_svolti` richiedono da alcuni secondi a qualche minuto.
3. Dalla cartella principale: `pdflatex main.tex` due volte (o `latexmk -pdf main.tex`).

## A.4 Codici in rete e QR code

I programmi sono pubblicati all'indirizzo

`https://braucci.github.io/CFD/monografia1/codici/`

La pagina elenca tutti i file con le rispettive dipendenze, li mostra con l'evidenziazione della sintassi e permette di scaricarli singolarmente o in un unico archivio. Il QR code nel margine accanto a ogni listato codifica l'indirizzo del file corrispondente e, nel PDF, è anche un collegamento cliccabile.

**Per l'autore: configurazione e pubblicazione**

1. In `main.tex` la macro `\nomemonografia` (qui `monografia1`) determina tutti gli indirizzi: `\urlcodici` vale `https://braucci.github.io/CFD/<nome>/codici/` e `\urlfile{file.py}` aggiunge il nome del file. Per la monografia successiva basta cambiare `\nomemonografia`.
2. Un listato completo si inserisce con `\listatopy[opzioni]{file.py}` (legge `codice/file.py` e mette il QR nel margine); prima di un estratto in `lstlisting` si scrive `\qrlistato{file.py}`.
3. `python3 pubblica_github.py` crea `github/monografia1/` con il PDF, la cartella `codici/`, la pagina `index.html` e l'archivio zip: va copiata così com'è nella cartella `CFD/` del repository.

## A.5 Come sostituire le figure

Le figure del documento sono di due tipi, gestite da due macro definite in `main.tex`.

**`\figuraesterna[larghezza]{nome}{testo}`** figure schematiche (sistemi di riferimento, diagrammi delle forze, costruzioni geometriche). Se nella cartella `immagini/` esiste il file `nome.png` (oppure `.pdf` o `.jpg`) viene incluso con la larghezza indicata; altrimenti al suo posto compare un *segnaposto* grigio con il nome del file atteso e una breve descrizione. Per completare il documento basta quindi copiare in `immagini/` le immagini con i nomi elencati in Tab. A.2 e ricompilare: nessuna modifica al sorgente.

**`\figuragenerata[larghezza]{nome}`** grafici prodotti dagli script Python e letti dalla cartella `figure/`. Anche questi possono essere sostituiti: se esiste `immagini/nome.png` questo ha la precedenza sul PDF generato. I nomi coincidono con quelli dei file in `figure/` (ad esempio `prog1_1_cd`, `prog1_7_scia`).

Tabella A.2: Figure con segnaposto e nome del file atteso in `immagini/`.

| Figura | File | Contenuto |
|---|---|---|
| 1.1 | `fig_1_1_1.png` | soluzione numerica di un IVP: curva $x(t)$ e nodi $x_i$ |
| 1.2 | `fig_eulero.png` | interpretazione geometrica di Eulero (facoltativa: in sua assenza resta il disegno TikZ) |
| 2.1 | `fig_1_2_1.png` | sfera in caduta libera e asse $z$ |
| 3.1 | `fig_1_3_1.png` | pendolo semplice in un fluido |
| 3.4 | `fig_1_3_3.png` | ala su molla e triangolo delle velocità |
| 4.1 | `fig_1_4_1.png` | moto bidimensionale di un corpo in un fluido |
| 4.3 | `fig_1_5_2.png` | determinazione della gittata (punti $P$ e $Q$) |
| 4.4 | `fig_1_5_3.png` | metodo del dimezzamento dell'intervallo |
| 5.1 | `fig_1_6_1.png` | forze su un aliante |
| 6.1 | `fig_1_7_1.png` | sistema di vortici di un'ala finita |
| 6.2 | `fig_1_7_3.png` | foglio vorticoso discretizzato in $m$ vortici |
| 6.3 | `fig_1_7_4.png` | velocità indotta dal vortice $j$ sul vortice $i$ |

# Appendice B

# Esercizi svolti

Gli esercizi che seguono completano i capitoli: alcuni verificano sperimentalmente un'affermazione del testo, altri estendono i programmi a situazioni nuove, altri ancora sono piccoli progetti. Ogni esercizio è svolto con lo stesso schema: *testo*, *impostazione* (che cosa ci si aspetta e perché, prima di calcolare), *svolgimento* passo per passo con il codice essenziale, *risultato* e *commento*. Il codice completo è nel file `esercizi_svolti.py` (Listato B.1), in cui ogni esercizio è la funzione `es_NN()`; i numeri riportati sono quelli stampati dal programma.

## B.1 Esercizio 1 – I metodi di Heun e di Ralston

**Testo**

Aggiungere al modulo `ode_solvers.py` il metodo di Heun ($a_1 = a_2 = \frac{1}{2}$, $p = q = 1$) e il metodo di Ralston ($a_2 = \frac{2}{3}$, $p = q = \frac{3}{4}$); verificare con il problema test (1.12) che entrambi sono del secondo ordine e confrontare le costanti d'errore con quelle del metodo del punto medio.

**Impostazione.** Le tre varianti soddisfano le condizioni (1.8), quindi hanno tutte errore globale $E \simeq Kh^2$: ciò che le distingue è la costante $K$, che dipende dai termini di ordine $h^3$ dello sviluppo (quelli non annullati dalle condizioni del secondo ordine) e quindi *dal problema*. Ralston ha scelto $a_2 = \frac{2}{3}$ per minimizzare un *limite superiore* dell'errore di troncamento, non l'errore su ogni specifica equazione: non è detto che risulti il più accurato su un problema dato.

**Svolgimento.**

1. Dalle (1.8): per Heun, $a_2 = \frac{1}{2} \Rightarrow p = q = 1$, cioè la seconda pendenza si valuta alla fine dell'intervallo, nel punto predetto da un passo di Eulero, e si fa la media delle due pendenze (è il metodo dei trapezi esplicito). Per Ralston, $a_2 = \frac{2}{3} \Rightarrow a_1 = \frac{1}{3}$, $p = q = \frac{3}{4}$.
2. Le due funzioni si aggiungono a `ode_solvers.py` con la stessa interfaccia di `passo_rk2`, e si registrano nel dizionario `METODI` così che `integra` le accetti per nome.

```python
def passo_heun(f, t, y, h):
    k1 = h * f(t, y)
    k2 = h * f(t + h, y + k1)
    return y + 0.5 * (k1 + k2)

def passo_ralston(f, t, y, h):
    k1 = h * f(t, y)
    k2 = h * f(t + 0.75 * h, y + 0.75 * k1)
    return y + (k1 + 2.0 * k2) / 3.0
```

ode_solvers.py

3. Si ripete la verifica dell'ordine del §1.5 con i tre metodi e si stima $K = E(h)/h^2$ al passo più piccolo.

**Risultato**

| $h$ | punto medio | Heun | Ralston |
|---|---|---|---|
| 0,400 | $5{,}924 \times 10^{-2}$ | $2{,}626 \times 10^{-1}$ | $1{,}609 \times 10^{-1}$ |
| 0,100 | $3{,}747 \times 10^{-3}$ | $1{,}890 \times 10^{-2}$ | $1{,}133 \times 10^{-2}$ |
| 0,025 | $2{,}304 \times 10^{-4}$ | $1{,}216 \times 10^{-3}$ | $7{,}233 \times 10^{-4}$ |
| ordine osservato | 2,01 | 1,99 | 1,99 |
| $K = E/h^2$ | 0,37 | 1,95 | 1,16 |

Tutti e tre i metodi sono del secondo ordine (Fig. B.1: rette parallele di pendenza 2), ma su *questo* problema il punto medio è cinque volte più accurato di Heun e tre volte più di Ralston.

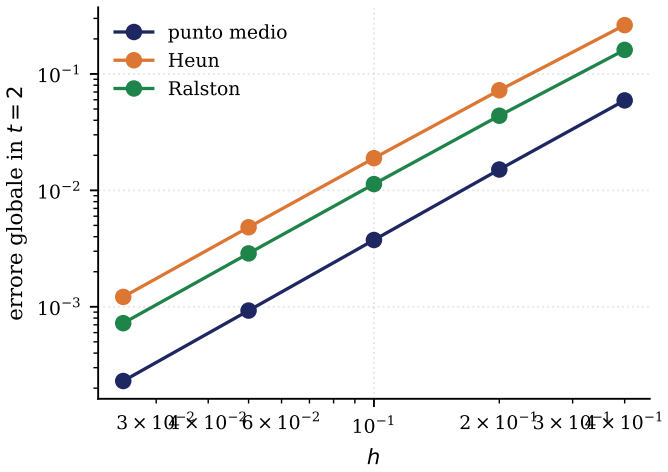



Figura B.1: Esercizio 1: errore globale dei tre metodi RK2.

**Commento.** L'ordine dice come l'errore *scala*, la costante dice *quanto vale*: due metodi dello stesso ordine possono differire di un fattore 5. La costante dipende dalle derivate terze della soluzione e dai coefficienti del metodo; nessuna scelta dei coefficienti è ottima per tutti i problemi, e la scelta di Ralston è ottima solo nel senso di un limite superiore. In pratica, quando l'accuratezza conta, si passa a RK4 piuttosto che cercare l'RK2 migliore.

## B.2 Esercizio 2 – Il limite di stabilità di RK4

**Testo**

Ripetere l'esperimento (b) di `es01` con RK4 per $h\lambda = 2{,}5$, $2{,}785$ e $3$ sul problema $\mathrm{d}x/\mathrm{d}t = -\lambda x$, $x(0) = 1$, e interpretare i risultati con la (1.16).

**Impostazione.** Per il problema modello un passo di RK4 moltiplica la soluzione per $R(z) = 1 + z + z^2/2 + z^3/6 + z^4/24$, $z = -h\lambda$. Dopo $N$ passi, $x_N = R^N$. Calcoliamo *prima* $R$ nei tre casi: $R(-2{,}5) = 0{,}648$, $R(-2{,}785) = 0{,}9996$, $R(-3) = 1{,}375$. Ci aspettiamo quindi decadimento (lento), quasi-stazionarietà, crescita.

**Svolgimento.** Si integra fino a $t = 30$ ($N = 30/h$ passi) con `integra(..., metodo="rk4")` e si confronta $x_N$ con $R^N$.

**Risultato**

| $h\lambda$ | $R$ | $N$ | $R^N$ | $x_N$ calcolato |
|---|---|---|---|---|
| 2,500 | 0,6484 | 12 | $5{,}526 \times 10^{-3}$ | $5{,}526 \times 10^{-3}$ |
| 2,785 | 0,9996 | 10 | $9{,}956 \times 10^{-1}$ | $9{,}956 \times 10^{-1}$ |
| 3,000 | 1,3750 | 10 | $2{,}416 \times 10^{1}$ | $2{,}416 \times 10^{1}$ |

Il calcolo riproduce esattamente $R^N$ (Fig. B.2): per $h\lambda = 2{,}5$ la soluzione decade, ma a $t = 30$ vale $5 \cdot 10^{-3}$ invece di $e^{-30} \simeq 10^{-13}$; per 2,785 resta "congelata" vicino a 1; per 3 esplode.

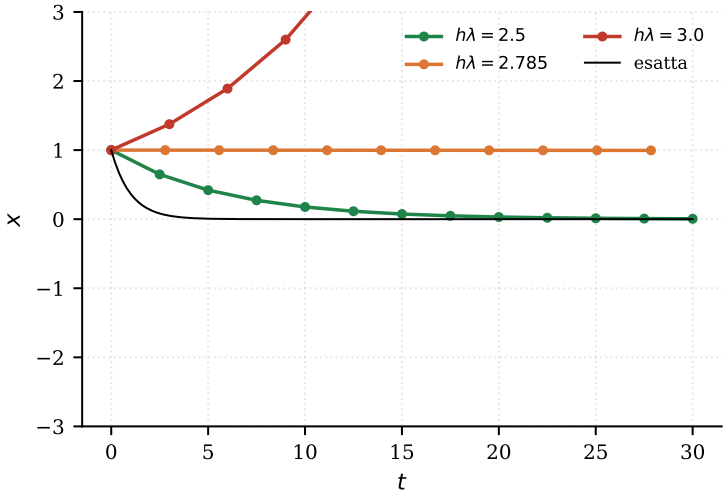



Figura B.2: Esercizio 2: RK4 sul problema modello con tre passi attorno al limite di stabilità.

**Commento.** Diversamente da Eulero, RK4 sull'asse reale negativo non produce oscillazioni di segno fino al limite ($R$ resta positivo), ma il risultato "stabile" per $h\lambda = 2{,}5$ è privo di significato fisico: il decadimento avviene con il fattore 0,65 per passo anziché $e^{-2{,}5} = 0{,}08$. Si ricordi la distinzione del §1.6: la stabilità impedisce l'esplosione, l'accuratezza richiede $h\lambda \ll 1$.

## B.3 Esercizio 3 – L'errore di arrotondamento

**Testo**

Estendere la Tab. 1.1 a $h = 10^{-3}, 10^{-4}, 10^{-5}$ per RK4 e osservare a che punto l'errore smette di decrescere.

**Impostazione.** L'errore totale è la somma di due contributi di segno opposto rispetto ad $h$: il troncamento, $\propto h^4$, che decresce con $h$, e l'arrotondamento, che per $N = T/h$ passi si accumula come $\sim \epsilon_m N \propto \epsilon_m / h$, con $\epsilon_m \simeq 2 \cdot 10^{-16}$ la precisione della macchina in doppia precisione. Il minimo si trova dove i due sono confrontabili: $h^4 \sim \epsilon_m / h \Rightarrow h \sim \epsilon_m^{1/5} \simeq 10^{-3}$.

**Svolgimento.** Basta estendere la lista dei passi; con $h = 10^{-5}$ i passi sono 200 000 (qualche secondo in Python puro).

**Risultato**

| $h$ | $N$ | errore |
|---|---|---|
| $1 \times 10^{-1}$ | 20 | $6{,}990 \times 10^{-6}$ |
| $1 \times 10^{-2}$ | 200 | $7{,}138 \times 10^{-10}$ |
| $1 \times 10^{-3}$ | 2000 | $1{,}705 \times 10^{-13}$ |
| $1 \times 10^{-4}$ | 20 000 | $1{,}048 \times 10^{-12}$ |
| $1 \times 10^{-5}$ | 200 000 | $3{,}748 \times 10^{-12}$ |

Da $h = 10^{-1}$ a $10^{-3}$ l'errore cala di $10^4$ per decade, come previsto; a $10^{-3}$ tocca il minimo ($1{,}7 \cdot 10^{-13}$) e poi *risale* proporzionalmente a $N$ (Fig. B.3).

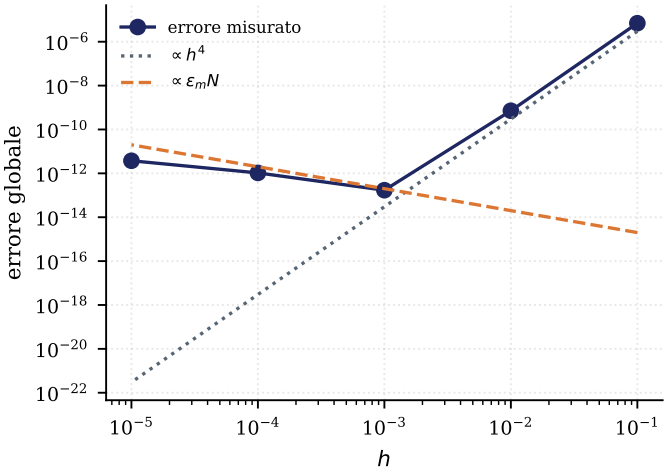



Figura B.3: Esercizio 3: errore di RK4 fino al limite dell'arrotondamento.

**Commento.** Esiste un passo ottimo al di sotto del quale "raffinare" peggiora il risultato. Per RK4 e doppia precisione è dell'ordine di $10^{-3}$ volte la scala dei tempi del problema: ben al di sotto di quanto l'accuratezza ingegneristica richieda, per cui in pratica l'arrotondamento non è mai il fattore limitante con RK4. Lo diventa invece con Eulero, che per arrivare a $10^{-6}$ richiederebbe $h \sim 10^{-6}$ e $N \sim 10^6$ passi.

## B.4 Esercizio 4 – Sfera d'acciaio in acqua

**Testo**

Calcolare la caduta di una sfera d'acciaio di 1 cm in acqua ($\rho_f = 1000$ kg/m$^3$, $\nu = 10^{-6}$ m$^2$/s): quanto valgono $A$, $B$, $C$ e la costante di tempo? Scegliere $h$ di conseguenza e confrontare la velocità terminale con la (2.13).

**Impostazione.** Con $\bar\rho = 1000/8000 = 0{,}125$ galleggiamento e massa aggiunta non sono più trascurabili: $A = 1{,}0625$ (la massa apparente cresce del 6%), $B = 0{,}875g = 8{,}575$ m/s$^2$ (la gravità efficace cala del 12,5%), $C = 3\bar\rho/(4d) = 9{,}375$ m$^{-1}$, 820 volte quello in aria. Ci aspettiamo una velocità terminale piccola e raggiunta in fretta.

**Svolgimento.**

1. *Stima a priori di $v_t$.* Supponendo $c_d = 0{,}5$: $v_t = \sqrt{B/(Cc_d)} = \sqrt{8{,}575/4{,}69} = 1{,}353$ m/s; il Reynolds corrispondente è $v_t d/\nu = 1{,}35 \cdot 10^4$, nel plateau $400 < Re < 3 \cdot 10^5$: l'ipotesi è coerente.
2. *Costante di tempo.* $\tau = A/(2Cc_d v_t) = 1{,}0625/(2 \cdot 9{,}375 \cdot 0{,}5 \cdot 1{,}353) = 0{,}084$ s; il limite di stabilità di RK4 richiede $h < 2{,}785\tau = 0{,}23$ s. Il passo $h = 0{,}1$ s è quindi ammissibile, ma per l'accuratezza del transitorio conviene $h \le \tau/4 \simeq 0{,}02$ s.
3. Si lancia `caduta(0.01, h, tmax=2, rho=8000, rhof=1000, nu=1e-6)` con $h =$ 0,1, 0,02, 0,005 s.

**Risultato**

Per tutti e tre i passi $v(2\,\text{s}) = 1{,}3525$ m/s e $Re = 13\,525$; $z(2\,\text{s}) = 2{,}5476$ m con $h = 0{,}1$ e 2,5479 m con i passi più fini (differenza di 0,3 mm, dovuta alla cattiva risoluzione del transitorio di 0,3 s). La velocità terminale coincide con la stima della (2.13) e viene raggiunta in circa 0,4 s (Fig. B.4), contro i 10 s in aria.

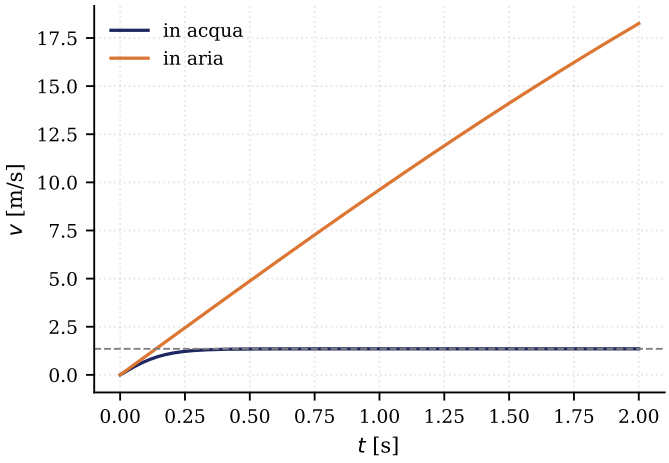



Figura B.4: Esercizio 4: la stessa sfera d'acciaio di 1 cm in acqua e in aria.

**Commento.** È un buon esempio di come *stimare prima di calcolare*: velocità terminale, Reynolds e costante di tempo si ottengono a mano in tre righe, e dicono quale passo usare e quale risultato attendersi. Se il calcolo dà qualcosa di molto diverso dalla stima, c'è un errore nel programma o nella stima, e in entrambi i casi si è imparato qualcosa.

## B.5 Esercizio 5 – La goccia che gela

**Testo**

Verificare numericamente che una goccia d'acqua di 0,2 mm che gela ($\rho$ da 1000 a 917 kg/m$^3$, a parità di massa) rallenta.

**Impostazione.** A parità di massa $\rho d^3$ è costante: il diametro del chicco di ghiaccio è $d_i = d_w(\rho_w/\rho_i)^{1/3} = 0{,}2 \cdot (1{,}0905)^{1/3} = 0{,}2059$ mm. Nel regime di Stokes, a parità di fluido, $v_t \propto \rho d^2$ (eq. (2.14) con $\bar\rho \ll 1$), cioè $v_t \propto \rho^{1/3}$ a massa fissata: gelando, $v_t$ dovrebbe calare del fattore $(917/1000)^{1/3} = 0{,}971$, cioè del 3%.

**Svolgimento.** Si integra con `caduta(d, h=0.002, tmax=1.0, rho=...)` per i due corpi e si legge la velocità a 1 s (il transitorio dura pochi centesimi di secondo).

**Risultato**

Acqua: $v_t = 0{,}5790$ m/s, $Re = 7{,}8$. Ghiaccio: $v_t = 0{,}5625$ m/s, $Re = 7{,}8$. Il chicco è più lento del 2,9%, in accordo con la stima. La formula di Stokes (2.14) darebbe invece 1,20 e 1,16 m/s: il doppio del valore corretto, perché a $Re \simeq 8$ siamo *fuori* dal suo campo di validità ($Re < 1$) e il programma usa il tratto (2.4) della spezzata.

**Commento.** Due lezioni. La prima è fisica: il rallentamento è modesto perché, a parità di massa, la resistenza cresce con $d^2$ ma il peso resta uguale. La seconda è numerica: una formula analitica usata fuori dalle sue ipotesi sbaglia di un fattore 2 senza alcun segnale d'allarme, mentre il programma, che contiene la curva completa $c_d(Re)$, si accorge da solo del regime in cui si trova. Il risultato qualitativo (rallenta) è però lo stesso: le dipendenze parametriche spesso sopravvivono anche quando i valori assoluti no.

## B.6 Esercizio 6 – Caduta da 20 km in atmosfera stratificata

**Testo**

Modificare il Programma 1.1 con una densità $\rho_f(y) = 1{,}22\, e^{-0{,}000118\, y}$ e calcolare la caduta di una sfera d'acciaio di 10 cm rilasciata da 20 km di quota: la velocità passa per un massimo?

**Impostazione.** A 20 km la densità è $1{,}22\, e^{-2{,}36} = 0{,}115$ kg/m$^3$, un decimo di quella al suolo: la velocità terminale locale, $\propto \rho_f^{-1/2}$, è circa tre volte quella al livello del mare. Scendendo, la sfera accelera verso una velocità terminale che però *diminuisce* man mano che l'aria si infittisce; quando la velocità raggiunge la terminale locale, da lì in poi la supera e la sfera *decelera*.

Prevediamo dunque un massimo di velocità a quota intermedia. La viscosità dinamica $\mu$ dipende poco dalla quota (trascurando la temperatura), quindi $\nu = \mu/\rho_f$ cresce in quota.

**Svolgimento.** La funzione $F(v)$ diventa $F(v, z)$: i coefficienti $A$, $B$, $C$ e $\nu$ vengono ricalcolati a ogni valutazione con la densità alla quota $y = 20\,000 - z$.

```
def f(t, s):
    z, v = s
    y = y0 - z
    rb = rhof(y) / rho
    A, B, C = 1 + rb / 2, (1 - rb) * G, 3 * rb / (4 * d)
    nu = mu / rhof(y)
    return np.array([v, (B - C * v * abs(v) * cd_sfera(v * d / nu)) / A])
```

esercizi_svolti.py

Si integra con RK4, $h = 0{,}05$ s, arrestandosi quando $z \geq 20\,000$ m.

**Risultato**

La velocità massima è 382 m/s, raggiunta dopo 56,7 s a 7,3 km di quota; al suolo la sfera arriva a 282 m/s dopo 78 s (Fig. B.5). La curva calcolata incrocia quella della velocità terminale locale esattamente nel punto di massimo, come deve essere ($\mathrm{d}v/\mathrm{d}t = 0$ quando $v = v_t$ locale).

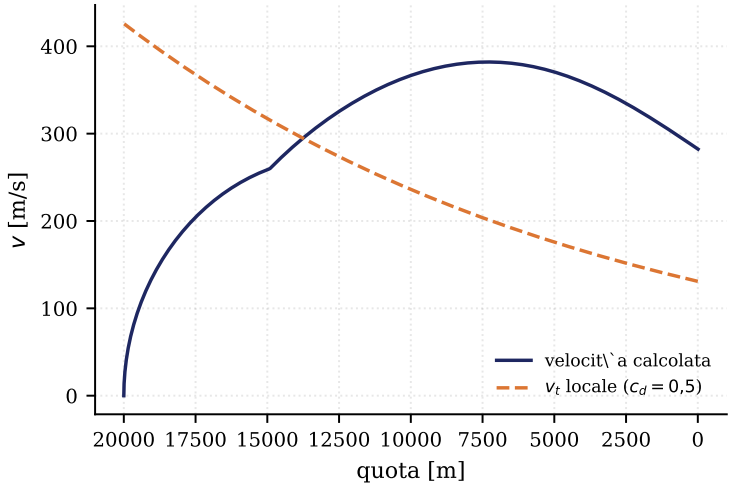



Figura B.5: Esercizio 6: velocità in funzione della quota per una sfera d'acciaio di 10 cm rilasciata da 20 km, confrontata con la velocità terminale locale.

**Commento.** È il comportamento dei corpi in rientro atmosferico e delle meteore: il massimo di velocità (e di decelerazione, poco più in basso) si trova a quote intermedie. Si noti che a 382 m/s la sfera è supersonica: il modello trascura la resistenza d'onda, che ridurrebbe il massimo; vale qui l'avvertenza del Cap. 2 sulla sfera "sonica". Dal punto di vista numerico il programma non è cambiato: è bastato far dipendere $F$ dalla posizione, come anticipato al §2.6.

## B.7 Esercizio 7 – Il guscio pieno d'aria in acqua

**Testo**

Il corpo del Programma 1.2 è sostituito da un guscio sferico sottile pieno d'aria ($d = 1$ cm, $\rho \simeq 1{,}22$ kg/m$^3$) in acqua, vincolato da un'asta rigida, priva di massa e di attrito, lunga $l = 2$ m. Trovare il moto a partire da fermo con 5° di inclinazione dalla posizione inferiore.

**Impostazione.** Con $\bar{\rho} = 820$ si ha $B = (1 - \bar{\rho})g = -8023$ m/s$^2$: il termine $-B \sin\theta$ nella (3.2) cambia segno rispetto al pendolo ordinario, e l'equilibrio stabile è a $\theta = 180°$ (il guscio "pende" verso l'alto). Partendo a 5° dalla posizione inferiore, che ora è instabile, il guscio risale lungo l'arco. Inoltre $A = 411$ e $C = 61\,475$ m$^{-1}$ (quattro volte quello della pallina da ping-pong, perché $d$ è quattro volte minore): la resistenza domina e il moto sarà sovrasmorzato, con velocità limitata dal bilancio fra spinta di Archimede tangenziale e resistenza, $|v| \le \sqrt{-B/(Cc_d)} = 0{,}51$ m/s.

**Svolgimento.**

1. *Passo.* Costante di tempo $\tau = A/(2Cc_d|v|) \simeq 411/(2 \cdot 61\,475 \cdot 0{,}5 \cdot 0{,}5) = 0{,}013$ s; occorre $h < 0{,}037$ s. Con $h = 0{,}1$ s il calcolo diverge (`nan` dopo pochi passi), con $h = 0{,}01$ s è stabile.
2. Si integra la (3.2) con i nuovi coefficienti per 40 s.

**Risultato**

Il guscio supera la quota del perno (90°) a 8,0 s, arriva a 170° a 15,0 s e a 179° a 17,2 s, assestandosi su 180° senza alcuna oscillazione (Fig. B.6). La velocità massima lungo l'arco è 0,511 m/s, esattamente la "velocità terminale" tangenziale stimata.

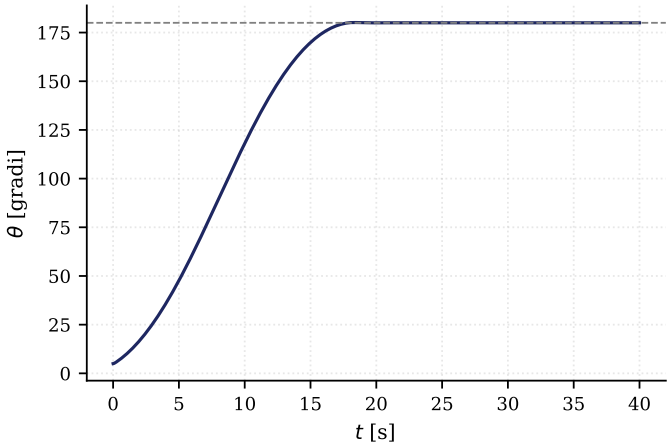



Figura B.6: Esercizio 7: risalita del guscio verso l'equilibrio superiore.

**Commento.** Il moto non è un'oscillazione ma una *sedimentazione al contrario*: il guscio sale alla velocità di equilibrio locale, che decresce come $\sqrt{\sin\theta}$ avvicinandosi alla verticale; per questo gli ultimi gradi richiedono molti secondi (l'avvicinamento a 180° è asintotico). È lo stesso

fenomeno della pallina da ping-pong del Cap. 2, con la differenza che qui la forza motrice si annulla all'equilibrio.

## B.8 Esercizio 8 – La linea elettrica nel vento oscillante

**Testo**

Un filo di rame ($\rho = 8950$ kg/m$^3$, $d = 5$ mm) è appeso come un pendolo di lunghezza $l = 0{,}3$ m, inizialmente fermo nella posizione più bassa, in un vento orizzontale $u = 30(1 + 0{,}2\sin\omega t)$ m/s, con $\omega = 0,\ 0{,}5,\ 1,\ 2$ rad/s; il coefficiente di resistenza del cilindro è $c_d = 1{,}2$. Simulare il moto.

**Impostazione.** Il rame è 7000 volte più denso dell'aria: galleggiamento e massa aggiunta sono trascurabili. La forza di resistenza per unità di lunghezza è $\frac{1}{2}\rho_f d c_d\,|\mathbf{w}_r|\,\mathbf{w}_r$, con $\mathbf{w}_r$ la velocità dell'aria relativa al filo; la sua componente tangenziale (l'unica che conta, l'altra è assorbita dal filo di sospensione) è $\frac{1}{2}\rho_f d c_d\,(u\cos\theta - v)\sqrt{(u\sin\theta)^2 + (u\cos\theta - v)^2}$, con segno dato da $(u\cos\theta - v)$. Dividendo per la massa per unità di lunghezza $\rho\pi d^2/4$:

$$\frac{\mathrm{d}v}{\mathrm{d}t} = -g\sin\theta + k\,(u\cos\theta - v)\sqrt{(u\sin\theta)^2 + (u\cos\theta - v)^2}, \qquad k = \frac{2\rho_f c_d}{\pi\rho d} = 0{,}0208\ \mathrm{m}^{-1}.$$

Per $\omega = 0$ ci aspettiamo oscillazioni smorzate attorno all'equilibrio statico, in cui $v = 0$ e $g\sin\theta = ku^2\cos\theta$: $\tan\theta_{\mathrm{eq}} = ku^2/g = 1{,}913$, $\theta_{\mathrm{eq}} = 62{,}4°$. Con $\omega \neq 0$ il filo oscilla attorno a questa posizione alla frequenza del vento.

**Svolgimento.** La struttura è quella del Programma 1.2 con la nuova $F$; passo $h = 0{,}005$ s (il periodo proprio è $2\pi\sqrt{l/g} = 1{,}1$ s e la costante di tempo dello smorzamento $1/(2ku) \simeq 0{,}8$ s).

```
def f(t, s):
    th, v = s[0] / l, s[1]
    u = 30.0 * (1 + 0.2 * np.sin(om * t))
    ur = u * np.cos(th) - v
    Ft = k * ur * np.hypot(u * np.sin(th), ur)
    return np.array([v, -G * np.sin(th) + Ft])
```

esercizi_svolti.py

**Risultato**

Per $\omega = 0$ il filo, dopo un transitorio di circa 8 s con escursioni iniziali oltre 100°, si ferma a 62,4°, in accordo con l'equilibrio statico. Per $\omega = 0{,}5,\ 1,\ 2$ oscilla attorno a 60–66° con escursioni picco-picco di 13°, 19° e 21° rispettivamente (Fig. B.7): l'ampiezza cresce con $\omega$, perché la pulsazione propria delle piccole oscillazioni attorno all'equilibrio inclinato, $\sqrt{(g\cos\theta_{\mathrm{eq}} + ku^2\sin\theta_{\mathrm{eq}})/l} \simeq 8$ rad/s, è più alta di tutte le frequenze del vento provate: crescendo $\omega$ ci si avvicina alla risonanza.

**Commento.** Il modello è grossolano (un pendolo rigido al posto di una catenaria flessibile, $c_d$ costante, vento uniforme) ma contiene l'essenziale del *galloping* delle linee aeree: un equilibrio inclinato dal vento, uno smorzamento aerodinamico e una forzante periodica. Un'estensione naturale è sostituire la sinusoide con una serie di Fourier che rappresenti la turbolenza atmosferica.

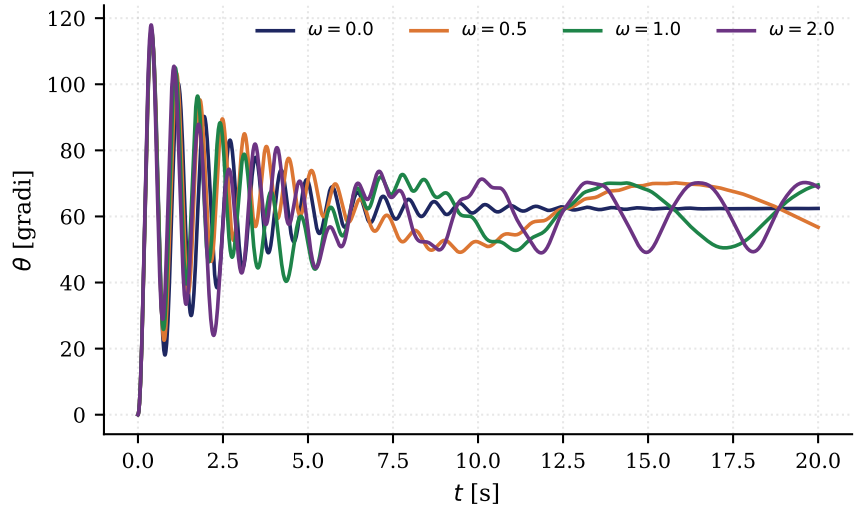



Figura B.7: Esercizio 8: angolo del filo nel vento per quattro valori di $\omega$.

## B.9 Esercizio 9 – Il vento critico dell'ala su molla

**Testo**

Con il Programma 1.3 trovare la velocità $U$ oltre la quale l'ala raggiunge l'equilibrio senza oscillare, e confrontarla con la stima $U = 2/\beta$ dello smorzamento aerodinamico. Controllare che non intervenga lo stallo.

**Impostazione.** L'analisi linearizzata del Cap. 3 dà $\zeta = \beta U/2$, quindi smorzamento critico per $U = 2/\beta = 1093$. La linearizzazione presuppone $V \ll U$ e angoli d'attacco entro lo stallo ($|\alpha| \leq 18°$); con $\alpha_0 = 10°$ lo stallo si raggiunge se $\arctan(V/U)$ supera $8°$ in valore assoluto, cioè se $V > 0{,}14U$: va verificato.

**Svolgimento.** Si integra per $U = 400$, 800, 1000, 1100, 1300 e si misura la *sovraelongazione* $Z_{\max}/Z_{\text{eq}} - 1$ e l'angolo d'attacco minimo raggiunto.

**Risultato**

| $U$ | $Z_{\text{eq}}$ | sovraelongazione [%] | $\alpha_{\min}$ [°] |
|---|---|---|---|
| 400 | 8,133 | 29,3 | 5,43 |
| 800 | 32,533 | 3,5 | 3,46 |
| 1000 | 50,833 | 0,1 | 2,87 |
| 1100 | 61,508 | 0,0 | 2,63 |
| 1300 | 85,908 | 0,0 | 2,24 |

La sovraelongazione si annulla fra $U = 1000$ e 1100 (Fig. B.8), in ottimo accordo con $2/\beta = 1093$. L'angolo d'attacco minimo resta sempre sopra $2°$: niente stallo, la linearizzazione è legittima.

**Commento.** Il confronto fra il calcolo non lineare e la previsione lineare è un *test di validazione* del programma: se non avessero concordato, l'errore sarebbe stato nel codice o nella teoria. Si

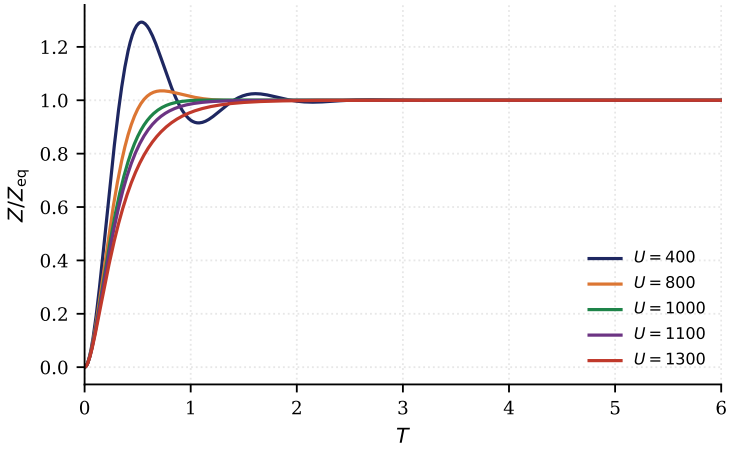



Figura B.8: Esercizio 9: risposta normalizzata $Z/Z_{\text{eq}}$ per venti crescenti; oltre $U \simeq 1100$ non c'è sovraelongazione.

osservi anche che per $U = 1300$ lo spostamento di equilibrio è 86 lunghezze di riferimento, cioè 2,6 m con $l_r = 3$ cm: fisicamente una molla reale sarebbe fuori dal campo lineare, ma il modello non lo sa.

## B.10 Esercizio 10 – La frequenza di massima risposta alle raffiche

**Testo**

Con il Programma 1.3 cercare, per $U = 100$ e $a = 0{,}2$, il valore di $\omega$ che massimizza l'ampiezza a regime dell'oscillazione, per raffica orizzontale e per raffica verticale.

**Impostazione.** In un oscillatore lineare forzato la risposta è massima alla risonanza, $\omega \simeq 1$ (frequenza propria in unità adimensionali). Qui però la forzante non è additiva: la raffica orizzontale modula la *pressione dinamica* (termine in $U^2$), quella verticale modula l'*angolo d'attacco*; inoltre lo smorzamento ($\zeta = 0{,}09$) e la non linearità possono spostare o cancellare il picco. Il Cap. 3 ha anticipato che non si osserva risonanza a $\omega = 1$.

**Svolgimento.** Per $\omega$ da 0,2 a 3 a passi di 0,1 si integra fino a $T = 80$ e si misura l'ampiezza $\frac{1}{2}(Z_{\max} - Z_{\min})$ nella finestra $T > 40$, dove il transitorio è esaurito.

**Risultato**

Raffica orizzontale: l'ampiezza cresce monotonamente con $\omega$ (da 0,05 a 0,28 fra $\omega = 0{,}2$ e 3), senza alcun picco. Raffica verticale: l'ampiezza è massima (6,4) al valore più basso provato, $\omega = 0{,}2$, e decresce con $\omega$, con oscillazioni dovute ai battimenti (Fig. B.9).

**Commento.** Nessuna risonanza: per la raffica orizzontale la risposta è quella "quasi-statica" alla pressione dinamica modulata, e cresce con $\omega$ perché a frequenze alte lo smorzamento aerodinamico ha meno tempo per agire in ogni ciclo; per la raffica verticale la forzante è proporzionale a $aU \sin \omega T$ nell'angolo d'attacco e la risposta decresce con $\omega$ come in un

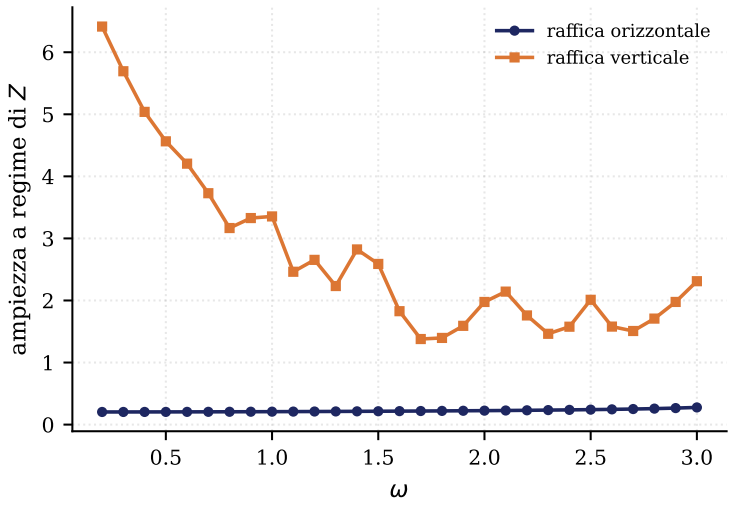



Figura B.9: Esercizio 10: ampiezza a regime in funzione della frequenza della raffica.

sistema passa-basso. Questo è coerente con la pratica: le raffiche pericolose per il carico alare sono quelle lente e di grande ampiezza, non quelle alla frequenza propria.

## B.11 Esercizio 11 – L'effetto Magnus

**Testo**

Aggiungere al Programma 1.4 una forza di portanza $\frac{1}{2}\rho_f w_r^2 \frac{1}{4}\pi d^2 c_l$ normale alla velocità relativa, con $c_l = \pm 0{,}2$ (sfera in rotazione): come cambiano traiettoria e gittata?

**Impostazione.** Una sfera in rotazione trascina lo strato limite in modo asimmetrico e genera una forza normale alla velocità (effetto Magnus); per *backspin* (rotazione all'indietro) la forza è verso l'alto e sostiene il proiettile, per *topspin* verso il basso. In aria ferma $\mathbf{w}_r = -\mathbf{w}$; il versore normale alla velocità, ruotato di $+90°$, è $(-v, u)/w$. La forza per unità di massa, con le stesse costanti del Cap. 4, è $(C c_l w/A)\,(-v,\, u)$, da sommare alle (4.5).

**Svolgimento.**

```python
def f(t, s):
    x, y, u, v = s
    w = np.hypot(u, v)
    cd = cd_sfera(w * d / NU)
    fx = (-C * cd * u * w - C * cl * w * v) / A
    fy = (-B - C * cd * v * w + C * cl * w * u) / A
    return np.array([u, v, fx, fy])
```

esercizi_svolti.py

Sfera d'acciaio $d = 5$ cm, $w_0 = 50$ m/s, $\theta_0 = 45°$, $c_l = 0, +0{,}2, -0{,}2$.

**Risultato**

Gittata 209,3 m senza rotazione, 214,5 m con backspin (+2,5%, quota massima da 57,1 a 60,6 m, volo più lungo), 203,4 m con topspin (−2,8%). La traiettoria con backspin è più "piatta" nel tratto discendente (Fig. B.10).

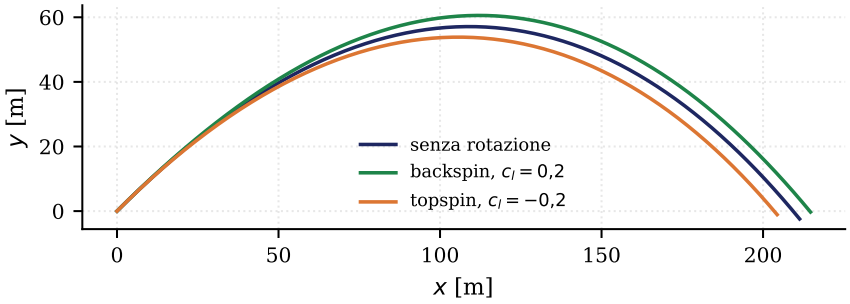



Figura B.10: Esercizio 11: traiettorie con e senza effetto Magnus.

**Commento.** Per una sfera d'acciaio di 5 cm l'effetto è piccolo perché $C \propto \bar{\rho}/d$ è piccolo; per una pallina da golf ($\bar{\rho} \simeq 10^{-3}$, $d = 4{,}3$ cm, $c_l$ fino a 0,3) la portanza è dello stesso ordine del peso e la traiettoria cambia radicalmente: è il motivo per cui il drive di un professionista sale, resta sospeso e cade quasi verticale. Lo studente può ripetere il calcolo con $\rho = 1100$ kg/m$^3$ (densità di una pallina da golf) e $c_l = 0{,}25$.

## B.12 Esercizio 12 – Colpire un bersaglio a distanza assegnata

**Testo**

Scrivere il programma che calcola gli angoli di tiro con cui il proietto del Problema 4.3 (atmosfera esponenziale) colpisce un bersaglio a distanza $x_t$, e provarlo per $x_t = 8000$ e 15 000 m in assenza di vento.

**Impostazione.** La curva $x_r(\theta_0)$ (Fig. B.11) parte da zero, sale fino al massimo di 13,3 km a 42,8° e ridiscende a zero a 90°. Un bersaglio entro la gittata massima è raggiunto da *due* angoli (tiro teso e tiro curvo); un bersaglio oltre non è raggiungibile. L'algoritmo è una ricerca di zeri della funzione $x_r(\theta_0) - x_t$ con il dimezzamento dell'intervallo.

**Svolgimento.**

1. Si parte da $\theta_{\text{old}} = 0$ ($x_r = 0$) con $\delta = 5°$ e si calcola $\theta_{\text{new}} = \theta_{\text{old}} + \delta$ e $x_{\text{new}}$.
2. Si esamina il segno di $[x_t - x_{\text{old}}]\,[x_t - x_{\text{new}}]$: se è positivo i due punti sono dalla stessa parte del bersaglio e $\delta$ resta invariato; se è negativo il bersaglio è stato scavalcato e si pone $\delta \leftarrow -\delta/2$. In entrambi i casi il punto nuovo diventa il vecchio e se ne calcola un altro.
3. Quando $|\delta| < \varepsilon = 0{,}01°$ lo scavalcamento è localizzato: l'angolo cercato è la media degli ultimi due. Si riparte da un punto $\delta$ oltre la soluzione con il $\delta$ iniziale, per cercare la seconda.
4. Se $\theta$ raggiunge 90° senza scavalcamenti, il bersaglio è fuori portata e il programma lo segnala.

```python
while th_new < 90.0:
    if (xt - xr_old) * (xt - xr_new) < 0:      # bersaglio scavalcato
        if abs(d) < eps:                        # soluzione isolata con precisione eps
            soluzioni.append(0.5 * (th_old + th_new))
            d = delta                           # riparte verso destra, oltre la soluzione
            th_old = max(th_old, th_new) + d; xr_old = xr(th_old)
            th_new = th_old + d; xr_new = xr(th_new)
            continue
        d = -d / 2
    th_old, xr_old = th_new, xr_new
    th_new = th_old + d; xr_new = xr(th_new)
```

esercizi_svolti.py

**Risultato**

$x_t = 8$ km: angoli 9,34° e 72,74° (gittate di verifica 7,999 e 8,001 km), con 45 traiettorie integrate. $x_t = 15$ km: fuori portata, segnalato dopo 18 traiettorie.

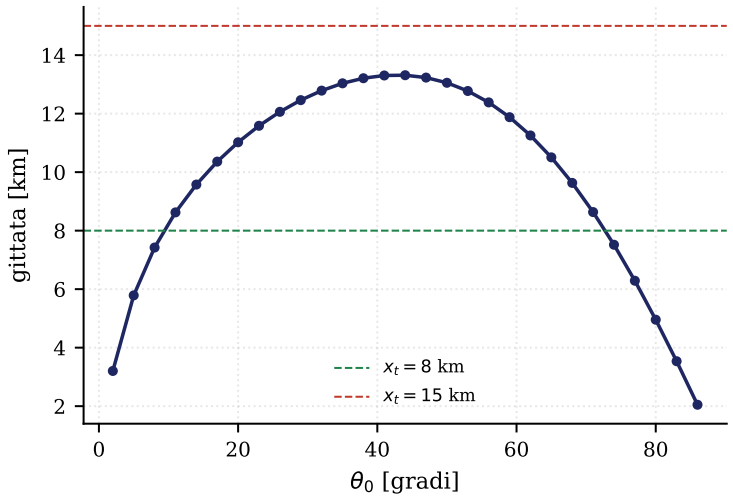



Figura B.11: Esercizio 12: gittata in funzione dell'angolo per il proietto in atmosfera esponenziale; le linee orizzontali sono i bersagli.

**Commento.** La funzione $x_r(\theta_0)$ non è nota in forma chiusa: ogni sua valutazione costa un'intera integrazione numerica. Per questo si usa un metodo a sole valutazioni di funzione (niente derivate) e si tiene conto del numero di valutazioni. Una variante più efficiente è il metodo della secante (*regula falsi*) una volta localizzato l'intervallo; ma la robustezza del dimezzamento, che non può fallire, è spesso più preziosa della velocità. Il tiro teso (9°) ha tempo di volo minore ed è meno sensibile al vento: in artiglieria si preferisce, salvo ostacoli.

## B.13 Esercizio 13 – Aliante con richiamata

**Testo**

Nel Programma 1.6 far dipendere $A$ dal tempo, simulando una "richiamata" (aumento di incidenza) con $A$ che cresce da 1,5 a 2,5 per $T \in [2, 3]$, e osservare la risposta a partire dalla picchiata verticale.

**Impostazione.** $A$ è proporzionale a $c_l$: aumentarlo equivale a tirare la barra. Un $A$ maggiore produce una curvatura maggiore della traiettoria (il raggio del moto circolare ideale è $\propto 1/A$), quindi ci aspettiamo che la picchiata venga arrestata prima e a quota più alta; ma più portanza significa anche più resistenza indotta (qui $B$ è fisso, quindi $D \propto A$) e una perdita di velocità più rapida.

**Svolgimento.** La funzione $f(T, \mathbf{s})$ riceve $A(T) = 1{,}5 + \min(\max(T - 2, 0), 1)$; si integra con $h = 0{,}05$ per $T = 10$ e si confronta con $A$ costante.

**Risultato**

Con $A$ costante la picchiata arriva a $Y_{\min} = -0{,}947$; con la richiamata a $Y_{\min} = -0{,}824$: la perdita di quota si riduce del 13%. La velocità minima nel successivo stallo scende però da 0,077 a 0,049 e a $T = 10$ l'aliante richiamato è leggermente *più basso* ($-0{,}596$ contro $-0{,}549$): l'energia spesa nella richiamata non torna indietro (Fig. B.12).

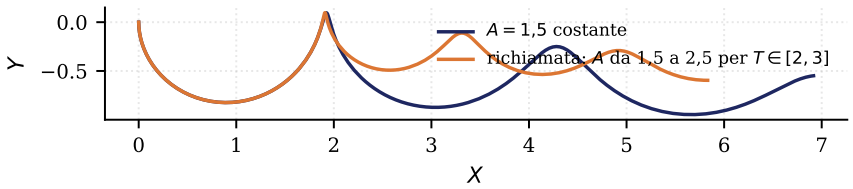



Figura B.12: Esercizio 13: traiettoria con $A$ costante e con richiamata fra $T = 2$ e 3.

**Commento.** Il programma a incidenza variabile è già un simulatore di volo elementare a tre gradi di libertà, con il pilota rappresentato dalla funzione $A(T)$. La prossima estensione naturale è rendere $A$ e $B$ funzioni dell'angolo d'attacco attraverso la polare del velivolo, $C_D = C_{D,0} + C_L^2/(\pi\, AR\, e)$, cosicché la richiamata costi anche in resistenza indotta.

## B.14 Esercizio 14 – Il razzo e il nodo allineato allo spegnimento

**Testo**

Ripetere il Problema 5.1 con passi $h$ per cui l'istante di spegnimento $t = 4$ s coincide o no con un nodo della griglia temporale, e confrontare l'istante d'impatto con un riferimento accurato.

**Impostazione.** La spinta è una funzione *discontinua* del tempo: $\dot{m}_p v_e$ fino a $t = 4$ s, zero dopo. I metodi di Runge–Kutta presuppongono $f$ regolare entro ogni passo; se un passo scavalca

la discontinuità, la spinta viene applicata (RK2 valuta $f$ all'inizio e a metà del passo) anche per la frazione del passo in cui dovrebbe essere spenta. Il razzo riceve un impulso in eccesso dell'ordine di $\dot{m}_p v_e \cdot \Delta t$ con $\Delta t < h$: con spinta 9184 N e massa $\simeq 50$ kg, un eccesso di 0,05 s vale $\simeq 9$ m/s, che si ripercuote sulla quota massima e sul tempo di volo. Se invece un nodo cade esattamente in $t = 4$, ogni passo è interamente "acceso" o "spento" e il problema non si pone.

**Svolgimento.** Riferimento: `solve_ivp` RK45 con tolleranze $10^{-9}$ e passo massimo 0,05 s. Per $h = 0{,}1,\ 0{,}09,\ 0{,}07,\ 0{,}05,\ 0{,}03$ s si integra con RK2 e si interpola linearmente l'istante d'impatto fra gli ultimi due nodi.

**Risultato**

| $h$ [s] | nodo in 4 s | $t_{\text{imp}}$ [s] | errore [s] | $z_{\max}$ [m] |
|---|---|---|---|---|
| 0,10 | sì | 57,8314 | $-0{,}0019$ | 3726,39 |
| 0,09 | no | 57,1966 | $-0{,}6367$ | 3646,98 |
| 0,07 | no | 57,6720 | $-0{,}1613$ | 3706,51 |
| 0,05 | sì | 57,8316 | $-0{,}0017$ | 3726,55 |
| 0,03 | no | 57,6749 | $-0{,}1584$ | 3706,78 |

Riferimento: $t_{\text{imp}} = 57{,}833$ s, $z_{\max} = 3726{,}7$ m. Con i nodi allineati l'errore è di 2 millesimi di secondo qualunque sia $h$; con i nodi non allineati l'errore sale a 0,16–0,64 s e $z_{\max}$ sbaglia fino a 80 m. Il passo $h = 0{,}03$ (non allineato) è *peggiore* di $h = 0{,}1$ (allineato).

**Commento.** È uno degli errori più comuni e più insidiosi nella simulazione di sistemi con eventi (accensioni, spegnimenti, sgancio di stadi, apertura di paracadute): l'errore non diminuisce riducendo il passo in modo generico, perché dipende dalla *posizione* della discontinuità rispetto alla griglia, non dalla sua ampiezza. I rimedi sono: scegliere $h$ che divida gli istanti di evento (come qui), oppure fermare l'integrazione sull'evento e ripartire (è ciò che fa `solve_ivp` con `events`), oppure spezzare l'integrazione in tratti $[0, 4]$ e $[4, t_{\text{imp}}]$.

## B.15 Esercizio 15 – Vortex blob

**Testo**

Nel Programma 1.7 sostituire $R^2$ con $R^2 + \delta^2$ ($\delta = 0{,}05$) e confrontare le spirali a $T = 0{,}35$ con e senza regolarizzazione.

**Impostazione.** La velocità indotta da un vortice puntiforme diverge come $1/R$: due vortici che si avvicinano si fanno ruotare l'uno attorno all'altro sempre più in fretta, e la spirale, dove i vortici si addensano, diventa caotica. Sostituendo $R^2$ con $R^2 + \delta^2$ il nucleo di ciascun vortice diventa un "blob" di raggio $\delta$ entro cui la velocità è limitata ($\leq G/(2\delta)$): a distanze $\gg \delta$ nulla cambia, a distanze $\lesssim \delta$ le interazioni sono smussate. Ci aspettiamo la stessa forma globale del foglio e una spirale regolare al posto di quella contorta.

**Svolgimento.** Basta una riga in `velocita`: `R2 = DX**2 + DY**2 + delta**2`. La simmetrizzazione e l'integrazione restano identiche.

**Risultato**

A $T = 0{,}35$ il punto medio del foglio è sceso a $Y = -0{,}856$ con $\delta = 0$ e a $-0{,}841$ con $\delta = 0{,}05$ (2% di differenza): la parte piana del foglio è insensibile alla regolarizzazione. La spirale invece cambia aspetto (Fig. B.13): senza regolarizzazione i vortici d'estremità sono disposti in modo irregolare, con il blob descrivono una spirale ordinata.

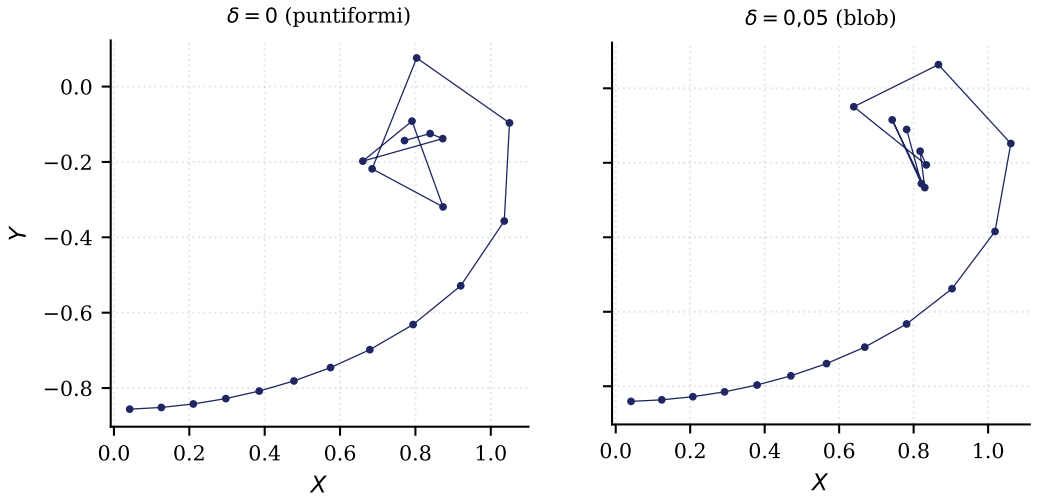



Figura B.13: Esercizio 15: foglio vorticoso a $T = 0{,}35$ con vortici puntiformi e con vortex blob ($\delta = 0{,}05$).

**Commento.** Il parametro $\delta$ non ha un valore "giusto": rappresenta la scala al di sotto della quale si rinuncia a risolvere la struttura del foglio. Krasny [16] ha mostrato che, per $\delta \to 0$ con $m \to \infty$ opportunamente, la soluzione regolarizzata converge a quella del foglio continuo fino all'istante in cui questo sviluppa una singolarità. È lo stesso spirito dei modelli di sottogriglia in turbolenza: non si risolve tutto, ma si controlla che cosa si trascura.

## B.16 Esercizio 16 – Convergenza in $m$

**Testo**

Ripetere il Programma 1.7 con $m = 20, 40, 80, 160$ (e $\mathrm{d}T = 1/(25m)$) e confrontare la posizione dei vortici a $T = 0{,}1$.

**Impostazione.** Raffinare la discretizzazione dovrebbe far convergere la forma del foglio nelle regioni dove il foglio è liscio (la parte centrale), mentre nella spirale, dove la soluzione continua stessa è singolare, non c'è motivo di attendersi convergenza vortice per vortice. Un buon "termometro" è il *baricentro di vorticità* di mezzo foglio, $\bar{Y} = \sum G_i Y_i / \sum G_i$: la sua velocità dipende solo dalla circolazione e dal baricentro dell'altra metà, entrambi conservati, quindi deve risultare quasi indipendente da $m$.

**Svolgimento.** Il programma è parametrizzato in $m$; il costo cresce come $m^2$ per passo e $m$ per il numero di passi ($m^3$ in totale): con $m = 160$ servono 400 passi con matrici $160 \times 160$, ancora rapidi grazie a NumPy.

**Risultato**

| $m$ | $\mathrm{d}T$ | $Y$ centro | $X$ estremità | $Y$ estremità | $\bar{Y}$ |
|---|---|---|---|---|---|
| 20 | 0,002 00 | −0,2948 | 0,8695 | 0,0159 | −0,0671 |
| 40 | 0,001 00 | −0,2985 | 0,8697 | 0,0186 | −0,0666 |
| 80 | 0,000 50 | −0,3005 | 0,8589 | −0,0175 | −0,0665 |
| 160 | 0,000 25 | −0,3015 | 0,8889 | 0,0269 | −0,0664 |

La posizione del centro converge regolarmente (differenze successive 3,7, 2,0, $1{,}0 \cdot 10^{-3}$: dimezzano raddoppiando $m$, convergenza del primo ordine in $1/m$); il baricentro è costante alla quarta cifra; il vortice d'estremità invece *non converge*: la sua posizione dipende dalla discretizzazione, perché rappresenta una porzione di spirale la cui struttura cambia con $m$.

**Commento.** È un esempio di convergenza *in senso debole*: le quantità integrali (posizione media, baricentro, e con esse il campo lontano e quindi il downwash sull'ala) convergono, i dettagli locali no. In un calcolo pratico si deve sempre dichiarare *quale* grandezza si vuole e verificarne la convergenza, non "la soluzione" in astratto.

## B.17 Esercizio 17 – Vortici di uguale intensità

**Testo**

Sostituire il foglio con 40 vortici di uguale intensità, posizionati in modo che ogni intervallo contenga la stessa circolazione; disegnare la configurazione iniziale e confrontare le forme del foglio con quelle della discretizzazione equispaziata.

**Impostazione.** La circolazione cumulata dall'estremità sinistra $-a$ fino a $x$ è $\Gamma(x)/\Gamma_0 = \sqrt{1-X^2}$ (con $X = x/a$). Se la metà sinistra deve contenere 20 vortici di intensità $1/20$, il $k$-esimo intervallo va da $\sqrt{1-X^2} = (k-1)/20$ a $k/20$, e il suo vortice si colloca al punto medio in circolazione, $\sqrt{1-X_k^2} = (k-\tfrac{1}{2})/20$, cioè

$$X_k = -\sqrt{1-\left(\tfrac{k-1/2}{20}\right)^2}, \qquad k = 1, \dots, 20.$$

I vortici si addensano verso l'estremità, dove $\gamma$ è singolare: il primo è a $X = -0{,}9997$, il ventesimo a $-0{,}222$.

**Svolgimento.** Si costruiscono le ascisse come sopra, si assegna $G_i = \pm 1/20$, e si integra con la stessa funzione `velocita` (che accetta un array $G$ qualsiasi) e lo stesso $\mathrm{d}T$; si confrontano le due discretizzazioni a $T = 0{,}05$, 0,1, 0,2.

**Risultato**

Le due discretizzazioni danno forme del foglio sovrapponibili nella parte piana e nella testa della spirale (Fig. B.14); il punto medio del foglio scende a $Y = -0{,}153/-0{,}147$ ($T = 0{,}05$), $-0{,}299/-0{,}284$ ($T = 0{,}1$), $-0{,}554/-0{,}521$ ($T = 0{,}2$) nelle due versioni (equispaziati/uguale intensità): differenze del 5–6%, dovute al fatto che con vortici di uguale intensità la parte centrale è rappresentata da pochi vortici distanti (solo 2 nei $|X| < 0{,}3$ centrali).

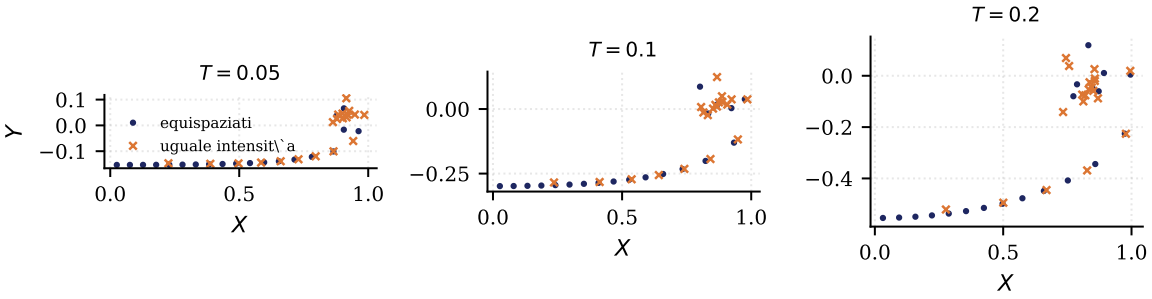



Figura B.14: Esercizio 17: foglio discretizzato con vortici equispaziati (punti) e di uguale intensità (croci) a tre istanti.

**Commento.** Le due discretizzazioni distribuiscono diversamente la risoluzione: quella equispaziata è accurata al centro e povera all'estremità, dove la circolazione per vortice è massima; quella a intensità uguale concentra i vortici dove serve per la spirale e li dirada al centro. L'accordo complessivo conferma che il risultato non dipende dai dettagli della discretizzazione, il che è la prima condizione perché un calcolo numerico sia credibile. Una scelta ancora migliore, usata nei codici a vortici moderni, è ridistribuire periodicamente i vortici lungo il foglio deformato (*remeshing*).

## B.18 Listato completo

esercizi_svolti.py

```python
"""
esercizi_svolti.py -- codice degli esercizi svolti dell'Appendice B.
Ogni esercizio e' una funzione es_NN(); eseguendo il file si svolgono tutti,
stampando i risultati e salvando le figure es_NN*.pdf nella cartella figure/.
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from ode_solvers import integra, passo_rk4, cd_sfera, METODI
from stile_grafici import salva, PALETTE, BLU, ARANCIO, VERDE, VIOLA, ROSSO, GRIGIO

G = 9.8


def titolo(n, testo):
    print(f"\n{'=' * 70}\nEsercizio {n}: {testo}\n{'=' * 70}")


# ============================================================================ 1
def es_01():
```

```
    titolo(1, "metodi di Heun e Ralston")
    f = lambda t, x: x - t ** 2 + 1.0
    esatta = lambda t: (t + 1) ** 2 - 0.5 * np.exp(t)
    passi = [0.4, 0.2, 0.1, 0.05, 0.025]
    print(f"{'h':>6} {'punto medio':>12} {'Heun':>12} {'Ralston':>12}")
    err = {m: [] for m in ("rk2", "heun", "ralston")}
    for h in passi:
        n = int(round(2 / h))
        for m in err:
            t, X = integra(f, 0.0, 0.5, h, n, metodo=m)
            err[m].append(abs(X[-1, 0] - esatta(2.0)))
        print(f"{h:6.3f} {err['rk2'][-1]:12.3e} {err['heun'][-1]:12.3e}
    {err['ralston'][-1]:12.3e}")
    for m in err:
        e = np.array(err[m])
        print(f"  {m:11s}: ordine osservato {np.log2(e[:-1]/e[1:]).round(2)},  K =
    E/h^2 a h=0.025: {e[-1]/0.025**2:.3f}")
    fig, ax = plt.subplots(figsize=(4.2, 3.0))
    for m, lab, c in (("rk2", "punto medio", BLU), ("heun", "Heun", ARANCIO),
    ("ralston", "Ralston", VERDE)):
        ax.loglog(passi, err[m], "o-", color=c, label=lab)
    ax.set_xlabel("$h$"); ax.set_ylabel("errore globale in $t=2$"); ax.legend()
    salva(fig, "es_01")


# ============================================================================ 2
def es_02():
    titolo(2, "stabilita' di RK4 su dx/dt = -x")
    R = lambda z: 1 + z + z**2/2 + z**3/6 + z**4/24
    fig, ax = plt.subplots(figsize=(4.6, 3.0))
    for hl, c in ((2.5, VERDE), (2.785, ARANCIO), (3.0, ROSSO)):
        n = int(30 / hl)
        t, X = integra(lambda t, x: -x, 0.0, 1.0, hl, n, metodo="rk4")
        print(f"  h*lambda = {hl:5.3f}: R = {R(-hl):+.4f}, |R|^N =
    {abs(R(-hl))**n:.3e}, x_N = {X[-1,0]:+.3e} (N = {n})")
        ax.plot(t, X[:, 0], "o-", ms=3, color=c, label=f"$h\\lambda={hl}$")
    tt = np.linspace(0, 30, 300)
    ax.plot(tt, np.exp(-tt), "k-", lw=0.8, label="esatta")
    ax.set_ylim(-3, 3); ax.set_xlabel("$t$"); ax.set_ylabel("$x$");
    ax.legend(ncol=2, fontsize=7)
    salva(fig, "es_02")


# ============================================================================ 3
def es_03():
    titolo(3, "errore di arrotondamento di RK4")
    f = lambda t, x: x - t ** 2 + 1.0
    esatta = (2 + 1) ** 2 - 0.5 * np.exp(2.0)
    print(f"{'h':>8} {'N':>8} {'errore':>12}")
    hh, ee = [], []
    for h in (1e-1, 1e-2, 1e-3, 1e-4, 1e-5):
        n = int(round(2 / h))
        t, X = integra(f, 0.0, 0.5, h, n, metodo="rk4")
        e = abs(X[-1, 0] - esatta); hh.append(h); ee.append(e)
        print(f"{h:8.0e} {n:8d} {e:12.3e}")
    fig, ax = plt.subplots(figsize=(4.2, 3.0))
    ax.loglog(hh, ee, "o-", color=BLU, label="errore misurato")
    ax.loglog(hh, 0.03 * np.array(hh) ** 4, ":", color=GRIGIO, label="$\\propto
    h^4$")
```

```
    ax.loglog(hh, 1e-16 * 2 / np.array(hh), "--", color=ARANCIO, label="$\\propto \\epsilon_m N$")
    ax.set_xlabel("$h$"); ax.set_ylabel("errore globale"); ax.legend(fontsize=7)
    salva(fig, "es_03")


# ============================================================================ 4
def es_04():
    titolo(4, "sfera d'acciaio in acqua")
    from prog1_1_caduta_sfera import caduta, costanti
    d, rho, rhof, nu = 0.01, 8000.0, 1000.0, 1e-6
    A, B, C = costanti(d, rho, rhof)
    print(f"  rho_bar = {rhof/rho}, A = {A:.4f}, B = {B:.3f} m/s^2, C = {C:.3f} 1/m")
    # stima della velocita' terminale (ipotesi c_d = 0.5) e della costante di tempo
    vt = np.sqrt(B / (C * 0.5)); Re = vt * d / nu; tau = A / (2 * C * 0.5 * vt)
    print(f"  stima: v_t = {vt:.3f} m/s, Re = {Re:.0f} (c_d = 0.5 coerente), tau = {tau:.4f} s -> h <= {2.785*tau:.3f} s")
    for h in (0.1, 0.02, 0.005):
        T, Z, V, RE = caduta(d, h=h, tmax=2.0, rho=rho, rhof=rhof, nu=nu)
        print(f"  h = {h:5.3f}: v(2 s) = {V[-1]:.4f} m/s, z(2 s) = {Z[-1]:.4f} m, Re = {RE[-1]:.0f}")
    T, Z, V, RE = caduta(d, h=0.005, tmax=2.0, rho=rho, rhof=rhof, nu=nu)
    Ta, Za, Va, REa = caduta(d, h=0.1, tmax=2.0)
    fig, ax = plt.subplots(figsize=(4.4, 3.0))
    ax.plot(T, V, color=BLU, label="in acqua"); ax.plot(Ta, Va, color=ARANCIO, label="in aria")
    ax.axhline(vt, color="gray", ls="--", lw=0.8)
    ax.set_xlabel("$t$ [s]"); ax.set_ylabel("$v$ [m/s]"); ax.legend()
    salva(fig, "es_04")


# ============================================================================ 5
def es_05():
    titolo(5, "goccia che gela")
    from prog1_1_caduta_sfera import caduta
    d_w, rho_w, rho_i = 0.2e-3, 1000.0, 917.0
    d_i = d_w * (rho_w / rho_i) ** (1 / 3)
    for nome, d, rho in (("acqua", d_w, rho_w), ("ghiaccio", d_i, rho_i)):
        T, Z, V, RE = caduta(d, h=0.002, tmax=1.0, rho=rho)
        print(f"  {nome:9s}: d = {d*1e3:.4f} mm, v_t = {V[-1]:.4f} m/s, Re = {RE[-1]:.2f}, "
              f"formula di Stokes (fuori validita', Re>1): {(1-1.22/rho)*G*d**2/(18*1.22/rho*1.49e-5):.4f} m/s")


# ============================================================================ 6
def es_06():
    titolo(6, "caduta da 20 km in atmosfera stratificata")
    d, rho, mu, y0 = 0.10, 8000.0, 1.818e-5, 20000.0
    rhof = lambda y: 1.22 * np.exp(-0.000118 * y)

    def f(t, s):
        z, v = s
        y = y0 - z
        rb = rhof(y) / rho
        A, B, C = 1 + rb / 2, (1 - rb) * G, 3 * rb / (4 * d)
        nu = mu / rhof(y)
        return np.array([v, (B - C * v * abs(v) * cd_sfera(v * d / nu)) / A])

```

```
    T, S = integra(f, 0.0, [0.0, 0.0], 0.05, 4000, stop=lambda t, s: s[0] >= y0)
    z, v = S[:, 0], S[:, 1]
    i = np.argmax(v)
    print(f"  v_max = {v[i]:.1f} m/s a t = {T[i]:.1f} s, quota {y0 - z[i]:.0f} m; "
          f"v al suolo = {v[-1]:.1f} m/s, t di caduta = {T[-1]:.1f} s")
    # velocita' terminale locale (c_d = 0.18 oltre Re=2e6, altrimenti 0.5) per confronto
    yy = np.linspace(0, y0, 200)
    vt_loc = [np.sqrt((1 - rhof(y)/rho) * G / (3 * rhof(y)/rho / (4*d) * 0.5)) for y in yy]
    fig, ax = plt.subplots(figsize=(4.6, 3.0))
    ax.plot(y0 - z, v, color=BLU, label="velocit\\'a calcolata")
    ax.plot(yy, vt_loc, "--", color=ARANCIO, label="$v_t$ locale ($c_d=0{,}5$)")
    ax.set_xlabel("quota [m]"); ax.set_ylabel("$v$ [m/s]"); ax.invert_xaxis(); ax.legend(fontsize=7)
    salva(fig, "es_06")


# ============================================================================ 7
def es_07():
    titolo(7, "guscio pieno d'aria in acqua (pendolo invertito)")
    d, rho, rhof, nu, l = 0.01, 1.22, 1000.0, 1e-6, 2.0
    rb = rhof / rho
    A, B, C = 1 + rb / 2, (1 - rb) * G, 3 * rb / (4 * d)
    print(f"  A = {A:.1f}, B = {B:.0f} m/s^2, C = {C:.0f} 1/m")

    def f(t, s):
        y, v = s
        return np.array([v, (-B * np.sin(y / l) - C * v * abs(v) * cd_sfera(v * d / nu)) / A])

    y0 = l * np.radians(5.0)
    tau = A / (2 * C * 0.5 * 0.5)                    # stima con |v| ~ 0.5 m/s
    print(f"  costante di tempo stimata tau = {tau:.4f} s -> h < 2.785 tau = {2.785*tau:.3f} s")
    for h in (0.1, 0.01):
        with np.errstate(all="ignore"):
            T, S = integra(f, 0.0, [y0, 0.0], h, int(40 / h))
        th = np.degrees(S[:, 0] / l)
        print(f"  h = {h}: theta(40 s) = {th[-1]:.2f} deg, |v| max = {abs(S[:,1]).max():.3f} m/s")
    T, S = integra(f, 0.0, [y0, 0.0], 0.01, 4000)
    th = np.degrees(S[:, 0] / l)
    for soglia in (90, 170, 179):
        i = np.where(th >= soglia)[0]
        print(f"  theta = {soglia} gradi raggiunto a t = {T[i[0]]:.2f} s" if len(i) else f"  theta = {soglia} non raggiunto in 40 s")
    print(f"  equilibrio (theta=180) stabile? B<0 -> si; velocita' massima lungo l'arco stimata con eq. (v_t): "
          f"{np.sqrt(-B/(C*0.5)):.3f} m/s")
    fig, ax = plt.subplots(figsize=(4.6, 3.0))
    ax.plot(T, th, color=BLU); ax.axhline(180, color="gray", ls="--", lw=0.8)
    ax.set_xlabel("$t$ [s]"); ax.set_ylabel("$\\theta$ [gradi]")
    salva(fig, "es_07")


# ============================================================================ 8
def es_08():
    titolo(8, "linea elettrica (cilindro) nel vento oscillante")
    rho, rhof, d, l, cd = 8950.0, 1.22, 0.005, 0.3, 1.2
```

```
    k = 2 * rhof * cd / (np.pi * rho * d)          # coefficiente della forza tangenziale per unita' di massa
    print(f"  k = 2 rho_f c_d/(pi rho d) = {k:.5f} 1/m")

    def modello(om):
        def f(t, s):
            th, v = s[0] / l, s[1]
            u = 30.0 * (1 + 0.2 * np.sin(om * t))
            ur = u * np.cos(th) - v
            Ft = k * ur * np.hypot(u * np.sin(th), ur)
            return np.array([v, -G * np.sin(th) + Ft])
        return f

    fig, ax = plt.subplots(figsize=(5.2, 3.0))
    for om, c in zip((0.0, 0.5, 1.0, 2.0), PALETTE):
        T, S = integra(modello(om), 0.0, [0.0, 0.0], 0.005, 4000)
        th = np.degrees(S[:, 0] / l)
        ax.plot(T, th, color=c, label=f"$\\omega={om}$")
        print(f"  omega = {om}: theta medio (ultimi 5 s) = {th[T>15].mean():.2f} deg, "
              f"escursione = {th[T>15].max()-th[T>15].min():.2f} deg")
    # equilibrio statico per omega = 0 (v = 0): g sin(theta) = k u cos(theta) * u -> tan(theta) = k u^2 / g
    th_eq = np.arctan(k * 30.0 ** 2 / G)
    print(f"  equilibrio statico (omega=0): tan(theta_eq) = k u^2/g = {k*900/G:.3f} -> theta_eq = {np.degrees(th_eq):.2f} deg")
    ax.set_xlabel("$t$ [s]"); ax.set_ylabel("$\\theta$ [gradi]"); ax.legend(ncol=4, fontsize=7)
    salva(fig, "es_08")


# ============================================================================ 9
def es_09():
    titolo(9, "vento critico per l'ala su molla")
    from prog1_3_ala_molla import sistema, beta, cl, alpha0
    print(f"  stima lineare: U_crit = 2/beta = {2/beta:.0f}")
    h, n = 0.01, 2000
    fig, ax = plt.subplots(figsize=(5.0, 3.0))
    for U, c in zip((400, 800, 1000, 1100, 1300), PALETTE):
        T, S = integra(sistema(lambda T, U=U: U), 0.0, [0.0, 0.0], h, n)
        Zeq = beta * cl(alpha0) * U ** 2 / (2 * np.pi) ** 2
        over = S[:, 0].max() / Zeq - 1
        amin = np.degrees(alpha0 - np.arctan2(S[:, 1], U)).min()
        ax.plot(T, S[:, 0] / Zeq, color=c, label=f"$U={U}$")
        print(f"  U = {U:5d}: Z_eq = {Zeq:7.3f}, sovraelongazione = {100*over:5.1f} %, alpha min = {amin:5.2f} deg")
    ax.set_xlabel("$T$"); ax.set_ylabel("$Z/Z_{\\rm eq}$"); ax.set_xlim(0, 6); ax.legend(fontsize=7)
    salva(fig, "es_09")


# ============================================================================ 10
def es_10():
    titolo(10, "ricerca della frequenza di massima risposta")
    from prog1_3_ala_molla import sistema
    U0, a, h = 100.0, 0.2, 0.01
    omegas = np.linspace(0.2, 3.0, 29)
    amp_h, amp_v = [], []
    for om in omegas:
```

```
        T, S = integra(sistema(lambda T, om=om: U0 * (1 + a * np.sin(om * T))), 0.0, [0, 0], h, 8000)
        Z = S[T > 40, 0]; amp_h.append(0.5 * (Z.max() - Z.min()))      # ampiezza a regime (T > 40)
        T, S = integra(sistema(lambda T: U0, lambda T, om=om: -a * U0 * np.sin(om * T)), 0.0, [0, 0], h, 8000)
        Z = S[T > 40, 0]; amp_v.append(0.5 * (Z.max() - Z.min()))
    amp_h, amp_v = np.array(amp_h), np.array(amp_v)
    print(f"  raffica orizzontale: ampiezza max {amp_h.max():.3f} a omega = {omegas[amp_h.argmax()]:.2f}")
    print(f"  raffica verticale  : ampiezza max {amp_v.max():.3f} a omega = {omegas[amp_v.argmax()]:.2f}")
    fig, ax = plt.subplots(figsize=(4.6, 3.0))
    ax.plot(omegas, amp_h, "o-", ms=3, color=BLU, label="raffica orizzontale")
    ax.plot(omegas, amp_v, "s-", ms=3, color=ARANCIO, label="raffica verticale")
    ax.set_xlabel("$\\omega$"); ax.set_ylabel("ampiezza a regime di $Z$"); ax.legend(fontsize=7)
    salva(fig, "es_10")


# ============================================================================ 11
def es_11():
    titolo(11, "effetto Magnus su un proiettile sferico")
    from prog1_4_proiettile import traiettoria, gittata, RHO, RHOF, NU
    d, w0 = 0.05, 50.0
    rb = RHOF / RHO
    A, B, C = 1 + rb / 2, (1 - rb) * G, 3 * rb / (4 * d)

    def modello_magnus(cl):
        def f(t, s):
            x, y, u, v = s
            w = np.hypot(u, v)                          # aria ferma: w_rel = w
            cd = cd_sfera(w * d / NU)
            fx = (-C * cd * u * w - C * cl * w * v) / A
            fy = (-B - C * cd * v * w + C * cl * w * u) / A
            return np.array([u, v, fx, fy])
        return f

    fig, ax = plt.subplots(figsize=(5.4, 3.0))
    for cl, c, lab in ((0.0, BLU, "senza rotazione"), (0.2, VERDE, "backspin, $c_l=0{,}2$"), (-0.2, ARANCIO, "topspin, $c_l=-0{,}2$")):
        S, sP, sQ = traiettoria(modello_magnus(cl), w0, 45.0)
        ax.plot(S[:, 1], S[:, 2], color=c, label=lab)
        print(f"  c_l = {cl:+.1f}: gittata = {gittata(sP, sQ):7.2f} m, quota max = {S[:,2].max():6.2f} m, t_volo ~ {S[-1,0]:.1f} s")
    ax.set_xlabel("$x$ [m]"); ax.set_ylabel("$y$ [m]"); ax.set_aspect("equal"); ax.legend(fontsize=7)
    salva(fig, "es_11")


# ============================================================================ 12
def es_12():
    titolo(12, "angoli di tiro per colpire un bersaglio")
    from prog1_8_cannone import modello_cannone, w0
    from prog1_4_proiettile import traiettoria, gittata
    f = modello_cannone(lambda y: 1.22 * np.exp(-0.000118 * y))
    xr = lambda th: gittata(*traiettoria(f, w0, th, 0.05)[1:])

    def angoli_bersaglio(xt, delta=5.0, eps=0.01):
        soluzioni, n = [], 0
```

```
        th_old, xr_old = 0.0, 0.0            # a theta = 0 la gittata e' nulla
        th_new = th_old + delta; xr_new = xr(th_new); n += 1
        d = delta
        while th_new < 90.0:
            if (xt - xr_old) * (xt - xr_new) < 0:  # bersaglio scavalcato
                if abs(d) < eps:                    # soluzione isolata con precisione eps
                    soluzioni.append(0.5 * (th_old + th_new))
                    d = delta                       # riparte verso destra, oltre la soluzione
                    th_old = max(th_old, th_new) + d; xr_old = xr(th_old); n += 1
                    th_new = th_old + d; xr_new = xr(th_new); n += 1
                    continue
                d = -d / 2
            th_old, xr_old = th_new, xr_new
            th_new = th_old + d; xr_new = xr(th_new); n += 1
        return soluzioni, n

    for xt in (8000.0, 15000.0):
        sol, n = angoli_bersaglio(xt)
        if sol:
            print(f"  x_t = {xt/1e3:.0f} km: angoli = {', '.join(f'{s:.2f}' for s in sol)} deg "
                  f"(verifica gittate: {', '.join(f'{xr(s)/1e3:.3f}' for s in sol)} km), {n} traiettorie")
        else:
            print(f"  x_t = {xt/1e3:.0f} km: FUORI PORTATA (nessun angolo entro 90 gradi), {n} traiettorie")
    thetas = np.arange(2, 89, 3)
    xrs = [xr(th) / 1e3 for th in thetas]
    fig, ax = plt.subplots(figsize=(4.6, 3.0))
    ax.plot(thetas, xrs, "o-", ms=3, color=BLU)
    for xt, c in ((8.0, VERDE), (15.0, ROSSO)):
        ax.axhline(xt, color=c, ls="--", lw=0.8, label=f"$x_t={xt:.0f}$ km")
    ax.set_xlabel("$\\theta_0$ [gradi]"); ax.set_ylabel("gittata [km]"); ax.legend(fontsize=7)
    salva(fig, "es_12")


# ============================================================================ 13
def es_13():
    titolo(13, "aliante con richiamata (A variabile)")
    B, W0, h = 0.06, 1.0, 0.05

    def aliante(A_di_T):
        def f(T, s):
            X, Y, U, V = s
            A = A_di_T(T); W = np.hypot(U, V)
            return np.array([U, V, -A * W * (B * U + V), -1.0 + A * W * (U - B * V)])
        return f

    fig, ax = plt.subplots(figsize=(5.4, 3.0))
    for lab, Af, c in (("$A=1{,}5$ costante", lambda T: 1.5, BLU),
                       ("richiamata: $A$ da 1,5 a 2,5 per $T\\in[2,3]$", lambda T: 1.5 + np.clip(T - 2.0, 0, 1), ARANCIO)):
        T, S = integra(aliante(Af), 0.0, [0, 0, 0.0, -W0], h, 200)
        ax.plot(S[:, 0], S[:, 1], color=c, label=lab)
        W = np.hypot(S[:, 2], S[:, 3])
```

```
        print(f"  {lab}: Y min = {S[:,1].min():.3f}, Y a T=10: {S[-1,1]:.3f}, W min = {W.min():.3f}")
    ax.set_xlabel("$X$"); ax.set_ylabel("$Y$"); ax.set_aspect("equal"); ax.legend(fontsize=7)
    salva(fig, "es_13")


# ============================================================================ 14
def es_14():
    titolo(14, "razzo: nodo allineato allo spegnimento")
    from prog1_9_razzo import f
    ev = lambda t, s: s[0]; ev.terminal, ev.direction = True, -1
    rif = solve_ivp(f, [0.5, 200], integra(f, 0.0, [0, 0], 0.1, 5, metodo="rk2")[1][-1], method="RK45",
                    events=ev, max_step=0.05, rtol=1e-9, atol=1e-11)
    t_rif, v_rif = rif.t_events[0][0], rif.y_events[0][0][1]
    print(f"  riferimento (RK45, tolleranze strette): impatto t = {t_rif:.3f} s, v = {v_rif:.3f} m/s")
    for h in (0.1, 0.09, 0.07, 0.05, 0.03):
        T, S = integra(f, 0.0, [0, 0], h, 20000, metodo="rk2", stop=lambda t, s: s[0] < 0 and t > 1)
        # interpolazione lineare dell'istante di impatto
        t_imp = T[-2] + (T[-1] - T[-2]) * S[-2, 0] / (S[-2, 0] - S[-1, 0])
        allineato = "si" if abs(round(4 / h) - 4 / h) < 1e-6 else "no"
        print(f"  h = {h:4.2f} (nodo a 4 s: {allineato:2s}): impatto t = {t_imp:.4f} s, "
              f"errore = {t_imp - t_rif:+.4f} s, z_max = {S[:,0].max():.2f} m")


# ============================================================================ 15
def es_15():
    titolo(15, "vortex blob (regolarizzazione di Krasny)")
    import prog1_7_scia_vorticosa as sv
    m, dT = sv.m, sv.dT

    def velocita_blob(delta):
        def vel(T, s):
            X, Y = s[:m], s[m:]
            DX = X[:, None] - X[None, :]; DY = Y[:, None] - Y[None, :]
            R2 = DX ** 2 + DY ** 2 + delta ** 2
            np.fill_diagonal(R2, np.inf)
            return np.concatenate([(sv.G[None, :] * DY / R2).sum(axis=1), -(sv.G[None, :] * DX / R2).sum(axis=1)])
        return vel

    fig, axs = plt.subplots(1, 2, figsize=(6.4, 2.8), sharey=True)
    for ax, delta, lab in zip(axs, (0.0, 0.05), ("$\\delta = 0$ (puntiformi)", "$\\delta = 0{,}05$ (blob)")):
        s = np.concatenate([sv.X0, np.zeros(m)]); T = 0.0
        while T < 0.35 - 1e-12:
            s = sv.simmetrizza(passo_rk4(velocita_blob(delta), T, s, dT)); T += dT
        ax.plot(s[m//2:m], s[m + m//2:], ".-", color=BLU, ms=4, lw=0.5)
        ax.set_title(lab, fontsize=8); ax.set_xlabel("$X$"); ax.set_aspect("equal")
        print(f"  delta = {delta}: Y centro = {s[m + m//2 - 1]:+.4f}, estremita' (X,Y) = ({s[2*m-1 - m]:.3f}, {s[2*m-1]:+.3f})")
    axs[0].set_ylabel("$Y$")
    salva(fig, "es_15")


# ============================================================================ 16
```

```
def es_16():
    titolo(16, "convergenza in m a T = 0.1")
    radice = lambda x: np.sqrt(np.clip(1 - x ** 2, 0, None))
    print(f"{'m':>4} {'dT':>8} {'Y centro':>10} {'X estr.':>9} {'Y estr.':>9} {'Y
    medio':>9}")
    for m in (20, 40, 80, 160):
        dX = 1.0 / m
        X0 = -1 + dX + 2 * dX * np.arange(m)
        Gm = radice(X0 + dX) - radice(X0 - dX)
        dT = 1.0 / (25 * m)

        def vel(T, s):
            X, Y = s[:m], s[m:]
            DX = X[:, None] - X[None, :]; DY = Y[:, None] - Y[None, :]
            R2 = DX ** 2 + DY ** 2; np.fill_diagonal(R2, np.inf)
            return np.concatenate([(Gm[None, :] * DY / R2).sum(axis=1), -(Gm[None,
    :] * DX / R2).sum(axis=1)])

        s = np.concatenate([X0, np.zeros(m)]); T = 0.0
        n = int(round(0.1 / dT))
        for _ in range(n):
            s = passo_rk4(vel, T, s, dT); T += dT
        # baricentro della meta' destra (pesato con |G|): invariante del moto
        Xr, Yr, Gr = s[m//2:m], s[m + m//2:], Gm[m//2:]
        print(f"{m:4d} {dT:8.5f} {s[m + m//2 - 1]:+10.4f} {s[m-1]:9.4f}
    {s[2*m-1]:+9.4f} {np.sum(Gr*Yr)/np.sum(Gr):+9.4f}")


# ============================================================================ 17
def es_17():
    titolo(17, "vortici di uguale intensita'")
    import prog1_7_scia_vorticosa as sv
    m, dT = 40, 1.0 / (25 * 40)
    # meta' sinistra: 20 vortici con G = 1/20 ciascuno, in X_k tali che
    Gamma(X)/Gamma_0 = 1 - (k - 1/2)/20
    k = np.arange(1, m // 2 + 1)
    Gam = 1 - (k - 0.5) / (m / 2)                    # circolazione cumulata
    dall'estremita'... sqrt(1-X^2)
    Xl = -np.sqrt(1 - Gam ** 2)                       # X_k < 0, ordinati
    dall'estremita' verso il centro
    Xl = np.sort(Xl)
    X0 = np.concatenate([Xl, -Xl[::-1]])
    Gq = np.concatenate([np.full(m // 2, 1 / (m / 2)), np.full(m // 2, -1 / (m /
    2))])
    print("  ascisse iniziali (meta' sinistra):", np.round(Xl, 4))

    def vel(T, s):
        X, Y = s[:m], s[m:]
        DX = X[:, None] - X[None, :]; DY = Y[:, None] - Y[None, :]
        R2 = DX ** 2 + DY ** 2; np.fill_diagonal(R2, np.inf)
        return np.concatenate([(Gq[None, :] * DY / R2).sum(axis=1), -(Gq[None, :] *
    DX / R2).sum(axis=1)])

    istanti = (0.05, 0.1, 0.2)
    fig, axs = plt.subplots(1, 3, figsize=(6.6, 2.6))
    s_eq = np.concatenate([X0, np.zeros(m)]); s_un = np.concatenate([sv.X0,
    np.zeros(m)]); T = 0.0; k = 0
    while k < len(istanti):
        s_eq = sv.simmetrizza(passo_rk4(vel, T, s_eq, dT)); s_un =
    sv.simmetrizza(passo_rk4(sv.velocita, T, s_un, dT)); T += dT
```

```
        if T >= istanti[k] - 1e-12:
            ax = axs[k]
            ax.plot(s_un[m//2:m], s_un[m + m//2:], ".", color=BLU, ms=3,
    label="equispaziati")
            ax.plot(s_eq[m//2:m], s_eq[m + m//2:], "x", color=ARANCIO, ms=3,
    label="uguale intensit\\`a")
            ax.set_title(f"$T={istanti[k]}$", fontsize=8); ax.set_aspect("equal");
    ax.set_xlabel("$X$")
            print(f"  T = {istanti[k]}: Y centro equisp. = {s_un[m + m//2 -
    1]:+.4f}, ug. int. = {s_eq[m + m//2 - 1]:+.4f}")
            k += 1
    axs[0].set_ylabel("$Y$"); axs[0].legend(fontsize=6, loc="lower left")
    fig.tight_layout()
    salva(fig, "es_17")


if __name__ == "__main__":
    for fn in (es_01, es_02, es_03, es_04, es_05, es_06, es_07, es_08, es_09, es_10,
               es_11, es_12, es_13, es_14, es_15, es_16, es_17):
        fn()
```

Listato B.1: `esercizi_svolti.py`: codice di tutti gli esercizi.

# Bibliografia


[1] S. Biringen, C.-Y. Chow, *An Introduction to Computational Fluid Mechanics by Example*, Wiley, Hoboken, 2011. Capitolo 1: *Flow Topics Governed by Ordinary Differential Equations: Initial-Value Problems*, pp. 1–49. Programmi MATLAB su `www.wiley.com/go/biringen`.

[2] C.-Y. Chow, *An Introduction to Computational Fluid Mechanics*, Wiley, New York, 1979 (edizione originale, con programmi FORTRAN).

[3] S. C. Chapra, R. P. Canale, *Numerical Methods for Engineers*, 7ª ed., McGraw-Hill, New York, 2015. Parte 7: *Ordinary Differential Equations* (Cap. 25, metodi di Runge–Kutta e passo adattivo; Cap. 26, rigidità e metodi multistep).

[4] J. H. Ferziger, M. Perić, *Computational Methods for Fluid Dynamics*, 3ª ed., Springer, Berlin, 2002. Cap. 6: *Methods for Unsteady Problems*.

[5] R. J. LeVeque, *Finite Volume Methods for Hyperbolic Problems*, Cambridge University Press, 2002. Cap. 8: accuratezza, consistenza, stabilità e convergenza.

[6] B. Andersson, R. Andersson, L. Håkansson, M. Mortensen, R. Sudiyo, B. van Wachem, *Computational Fluid Dynamics for Engineers*, Cambridge University Press, 2012.

[7] G. Caramia, E. Distaso, *A Practical Approach to Computational Fluid Dynamics Using OpenFOAM®*, Springer, Cham, 2025. Cap. 1: concetti preliminari di calcolo numerico.

[8] A. M. Kuethe, C.-Y. Chow, *Foundations of Aerodynamics: Bases of Aerodynamic Design*, 5ª ed., Wiley, New York, 1998.

[9] H. Lamb, *Hydrodynamics*, 6ª ed., Cambridge University Press, 1932 (massa aggiunta della sfera, §92).

[10] S. Goldstein (a cura di), *Modern Developments in Fluid Dynamics*, Oxford University Press, 1938 (curva $c_d(Re)$ della sfera, p. 16).

[11] R. Gunn, G. D. Kinzer, *The terminal velocity of fall for water droplets in stagnant air*, Journal of Meteorology **6** (1949) 243–248.

[12] J. Happel, H. Brenner, *Low Reynolds Number Hydrodynamics*, Prentice-Hall, 1965.

[13] F. W. Lanchester, *Aerodonetics*, Constable, London, 1908 (moto fugoide).

[14] T. Theodorsen, *General theory of aerodynamic instability and the mechanism of flutter*, NACA Report 496, 1935.

[15] D. W. Moore, *A numerical study of the roll-up of a finite vortex sheet*, Journal of Fluid Mechanics **63** (1974) 225–235.

[16] R. Krasny, *Desingularization of periodic vortex sheet roll-up*, Journal of Computational Physics **65** (1986) 292–313.

[17] E. Hairer, S. P. Nørsett, G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2ª ed., Springer, Berlin, 1993 (teoria generale dei metodi di Runge–Kutta e tableau di Butcher).

[18] OpenStax, *Introduction to Python Programming*, Rice University, 2024 (introduzione al linguaggio usata nel modulo propedeutico del corso).

[19] P. Virtanen *et al.*, *SciPy 1.0: fundamental algorithms for scientific computing in Python*, Nature Methods **17** (2020) 261–272 (documentazione di `scipy.integrate.solve_ivp`).